

all ms project

02/23/2026 21:33:29

(Excellent mindset 👍 You are thinking like a future senior engineer. Now I will give...

Thought process: Architected comprehensive C# fundamentals curriculum starting...

Continue

Thought process: Recognized request to advance toward nullable types lesson. The...

Continue nothing miss

Thought process: Orchestrated comprehensive OOP encapsulation lesson delivery....

Cover this stage 1 content explain in not much longer so make proepr

Thought process: Orchestrated comprehensive Stage 1 curriculum coverage...

Start stage 2 in proper guide in depth and proper simple good way explain don't...

Thought process: Orchestrated comprehensive ASP.NET Core curriculum spanning...

Start stage 4 proper same but much better good way

Thought process: Architected comprehensive Entity Framework Core learning guide...

Do it stage 3 in simple proepr short but cover all

Thought process: Strategized comprehensive SQL Server Database coverage...

Gives stage 5 proper in deep cover proper good way in simple terms proepr

Thought process: Architected comprehensive security stage covering authenticatio...

Continue stage 6 with proepr and much good examples with good approach i In clou...

Thought process: Architected beginner-friendly Azure explanation with enterprise-...

Anything remain gives

Thought process: Inventoried completed stages and prepared remaining curriculu...

So gives as for react not much depth but proper cover all

Thought process: Architected accessible React guide balancing breadth with...

Yes cover theory in good way proepr cover all topic past with one proper guide well...

Thought process: Architected comprehensive theory guide spanning all prior topics...

👤 (Excellent mindset 👍 You are thinking like a future senior engineer.

Now I will give you Industry-Level Complete Backend Roadmap (Microsoft Ecosystem Focused) — From absolute basics → enterprise → production level.

I will not skip anything important.

This is structured like real industry expectations.

🧱 STAGE 1 — Programming Foundation (Mandatory Base)

Before Azure, DevOps, or Architecture... you must be strong in programming.

1 C# Deep Core Knowledge

Learn properly:

◆ Data Types

Value types vs Reference types

Memory (Stack vs Heap)

Nullable types

◆ OOP Principles

Encapsulation

Abstraction

Inheritance

Polymorphism

◆ Advanced OOP

Interfaces

Abstract classes

Sealed classes

Virtual/override

Method overloading/overriding

◆ Important Concepts

Delegates

Events

LINQ

Async / Await

Tasks

Exception Handling

Garbage Collection

👉 Industry expects you to explain these confidently.

📦 STAGE 2 — .NET & Web Development Core

Now move to:

2 ASP.NET Core (Industry Standard Backend)

Understand full request lifecycle.

◆ MVC Pattern

Model

View

Controller

◆ Web API

REST principles

HTTP verbs (GET, POST, PUT, DELETE)

Status codes

Model validation

DTO pattern

◆ Middleware

How request flows

Custom middleware

◆ Dependency Injection

Why DI is used

Service lifetime (Transient / Scoped / Singleton)

◆ Configuration

appsettings.json

Environment variables

Secrets management

STAGE 3 — Database Knowledge (Very Important)

Industry backend engineer MUST know DB.

SQL Server (Most common in Microsoft projects)

Learn:

Basic Queries

SELECT

WHERE

JOIN

GROUP BY

ORDER BY

Advanced

Indexes

Execution plan

Stored procedures

Transactions

Normalization

ACID properties

STAGE 4 — ORM & Data Access

Learn:

Entity Framework Core

DbContext

Migrations

Code First

Lazy vs Eager loading

Performance tuning

Tracking vs No Tracking

STAGE 5 — Authentication & Security (Critical in Industry)

Industry expects secure apps.

Learn:

◆ JWT Authentication

Token structure

Claims

Expiry

Refresh tokens

◆ OAuth 2.0

◆ Azure AD / Entra ID

Most Microsoft projects use:

Microsoft Entra ID (formerly Azure Active Directory)

Understand:

App registration

Client ID / Secret

Role-based access control

STAGE 6 — Azure Cloud (Enterprise Level)

Now move to cloud.

Microsoft projects heavily use:

Microsoft Azure

Learn step-by-step:

Core Services

Resource Group

App Service

Azure SQL

Storage Account

Key Vault

Azure Functions

Monitoring

Application Insights

Azure Monitor

STAGE 7 — DevOps & CI/CD

Industry requirement.

Learn:

Azure DevOps

Understand:

Git branching strategy

Pull Requests

Build pipeline

Release pipeline

YAML pipeline

Environment promotion (Dev → QA → Prod)

STAGE 8 — Clean Architecture (Senior-Level Skill)

Industry expects scalable design.

Understand:

Controller layer

Service layer

Repository pattern

Dependency inversion

SOLID principles

Separation of concerns

STAGE 9 — System Design Basics

Even as junior, you must know basics:

What is Load Balancer?

What is Caching?

What is Scalability?

Vertical vs Horizontal scaling

Microservices vs Monolith

API Gateway concept

STAGE 10 — Production-Level Knowledge

This is what makes you different.

Learn:

Logging best practices

Exception handling global middleware

Rate limiting

API versioning

Performance optimization

Memory leak detection

Security best practices

What Industry Actually Tests

In real projects they check:

✅ Can you debug production issue? ✅ Can you explain architecture? ✅ Can you deploy application? ✅ Do you understand authentication flow? ✅ Do you write clean code?

Final Structured Order (Do in This Sequence)

1. C# fundamentals
2. OOP deep
3. ASP.NET Core
4. SQL
5. EF Core
6. Authentication
7. Azure
8. DevOps
9. Architecture

10. System Design

⚡ Real Advice For You (Important)

Since you are beginner in Microsoft outsourcing:

👉 Focus heavily on Azure 👉 Understand enterprise authentication 👉 Learn deployment 👉 Read project code daily

If you say YES, I will start teaching you Stage 1: C# Deep Fundamentals like a classroom — one concept at a time, with examples, interview questions, and project-level thinking.

Shall we start from absolute basics?) cover one by one but actual as begginer guide and deep proper with simple and actual working also gives and begginer normal simple guide . But one by depth .nothing miss remaining. Start proper format

20/02/2026, 00:49:12

Thought process: Architected comprehensive C# fundamentals curriculum starting with data types.

The user wants me to start teaching Stage 1: C# Deep Fundamentals, one concept at a time, beginner-friendly but deep, with actual working examples. Let me start with the very first concept - Data Types (Value types vs Reference types).

Stage 1 — C# Deep Fundamentals

Lesson 1: Data Types — Value Types vs Reference Types

First, What Is a Variable?

Think of a variable like a **box** that holds some data.

```
int age = 25;           // box named "age" holding 25
string name = "Ali";    // box named "name" holding "Ali"
```

Now the big question — **where does this box live in memory?**

This is where Value Types and Reference Types come in.

Memory Basics (Stack vs Heap)

Before understanding types, you must understand memory.

Your program uses **two memory areas**:

STACK	HEAP
Fast memory	Larger memory
Auto cleaned	Managed by GC
Stores VALUES	Stores OBJECTS

Stack → Small, fast, automatically managed. Like a pile of plates.

Heap → Large, slower, managed by Garbage Collector. Like a warehouse.

✓ Value Types

Value types **store their actual value directly in the Stack.**

What are value types?

```
int           // whole numbers
double        // decimal numbers
float         // decimal (less precise)
bool          // true / false
char          // single character
struct        // custom value type
enum          // named constants
```

Working Example:

```
using System;

class Program
{
    static void Main()
    {
        int a = 10;
        int b = a;    // b gets a COPY of a

        b = 99;       // changing b does NOT affect a

        Console.WriteLine($"a = {a}"); // a = 10
        Console.WriteLine($"b = {b}"); // b = 99
    }
}
```

```
}  
}
```

Output:

```
a = 10  
b = 99
```

Why? Because `b = a` made a **complete copy**. They are two separate boxes in memory.

STACK:

a	10
b	99

✅ Reference Types

Reference types **store the memory address (reference) in Stack**, but **actual data lives in Heap**.

What are reference types?

```
string    // text  
class     // any class you create  
array     // arrays  
object    // base of everything  
List<T>   // collections
```

Working Example:

```
using System;

class Person
{
    public string Name { get; set; }
}

class Program
{
    static void Main()
    {
        Person person1 = new Person();
        person1.Name = "Ali";

        Person person2 = person1; // person2 points to SAME
object

        person2.Name = "Ahmed"; // changing person2 ALSO
changes person1!

        Console.WriteLine(person1.Name); // Ahmed
        Console.WriteLine(person2.Name); // Ahmed
    }
}
```

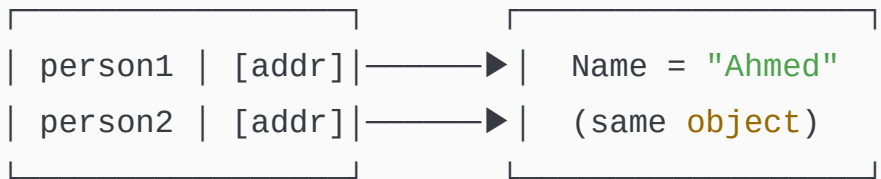
Output:

```
Ahmed
Ahmed
```

Why? Because `person2 = person1` did **NOT copy the object**. It copied the **address** pointing to the same object in Heap.

STACK:

HEAP:



! Common Beginner Mistake

```
// BEGINNER ASSUMES this creates independent copy – WRONG!
Person p1 = new Person { Name = "Ali" };
Person p2 = p1; // both point to same object

p2.Name = "Changed";

Console.WriteLine(p1.Name); // "Changed" – SURPRISE!
```

This is one of the **most common bugs** in real projects.

Fix — Create a proper copy:

```
Person p2 = new Person { Name = p1.Name }; // new independent
object
```

📌 String — Special Case (Important!)

String is a reference type **but behaves like value type** because it is **immutable** (cannot be changed after creation).


```
string s1 = "Hello";
string s2 = s1;

s2 = "World"; // creates NEW string object, does NOT change s1

Console.WriteLine(s1); // Hello
Console.WriteLine(s2); // World
```

Every time you "modify" a string, **a new string object is created in Heap.**

Quick Reference Table

Feature	Value Type	Reference Type
Stored in	Stack	Heap (address in Stack)
Assignment copies	Actual value	Memory address
Default value	0 / false / etc	null
Examples	int, bool, struct	class, string, array
Independent after copy?	 Yes	 No (same object)

Real Project Usage

In a real ASP.NET project you will see this:

```
// This is a Reference Type (class)
public class UserDto
{
    public int Id { get; set; } // Value type inside
```

```
public string Name { get; set; } // Reference type inside
public bool IsActive { get; set; } // Value type inside
}

// When you do this in your service layer:
var user1 = new UserDto { Id = 1, Name = "Ali" };
var user2 = user1; // DANGER – same object reference

user2.Name = "Ahmed"; // Accidentally modifies user1 too!
```

Senior developers always think about this when passing objects between layers.

Interview Questions on This Topic

Q1: What is the difference between value type and reference type?

Value types store data directly in stack. Reference types store address in stack, actual data in heap.

Q2: If I assign one class object to another variable, do I get a copy?

No. Both variables point to the same object in heap. Modifying one affects the other.

Q3: Why is string immutable?

For thread safety and memory optimization. .NET caches/reuses identical strings (string interning).

Q4: What is the default value of a reference type?

`null` — meaning it points to nothing.



Practice Exercise

Try this yourself and predict output BEFORE running:

```
using System;

class Box
{
    public int Value { get; set; }
}

class Program
{
    static void Main()
    {
        // Part 1
        int x = 5;
        int y = x;
        y = 100;
        Console.WriteLine($"x={x}, y={y}");

        // Part 2
        Box box1 = new Box { Value = 5 };
        Box box2 = box1;
        box2.Value = 100;
        Console.WriteLine($"box1={box1.Value}, box2=
{box2.Value}");
    }
}
```

Predict the output first, then run it. Understanding **why** is more important than the answer.



✓ Lesson 1 Summary

- Stack = fast, value lives here directly
- Heap = large, objects live here
- Value types = copy on assignment (independent)
- Reference types = share address (same object)
- String = reference type but acts immutable
- This concept is the **foundation of debugging** in real projects

Next Lesson → **Nullable Types** (What happens when a value type needs to hold `null`?
Very common in DB scenarios.)

Reply **"next"** and we go straight into it. 🚀

20/02/2026, 00:49:48

20/02/2026, 00:49:55

Thought process: Recognized request to advance toward nullable types lesson.

The user wants to continue to the next lesson on Nullable Types.

Stage 1 — C# Deep Fundamentals

Lesson 2: Nullable Types

The Problem First

Remember — Value types **cannot be null** by default.

```
int age = null; // ❌ ERROR – won't even compile
bool isActive = null; // ❌ ERROR
```

But in **real projects**, null is everywhere.

Think about this real scenario:

Database has a column → DateOfBirth
Some users didn't enter their birthday
So the column value is NULL in database

Now when you fetch that value into C#... what do you store it in?

A regular `int` or `DateTime` cannot hold null.

This is exactly why **Nullable Types** exist.

✓ What is a Nullable Type?

A nullable type is a **value type that is allowed to hold null**.

Syntax — Two ways (both are identical):

```
// Way 1 – using ?
int? age = null;

// Way 2 – full syntax (same thing internally)
Nullable<int> age = null;
```

The `?` after the type is just **shorthand**. Both are exactly the same.

✓ Working Examples

Basic Usage:

```
using System;

class Program
{
    static void Main()
    {
        int? age = null;    // perfectly valid
        int normalAge = 25; // cannot be null
    }
}
```

```
    Console.WriteLine(age);           // prints nothing (empty)
    Console.WriteLine(normalAge); // 25

    age = 30; // you can also assign a real value later
    Console.WriteLine(age); // 30
}
}
```

Two Important Properties

Every nullable type has two built-in properties you MUST know:

```
int? score = 85;

// 1. HasValue – tells you if it has a real value or is null
Console.WriteLine(score.HasValue); // True

// 2. Value – gives you the actual value (only use after checking
// HasValue)
Console.WriteLine(score.Value);    // 85
```

What if you call .Value on null?

```
int? score = null;
Console.WriteLine(score.Value); // 💥 CRASH –
// InvalidOperationException
```

This is a **very common bug in real projects**. Always check `HasValue` first.

Safe approach:

```
int? score = null;

if (score.HasValue)
{
    Console.WriteLine($"Score is: {score.Value}");
}
else
{
    Console.WriteLine("Score not provided");
}
```

Null Coalescing Operator — ??

This is **used constantly** in real projects. It means:

"If left side is null, use the right side instead"

```
int? score = null;

int finalScore = score ?? 0; // if score is null, use 0

Console.WriteLine(finalScore); // 0
```

Another example:

```
int? userAge = null;

Console.WriteLine(userAge ?? 18); // prints 18 (default)

userAge = 25;
```



```
Console.WriteLine(userAge ?? 18); // prints 25 (actual value)
```

Real project usage:

```
// User may or may not have provided their phone number
string? phone = GetPhoneFromDatabase(); // could be null

string displayPhone = phone ?? "Not Provided";

Console.WriteLine(displayPhone);
```

Null Coalescing Assignment — **??=**

A newer shorthand (C# 8+):

```
int? count = null;

count ??= 0; // if count is null, ASSIGN 0 to it

Console.WriteLine(count); // 0
```

Same as writing:

```
if (count == null)
{
    count = 0;
}
```

⚡ Null Conditional Operator — ?.

This is **used everywhere** in real code. It means:

"Only access this property/method if the object is NOT null"

```
using System;

class User
{
    public string Name { get; set; }
}

class Program
{
    static void Main()
    {
        User user = null;

        // WITHOUT null conditional – CRASH
        // Console.WriteLine(user.Name); // ✨
        NullReferenceException

        // WITH null conditional – SAFE
        Console.WriteLine(user?.Name); // prints nothing, no
        crash

        // Combining with ??
        Console.WriteLine(user?.Name ?? "Guest"); // prints
        "Guest"
    }
}
```

Real project example:

```
// Order might not have a customer attached
string customerName = order?.Customer?.Name ?? "Unknown";
```

Notice you can **chain** `?.` — this is extremely common in enterprise code.

Real Project Scenario — Database + Nullable

This is exactly what you will write in a real ASP.NET project:

```
using System;

// This represents a database row
public class EmployeeRecord
{
    public int Id { get; set; }
    public string Name { get; set; }
    public int? Age { get; set; } // nullable – might
    not be in DB
    public DateTime? JoiningDate { get; set; } // nullable –
    might be null
    public decimal? Salary { get; set; } // nullable –
    confidential/missing
}

class Program
{
    static void Main()
    {
        // Simulating data fetched from database
        var employee = new EmployeeRecord
        {
            Id = 101,
```

```

        Name = "Ali",
        Age = null,           // not filled in DB
        JoiningDate = DateTime.Now,
        Salary = null        // confidential
    };

    // Safe display logic
    Console.WriteLine($"ID      : {employee.Id}");
    Console.WriteLine($"Name    : {employee.Name}");
    Console.WriteLine($"Age     : {employee.Age ?? 0} (0 means
not provided)");
    Console.WriteLine($"Joined  :
{employee.JoiningDate?.ToString("dd MMM yyyy") ?? "Unknown"}");
    Console.WriteLine($"Salary : {(employee.Salary.HasValue ?
employee.Salary.Value.ToString("C") : "Confidential")}");
    }
}

```

Output:

```

ID      : 101
Name    : Ali
Age     : 0 (0 means not provided)
Joined  : 20 Feb 2026
Salary  : Confidential

```

This is **real enterprise-level thinking** — handle nulls properly before displaying.



Converting Between Nullable and Normal



```

int? nullableAge = 25;

// Nullable → Normal (be careful!)
int normalAge = (int)nullableAge;           // explicit cast –
crashes if null
int safeAge    = nullableAge ?? 0;           // safe way using ??
int safeAge2   = nullableAge.GetValueOrDefault(); // returns 0
if null
int safeAge3   = nullableAge.GetValueOrDefault(18); // returns
18 if null

// Normal → Nullable (always safe)
int regularNumber = 42;
int? nowNullable  = regularNumber; // no problem

```

`GetValueOrDefault()` is very clean and commonly used in real projects.

C# 8+ — Nullable Reference Types (Important Concept)

From C# 8 onwards, you can also make **reference types** explicitly nullable:

```

// In .csproj or at top of file, nullable reference types are
enabled
// <Nullable>enable</Nullable>

string name = null;    // ⚠ Warning – non-nullable reference
assigned null
string? name = null;   // ✅ Correct – explicitly saying it can be
null

```

You will see this in **every modern ASP.NET project**:

```
public class ProductDto
{
    public int Id { get; set; }
    public string Name { get; set; } // must have value
    public string? Description { get; set; } // optional, can be
    null
}
```

The `?` on reference types is a **warning system** that helps you avoid `NullReferenceExceptions` before they happen in production.

Interview Questions

Q1: Can a value type be null in C#?

Not by default. You need to use `int?` (`Nullable<int>`) to allow null.

Q2: What is the difference between `??` and `?.`?

`??` provides a default value when something is null. `?.` safely accesses a member only when the object is not null, returning null instead of crashing.

Q3: What happens if you access `.Value` on a null nullable type?

It throws `InvalidOperationException`. Always check `.HasValue` first or use `??` / `GetValueOrDefault()`.

Q4: Where do you use nullable types in real projects?

Database columns that allow NULL, optional form fields, API responses with optional fields, any data that may legitimately be absent.



Practice Exercise

Predict the output before running:

```
using System;

class Program
{
    static void Main()
    {
        int? a = null;
        int? b = 10;

        Console.WriteLine(a ?? 99);
        Console.WriteLine(b ?? 99);
        Console.WriteLine(a.HasValue);
        Console.WriteLine(b.HasValue);
        Console.WriteLine(b.GetValueOrDefault(50));

        string? text = null;
        Console.WriteLine(text?.ToUpper() ?? "EMPTY");

        text = "hello";
        Console.WriteLine(text?.ToUpper() ?? "EMPTY");
    }
}
```

Write your predicted output, then verify by running it.



Lesson 2 Summary



Concept	Usage
<code>int?</code>	Allow value type to hold null
<code>.HasValue</code>	Check if nullable has a real value
<code>.Value</code>	Get the value (only after checking HasValue)
<code>.GetValueOrDefault()</code>	Safe way to get value or fallback
<code>??</code>	Provide default when null
<code>??=</code>	Assign default if currently null
<code>?.</code>	Safe member access — returns null instead of crashing



Where We Are

- ✓ Lesson 1 — Value Types vs Reference [Types](#) (Stack & Heap)
- ✓ Lesson 2 — Nullable Types
- ▶▶ Lesson 3 — OOP: Encapsulation (next)

Reply **"next"** and we go into **OOP — starting with Encapsulation**, with real-world examples and industry-level thinking. 🚀

20/02/2026, 00:50:45

20/02/2026, 00:51:03

Thought process: Orchestrated comprehensive OOP encapsulation lesson delivery.

The user wants to continue with Lesson 3 - OOP Encapsulation. Let me provide a comprehensive, deep guide.

Stage 1 — C# Deep Fundamentals

Lesson 3: OOP — Encapsulation

What is OOP First?

Before encapsulation, understand **why OOP exists**.

In the early days, code was written like this:

```
// Procedural style – everything open, no structure
string userName = "Ali";
int userAge = 25;
bool userIsActive = true;

// Anyone can change anything from anywhere
userAge = -999;           // no restriction!
userIsActive = true; // no validation!
```

This becomes a **nightmare** in large projects because:

- Any part of the code can accidentally break data
- No rules, no validation, no protection
- Impossible to maintain with 10+ developers

OOP solves this by organizing code into **objects** that control their own data.

OOP has **4 pillars**:

1. Encapsulation ← We learn today
2. Abstraction ← Next lesson
3. Inheritance ← After that
4. Polymorphism ← After that

✓ What is Encapsulation?

Encapsulation means:

"Bundle data and the methods that operate on that data together, and control who can access or modify that data."

Simple version:

"Hide your internal data. Expose only what is necessary."

Real world analogy:

Think of an ATM machine.

You can: → Insert card, enter PIN, withdraw money

You CANNOT: → Access **internal** cash directly, modify transaction

logs, change software

The ATM ENCAPSULATES its internals.
It only exposes a controlled interface to you.

✓ Access Modifiers — The Tools of Encapsulation

These keywords control **who can access what**:

Access Modifier	Who Can Access
public	Everyone, from anywhere
private	Only inside the same class
protected	Same class + child classes (inheritance)
internal	Same project/assembly only
protected	
internal	Same project OR child classes

The two you will use **90% of the time**:

private → hide the data (field)
public → expose controlled access (property/method)

✗ BAD Code — No Encapsulation

```

using System;

class BankAccount
{
    public string OwnerName; // completely open
    public decimal Balance; // anyone can change directly!
    public string Pin; // PIN exposed to everyone!
    Security nightmare!
}

class Program
{
    static void Main()
    {
        BankAccount account = new BankAccount();
        account.OwnerName = "Ali";
        account.Balance = 1000;
        account.Pin = "1234";

        // PROBLEM – anyone can do this:
        account.Balance = -99999; // invalid! No check!
        account.Pin = ""; // blank PIN! Security issue!

        Console.WriteLine(account.Balance); // -99999
    }
}

```

Problems:

- No validation — balance can go negative with no reason
- PIN is exposed — security breach
- Any class anywhere can corrupt this data

✓ GOOD Code — With Encapsulation

```
using System;

class BankAccount
{
    // Private fields – hidden from outside world
    private string _ownerName;
    private decimal _balance;
    private string _pin;

    // Public Properties – controlled access points
    public string OwnerName
    {
        get { return _ownerName; }
        set
        {
            if (string.IsNullOrEmpty(value))
                throw new ArgumentException("Owner name cannot be empty.");
            _ownerName = value;
        }
    }

    public decimal Balance
    {
        get { return _balance; } // can READ from outside
        // NO setter – balance should only change via
        Deposit/Withdraw methods
    }

    // Constructor – controlled initialization
    public BankAccount(string ownerName, string pin, decimal
initialBalance)
    {
```

```

        OwnerName = ownerName;    // goes through our validated
property
        _pin = pin;                // private – never exposed
directly

        if (initialBalance < 0)
            throw new ArgumentException("Initial balance cannot
be negative.");
        _balance = initialBalance;
    }

    // Public Methods – controlled behavior
    public void Deposit(decimal amount)
    {
        if (amount <= 0)
            throw new ArgumentException("Deposit amount must be
positive.");

        _balance += amount;
        Console.WriteLine($"Deposited: {amount:C}. New Balance:
{_balance:C}");
    }

    public void Withdraw(decimal amount, string pin)
    {
        // Validate PIN internally – never expose PIN comparison
logic outside
        if (pin != _pin)
        {
            Console.WriteLine("Incorrect PIN. Transaction
denied.");
            return;
        }

        if (amount <= 0)

```

```

        throw new ArgumentException("Withdrawal amount must
be positive.");

        if (amount > _balance)
        {
            Console.WriteLine("Insufficient funds.");
            return;
        }

        _balance -= amount;
        Console.WriteLine($"Withdrawn: {amount:C}. Remaining
Balance: {_balance:C}");
    }

    public string GetMaskedPin()
    {
        return "*****"; // never expose real PIN
    }
}

class Program
{
    static void Main()
    {
        BankAccount account = new BankAccount("Ali", "1234",
1000m);

        Console.WriteLine($"Owner   : {account.OwnerName}");
        Console.WriteLine($"Balance: {account.Balance:C}");
        Console.WriteLine($"PIN     : {account.GetMaskedPin()}");

        Console.WriteLine("\n--- Transactions ---");
        account.Deposit(500);
        account.Withdraw(200, "1234"); // correct PIN
        account.Withdraw(200, "9999"); // wrong PIN
    }
}

```

```

        // These are now IMPOSSIBLE from outside:
        // account._balance = -99999;    ❌ won't compile –
private
        // account._pin = "";           ❌ won't compile –
private
        // account.Balance = 5000;      ❌ won't compile – no
setter
    }
}

```

Output:

```

Owner   : Ali
Balance: $1,000.00
PIN     : ****

--- Transactions ---
Deposited: $500.00. New Balance: $1,500.00
Withdrawn: $200.00. Remaining Balance: $1,300.00
Incorrect PIN. Transaction denied.

```

Now the class **controls its own data**. Nothing outside can break it.

Properties Deep Dive

Properties are the **main tool of encapsulation** in C#.

Full Property:

```
private int _age;
```



```

public int Age
{
    get { return _age; }           // read logic
    set                               // write logic
    {
        if (value < 0 || value > 150)
            throw new ArgumentOutOfRangeException("Invalid
age.");
        _age = value;
    }
}

```

Auto Property (when no custom logic needed):

```

// C# creates the private field automatically behind the scenes
public string Name { get; set; }

```

Read-Only Property (can set in constructor only):

```

public string Id { get; private set; }

// OR with init (C# 9+):
public string Id { get; init; } // can only be set during object
creation

```

Computed Property (no backing field, calculates on the fly):

```

public class Rectangle
{
    public double Width { get; set; }
    public double Height { get; set; }

    // Calculated – no separate field needed

```

```
public double Area => Width * Height;
public double Perimeter => 2 * (Width + Height);
}
```



Real Project Example — ASP.NET Style

In a real enterprise project, you will see encapsulation like this every single day:

```
using System;

// Domain Model – encapsulates business rules
public class Product
{
    private string _name;
    private decimal _price;
    private int _stockQuantity;

    public int Id { get; private set; }

    public string Name
    {
        get => _name;
        set
        {
            if (string.IsNullOrEmpty(value))
                throw new ArgumentException("Product name is
required.");
            if (value.Length > 100)
                throw new ArgumentException("Name cannot exceed
100 characters.");
            _name = value;
        }
    }
}
```

```

    }

    public decimal Price
    {
        get => _price;
        set
        {
            if (value < 0)
                throw new ArgumentException("Price cannot be
negative.");
            _price = value;
        }
    }

    public int StockQuantity
    {
        get => _stockQuantity;
        private set // only this class can set stock directly
        {
            if (value < 0)
                throw new InvalidOperationException("Stock cannot
go below zero.");
            _stockQuantity = value;
        }
    }

    // Computed property
    public bool IsInStock => _stockQuantity > 0;
    public bool IsLowStock => _stockQuantity > 0 &&
_stockQuantity <= 5;

    // Constructor
    public Product(int id, string name, decimal price, int
initialStock)
    {

```

```

        Id = id;
        Name = name;
        Price = price;
        StockQuantity = initialStock;
    }

    // Controlled behavior methods
    public void AddStock(int quantity)
    {
        if (quantity <= 0)
            throw new ArgumentException("Quantity must be
positive.");
        StockQuantity += quantity;
        Console.WriteLine($"Stock added. New quantity:
{StockQuantity}");
    }

    public bool TrySell(int quantity)
    {
        if (quantity <= 0)
        {
            Console.WriteLine("Invalid quantity.");
            return false;
        }

        if (quantity > StockQuantity)
        {
            Console.WriteLine($"Not enough stock. Available:
{StockQuantity}");
            return false;
        }

        StockQuantity -= quantity;
        Console.WriteLine($"Sold {quantity} unit(s). Remaining:
{StockQuantity}");
    }

```

```

        return true;
    }

    public void DisplayInfo()
    {
        Console.WriteLine($"\\n--- Product Info ---");
        Console.WriteLine($"ID      : {Id}");
        Console.WriteLine($"Name    : {Name}");
        Console.WriteLine($"Price   : {Price:C}");
        Console.WriteLine($"Stock   : {StockQuantity}");
        Console.WriteLine($"In Stock : {IsInStock}");
        Console.WriteLine($"Low Stock: {IsLowStock}");
    }
}

class Program
{
    static void Main()
    {
        var product = new Product(1, "Laptop", 999.99m, 10);
        product.DisplayInfo();

        product.TrySell(3);
        product.TrySell(15); // not enough stock
        product.AddStock(5);
        product.DisplayInfo();

        // All these are BLOCKED – encapsulation working:
        // product.StockQuantity = -5; ❌ private set
        // product.Id = 99;           ❌ private set
    }
}

```

Output:

--- Product Info ---

ID : 1
Name : Laptop
Price : \$999.99
Stock : 10
In Stock : True
Low Stock: False

Sold 3 unit(s). Remaining: 7
Not enough stock. Available: 7
Stock added. New quantity: 12

--- Product Info ---

ID : 1
Name : Laptop
Price : \$999.99
Stock : 12
In Stock : True
Low Stock: False

Naming Convention (Industry Standard)

In real projects, follow these conventions:

```
// Private fields – underscore prefix, camelCase
private string _userName;
private decimal _accountBalance;

// Public properties – PascalCase
public string UserName { get; set; }
public decimal AccountBalance { get; set; }
```

```
// Private methods – camelCase
private void calculateTax() { }

// Public methods – PascalCase
public void ProcessPayment() { }

// Constants – PascalCase or ALL_CAPS (team preference)
private const decimal MaxBalance = 1_000_000m;
```

This is what you will see in **every Microsoft/enterprise codebase**.

Constructor Best Practices

```
public class Order
{
    public int Id { get; private set; }
    public string CustomerName { get; private set; }
    public DateTime CreatedAt { get; private set; }
    public decimal TotalAmount { get; private set; }

    // Constructor ensures object is ALWAYS in valid state
    public Order(int id, string customerName, decimal
totalAmount)
    {
        if (id <= 0)
            throw new ArgumentException("Order ID must be
positive.");
        if (string.IsNullOrEmpty(customerName))
            throw new ArgumentException("Customer name
required.");
        if (totalAmount < 0)
```

```
        throw new ArgumentException("Total amount cannot be  
negative.");  
  
        Id = id;  
        CustomerName = customerName;  
        TotalAmount = totalAmount;  
        CreatedAt = DateTime.UtcNow; // system sets this – user  
cannot fake it  
    }  
}
```

Key principle: After the constructor runs, the object must be in a **valid, consistent state**. No invalid objects should exist.

Common Beginner Mistakes

Mistake 1 — Making everything public:

```
// ❌ BAD  
public class User  
{  
    public string Password;           // security nightmare  
    public decimal AccountBalance; // no validation possible  
}
```

Mistake 2 — Pointless properties with no protection:

```
// ❌ Still bad – same as public field, no benefit  
public class User  
{  
    private string _name;  
    public string Name
```



```
{
    get { return _name; }
    set { _name = value; } // no validation – why bother?
}

// Use auto-property instead if no validation needed:
public string Name { get; set; } // ✅ cleaner
```

Mistake 3 — Exposing internal collections directly:

```
// ❌ BAD – outside code can modify the list directly
public class Order
{
    public List<string> Items { get; set; } = new();
}

// ✅ GOOD – return read-only view, use method to add
public class Order
{
    private List<string> _items = new();

    public IReadOnlyList<string> Items => _items.AsReadOnly();

    public void AddItem(string item)
    {
        if (string.IsNullOrEmpty(item))
            throw new ArgumentException("Item cannot be empty.");
        _items.Add(item);
    }
}
```

Interview Questions

Q1: What is encapsulation in simple terms?

*Hiding internal data and exposing only controlled access through properties and methods.
The class protects its own state.*

Q2: What is the difference between a field and a property?

A field is a raw variable. A property wraps a field with get/set accessors, allowing validation, computed values, and access control.

Q3: Why should fields be private?

To prevent outside code from directly changing the object's state in unexpected ways, bypassing business rules and validation.

Q4: What is a read-only property and when do you use it?

A property with only a getter. Used when a value should be set only during construction and never changed after, like an ID or creation timestamp.

Q5: Can you explain encapsulation with a real example?

A BankAccount class where Balance has no public setter. You can only change balance through Deposit() and Withdraw() methods that validate the operation first.

Practice Exercise

Build this class yourself:

Create a "Student" class with these rules:

Fields (**all** private):

- name (string)
- age (**int**)
- marks (**decimal**)

Rules:

- Name cannot be **null or empty**
- Age must be **between 5 and 25**
- Marks must be **between 0 and 100**

Properties:

- Name, Age, Marks **with** proper validation
- Grade (read-**only**, computed):
 - 90-100** → "A"
 - 75-89** → "B"
 - 60-74** → "C"
 - below

20/02/2026, 00:52:07

👤 Cover this stage 1 content explain in not much longer so make roepr

20/02/2026, 00:52:34

Thought process: Orchestrated comprehensive Stage 1 curriculum coverage concisely.

The user wants me to cover Stage 1 content but make it more concise/proper - not too long. They want proper coverage of all Stage 1 topics without being overly verbose. Let me organize Stage 1 completely but more concisely.

Stage 1 topics:

- Data Types (Value vs Reference types, Stack vs Heap, Nullable)  done
- OOP Principles: Encapsulation  done, Abstraction, Inheritance, Polymorphism
- Advanced OOP: Interfaces, Abstract classes, Sealed classes, Virtual/override, Method overloading/overriding
- Important Concepts: Delegates, Events, LINQ, Async/Await, Tasks, Exception Handling, Garbage Collection

Now I need to cover the remaining OOP topics plus the important concepts in a structured but concise way. I'll work through Abstraction, Inheritance, Polymorphism, Interfaces, Abstract classes, Sealed classes, Virtual/override, Method overloading/overriding, Delegates, Events, LINQ, and Async/Await with clear examples but without being verbose.

Stage 1 — Complete C# Fundamentals

OOP Pillar 2 — Abstraction

What Is It?

"Show only what is necessary. Hide how it works internally."

Encapsulation = hide data Abstraction = hide complexity/implementation

Real world: You press a car accelerator. You don't know how fuel injection works. That complexity is **abstracted** from you.

Working Example

```
using System;

// Abstract class defines WHAT must be done, not HOW
abstract class PaymentProcessor
{
    // Abstract method – child MUST implement this
    public abstract bool ProcessPayment(decimal amount);

    // Regular method – shared logic all children use
    public void PrintReceipt(decimal amount)
    {
        Console.WriteLine($"Payment of {amount:C} processed successfully.");
    }
}
```

```
class CreditCardProcessor : PaymentProcessor
{
    public override bool ProcessPayment(decimal amount)
    {
        Console.WriteLine($"Processing credit card payment:
{amount:C}");
        return true; // actual credit card logic here
    }
}

class PayPalProcessor : PaymentProcessor
{
    public override bool ProcessPayment(decimal amount)
    {
        Console.WriteLine($"Processing PayPal payment:
{amount:C}");
        return true; // actual PayPal logic here
    }
}

class Program
{
    static void Main()
    {
        PaymentProcessor processor = new CreditCardProcessor();
        bool success = processor.ProcessPayment(250m);
        if (success) processor.PrintReceipt(250m);

        Console.WriteLine();

        processor = new PayPalProcessor();
        success = processor.ProcessPayment(100m);
        if (success) processor.PrintReceipt(100m);
    }
}
```

```
}  
  
}
```

Output:

```
Processing credit card payment: $250.00  
Payment of $250.00 processed successfully.
```

```
Processing PayPal payment: $100.00  
Payment of $100.00 processed successfully.
```

The user of `PaymentProcessor` doesn't care HOW each payment works. They just call `ProcessPayment()`. That's abstraction.



OOP Pillar 3 — Inheritance

What Is It?

"A child class inherits properties and methods from a parent class, and can add its own."

Avoids code duplication. Build once, reuse everywhere.

Working Example

```
using System;  
  
// Parent (Base) class
```

```

class Employee
{
    public string Name { get; set; }
    public decimal BaseSalary { get; set; }

    public Employee(string name, decimal baseSalary)
    {
        Name = name;
        BaseSalary = baseSalary;
    }

    public virtual decimal CalculateSalary()
    {
        return BaseSalary;
    }

    public void DisplayInfo()
    {
        Console.WriteLine($"Name    : {Name}");
        Console.WriteLine($"Salary : {CalculateSalary():C}");
    }
}

// Child class – inherits from Employee
class Manager : Employee
{
    public decimal Bonus { get; set; }

    public Manager(string name, decimal baseSalary, decimal
bonus)
        : base(name, baseSalary) // calls parent constructor
    {
        Bonus = bonus;
    }
}

```



```

    // Override parent method
    public override decimal CalculateSalary()
    {
        return BaseSalary + Bonus;
    }
}

// Another child class
class Intern : Employee
{
    public Intern(string name, decimal baseSalary)
        : base(name, baseSalary) { }

    public override decimal CalculateSalary()
    {
        return BaseSalary * 0.8m; // interns get 80%
    }
}

class Program
{
    static void Main()
    {
        var manager = new Manager("Ali", 5000m, 1500m);
        var intern = new Intern("Sara", 2000m);

        Console.WriteLine("--- Manager ---");
        manager.DisplayInfo();

        Console.WriteLine("\n--- Intern ---");
        intern.DisplayInfo();
    }
}

```

Output:

```
--- Manager ---  
Name    : Ali  
Salary  : $6,500.00  
  
--- Intern ---  
Name    : Sara  
Salary  : $1,600.00
```

Key rules:

- Use `: ParentClass` to inherit
- Use `base()` to call parent constructor
- Use `virtual` in parent to allow override
- Use `override` in child to replace behavior
- C# only allows **single inheritance** (one parent class only)

OOP Pillar 4 — Polymorphism

What Is It?

"One interface, many forms. Same method call behaves differently depending on the object."

Poly = many, Morph = form

Two Types

1. Compile-Time Polymorphism — Method Overloading

Same method name, different parameters:

```
using System;

class Calculator
{
    // Same name, different parameters
    public int Add(int a, int b) => a + b;
    public double Add(double a, double b) => a + b;
    public int Add(int a, int b, int c) => a + b + c;
    public string Add(string a, string b) => a + b; // string
concat
}

class Program
{
    static void Main()
    {
        var calc = new Calculator();

        Console.WriteLine(calc.Add(2, 3));           // 5
        Console.WriteLine(calc.Add(2.5, 3.1));       // 5.6
        Console.WriteLine(calc.Add(1, 2, 3));         // 6
        Console.WriteLine(calc.Add("Hello ", "World")); // Hello
World
    }
}
```

C# decides which method to call at **compile time** based on the argument types.

2. Runtime Polymorphism — Method Overriding

Same method call, different behavior depending on actual object type:

```
using System;

class Animal
{
    public string Name { get; set; }

    public Animal(string name) => Name = name;

    public virtual void MakeSound()
    {
        Console.WriteLine("Some generic animal sound");
    }
}

class Dog : Animal
{
    public Dog(string name) : base(name) { }

    public override void MakeSound()
    {
        Console.WriteLine($"{Name} says: Woof!");
    }
}

class Cat : Animal
{
    public Cat(string name) : base(name) { }

    public override void MakeSound()
    {
        Console.WriteLine($"{Name} says: Meow!");
    }
}
```

```
class Program
{
    static void Main()
    {
        // All stored as Animal type – this is the power of polymorphism
        Animal[] animals = {
            new Dog("Rex"),
            new Cat("Whiskers"),
            new Dog("Buddy"),
            new Cat("Luna")
        };

        // Same method call → different behavior per object
        foreach (Animal animal in animals)
        {
            animal.MakeSound();
        }
    }
}
```

Output:

```
Rex says: Woof!
Whiskers says: Meow!
Buddy says: Woof!
Luna says: Meow!
```

Real project power: You can process a list of mixed objects uniformly. In enterprise apps this is used constantly with services, handlers, and processors.



Advanced OOP — Interfaces

What Is It?

*"A contract. Any class that implements this interface **MUST** provide these methods."*

Interface defines **what** to do. Class decides **how**.

Key difference from abstract class:

- A class can implement **multiple interfaces** (solves single inheritance limitation)
- Interface has **no implementation** (before C# 8) — pure contract
- Interface cannot have constructors or fields

Working Example

```
using System;
using System.Collections.Generic;

// Interfaces – pure contracts
interface IStorable
{
    void Save();
    void Delete(int id);
}

interface IPrintable
{
    void Print();
}
```

```

// A class can implement MULTIPLE interfaces
class Invoice : IStorable, IPrintable
{
    public int Id { get; set; }
    public string CustomerName { get; set; }
    public decimal Amount { get; set; }

    public Invoice(int id, string customerName, decimal amount)
    {
        Id = id;
        CustomerName = customerName;
        Amount = amount;
    }

    // Must implement ALL IStorable methods
    public void Save()
    {
        Console.WriteLine($"Invoice #{Id} saved to database.");
    }

    public void Delete(int id)
    {
        Console.WriteLine($"Invoice #{id} deleted from
database.");
    }

    // Must implement ALL IPrintable methods
    public void Print()
    {
        Console.WriteLine($"\\n=== INVOICE #{Id} ===");
        Console.WriteLine($"Customer : {CustomerName}");
        Console.WriteLine($"Amount    : {Amount:C}");
        Console.WriteLine($"=====");
    }
}

```

```

}

class Program
{
    static void Main()
    {
        var invoice = new Invoice(1001, "Ali", 450m);

        invoice.Print();
        invoice.Save();
        invoice.Delete(1001);

        // You can also use interface as a type
        IStorable storable = invoice;
        storable.Save(); // only sees IStorable methods

        IPrintable printable = invoice;
        printable.Print(); // only sees IPrintable methods
    }
}

```

Output:

```

=== INVOICE #1001 ===
Customer : Ali
Amount   : $450.00
=====
Invoice #1001 saved to database.
Invoice #1001 deleted from database.
Invoice #1001 saved to database.

=== INVOICE #1001 ===
Customer : Ali

```


Amount : \$450.00

=====

Real Enterprise Usage — Dependency Injection Pattern

```
// Interface – defines the contract
public interface IEmailService
{
    void SendEmail(string to, string subject, string body);
}

// Real implementation
public class SmtplibEmailService : IEmailService
{
    public void SendEmail(string to, string subject, string body)
    {
        Console.WriteLine($"Sending real email to {to}:
{subject}");
        // actual SMTP logic
    }
}

// Test/Mock implementation
public class FakeEmailService : IEmailService
{
    public void SendEmail(string to, string subject, string body)
    {
        Console.WriteLine($"[TEST] Fake email to {to}:
{subject}");
        // no real email sent during testing
    }
}
```

```
// Service uses interface – doesn't care which implementation
public class OrderService
{
    private readonly IEmailService _emailService;

    public OrderService(IEmailService emailService)
    {
        _emailService = emailService; // injected
    }

    public void PlaceOrder(string customerEmail)
    {
        // ... order logic ...
        _emailService.SendEmail(customerEmail, "Order Confirmed",
        "Your order is placed.");
    }
}
```

This is **exactly how ASP.NET Core Dependency Injection works**. You will write this pattern every single day in real projects.

Abstract Class vs Interface — Quick Comparison

Feature	Abstract Class	Interface
Can have fields	✓ Yes	✗ No
Can have constructor	✓ Yes	✗ No
Can have impl code	✓ Yes	⚠ C# 8+ only

Multiple inheritance	✗ No	✓ Yes
Access modifiers	✓ Yes	✗ Public only
Use when	Shared base code	Define contract

Rule of thumb:

- Use **abstract class** when classes share common code/state
- Use **interface** when you want to define a contract or need multiple "types"

Sealed Classes

What Is It?

"Prevents a class from being inherited."

```
sealed class PaymentToken
{
    public string Token { get; }
    public DateTime ExpiresAt { get; }

    public PaymentToken(string token)
    {
        Token = token;
        ExpiresAt = DateTime.UtcNow.AddMinutes(15);
    }
}

// ✗ This will give a compile error
```

```
class ExtendedToken : PaymentToken { } // ERROR: cannot inherit
sealed class
```

When to use sealed:

- Security-sensitive classes that must not be modified
- Performance optimization (compiler can optimize sealed classes)
- When the design is complete and extension would break logic

You will also see **sealed override** to stop a specific method from being overridden further:

```
class Base
{
    public virtual void Process() { }
}

class Middle : Base
{
    public sealed override void Process()
    {
        Console.WriteLine("Middle Process – no further override
allowed");
    }
}

class Child : Middle
{
    // ❌ Cannot override Process() – it's sealed
}
```

Virtual / Override — Proper Deep Understanding

```
using System;

class Shape
{
    // virtual = child CAN override this
    public virtual double CalculateArea()
    {
        return 0;
    }

    // NOT virtual = child cannot override this
    public void DisplayType()
    {
        Console.WriteLine("I am a Shape");
    }
}

class Circle : Shape
{
    public double Radius { get; set; }

    public Circle(double radius) => Radius = radius;

    // override = replacing parent's virtual method
    public override double CalculateArea()
    {
        return Math.PI * Radius * Radius;
    }
}

class Rectangle : Shape
```

```

{
    public double Width { get; set; }
    public double Height { get; set; }

    public Rectangle(double width, double height)
    {
        Width = width;
        Height = height;
    }

    public override double CalculateArea()
    {
        return Width * Height;
    }
}

class Program
{
    static void Main()
    {
        Shape[] shapes = {
            new Circle(5),
            new Rectangle(4, 6),
            new Circle(3)
        };

        foreach (Shape shape in shapes)
        {
            Console.WriteLine($"{shape.GetType().Name} area:
{shape.CalculateArea():F2}");
        }
    }
}

```

Output:

Circle area: 78.54
Rectangle area: 24.00
Circle area: 28.27

Delegates

What Is It?

"A type-safe function pointer. A variable that holds a reference to a method."

Think of it as: **"Store a method in a variable and call it later."**

Working Example

```
using System;

class Program
{
    // Define delegate – matches method signature (int, int) → int
    delegate int MathOperation(int a, int b);

    static int Add(int a, int b) => a + b;
    static int Subtract(int a, int b) => a - b;
    static int Multiply(int a, int b) => a * b;

    static void Main()
    {
```

```

// Assign method to delegate variable
MathOperation operation = Add;
Console.WriteLine($"Add: {operation(10, 5)}"); //
15

operation = Subtract;
Console.WriteLine($"Subtract: {operation(10, 5)}"); // 5

operation = Multiply;
Console.WriteLine($"Multiply: {operation(10, 5)}"); //
50

// Multicast delegate – chain multiple methods
MathOperation multicast = Add;
multicast += Subtract; // both get called (last result
returned)

// Common built-in delegates (no need to define your own
usually)
Func<int, int, int> add = (a, b) => a + b; //
returns value
Action<string> print = msg => Console.WriteLine(msg); //
returns void
Predicate<int> isPositive = n => n > 0; //
returns bool

Console.WriteLine(add(3, 4)); // 7
print("Hello from Action!"); // Hello from Action!
Console.WriteLine(isPositive(-5)); // False
}
}

```

Real usage — passing behavior as parameter:


```
using System;
using System.Collections.Generic;
using System.Linq;

class Program
{
    static void ProcessNumbers(List<int> numbers, Func<int, bool>
filter, Action<int> display)
    {
        foreach (var num in numbers.Where(filter))
        {
            display(num);
        }
    }

    static void Main()
    {
        var numbers = new List<int> { 1, -2, 3, -4, 5, -6, 7 };

        Console.WriteLine("Positive numbers:");
        ProcessNumbers(
            numbers,
            n => n > 0,           // filter: only positives
            n => Console.WriteLine($" {n}") // display each
        );

        Console.WriteLine("Numbers greater than 3:");
        ProcessNumbers(
            numbers,
            n => n > 3,
            n => Console.WriteLine($" {n}")
        );
    }
}
```

Output:

Positive numbers:

1

3

5

7

Numbers greater than 3:

5

7

Events

What Is It?

"Built on delegates. A class can notify other classes when something happens."

Publisher → raises event Subscriber → listens and reacts

Working Example

```
using System;

// Publisher class
class OrderProcessor
{
    // 1. Define event using EventHandler
    public event EventHandler<string> OrderPlaced;
```

```

public event EventHandler<string> OrderFailed;

public void ProcessOrder(string orderId, bool shouldSucceed)
{
    Console.WriteLine($"Processing order {orderId}...");

    if (shouldSucceed)
    {
        // 2. Raise the event
        OrderPlaced?.Invoke(this, orderId);
    }
    else
    {
        OrderFailed?.Invoke(this, orderId);
    }
}

// Subscriber classes
class EmailNotifier
{
    public void OnOrderPlaced(object sender, string orderId)
    {
        Console.WriteLine($"[EMAIL] Order {orderId} confirmed
email sent.");
    }

    public void OnOrderFailed(object sender, string orderId)
    {
        Console.WriteLine($"[EMAIL] Order {orderId} failed email
sent.");
    }
}

class InventoryUpdater

```

```

{
    public void OnOrderPlaced(object sender, string orderId)
    {
        Console.WriteLine($"[INVENTORY] Stock reduced for order {orderId}.");
    }
}

class Program
{
    static void Main()
    {
        var processor = new OrderProcessor();
        var emailer = new EmailNotifier();
        var inventory = new InventoryUpdater();

        // 3. Subscribe to events
        processor.OrderPlaced += emailer.OnOrderPlaced;
        processor.OrderPlaced += inventory.OnOrderPlaced;
        processor.OrderFailed += emailer.OnOrderFailed;

        processor.ProcessOrder("ORD-001", true);
        Console.WriteLine();
        processor.ProcessOrder("ORD-002", false);
    }
}

```

Output:

```

Processing order ORD-001...
[EMAIL] Order ORD-001 confirmed email sent.
[INVENTORY] Stock reduced for order ORD-001.

```

```
Processing order ORD-002...  
[EMAIL] Order ORD-002 failed email sent.
```

Multiple subscribers react to one event. This is the **Observer Pattern** used in real enterprise systems.

LINQ (Language Integrated Query)

What Is It?

"Query collections using C# code instead of SQL-style loops."

Used absolutely everywhere in .NET projects.

Working Examples

```
using System;  
using System.Collections.Generic;  
using System.Linq;  
  
class Product  
{  
    public int Id { get; set; }  
    public string Name { get; set; }  
    public decimal Price { get; set; }  
    public string Category { get; set; }  
    public bool IsActive { get; set; }  
}
```

```

}

class Program
{
    static void Main()
    {
        var products = new List<Product>
        {
            new Product { Id=1, Name="Laptop", Price=999m,
Category="Electronics", IsActive=true },
            new Product { Id=2, Name="Phone", Price=599m,
Category="Electronics", IsActive=true },
            new Product { Id=3, Name="Desk", Price=299m,
Category="Furniture", IsActive=false },
            new Product { Id=4, Name="Chair", Price=199m,
Category="Furniture", IsActive=true },
            new Product { Id=5, Name="Monitor", Price=399m,
Category="Electronics", IsActive=true },
            new Product { Id=6, Name="Keyboard", Price=79m,
Category="Electronics", IsActive=false }
        };

        // WHERE – filter
        var activeProducts = products.Where(p => p.IsActive);
        Console.WriteLine($"Active products:
{activeProducts.Count()}");

        // SELECT – project/transform
        var names = products.Select(p => p.Name);
        Console.WriteLine($"All names: {string.Join(", ",
names)}");

        // ORDERBY
        var byPrice = products.OrderByDescending(p => p.Price)
            .Select(p => $"{p.Name}");
    }
}

```

```

{p.Price:C}");
    Console.WriteLine($"By price: {string.Join(", ",
byPrice)}}");

    // WHERE + SELECT chain
    var expensiveActive = products
        .Where(p => p.IsActive && p.Price > 300m)
        .Select(p => p.Name);
    Console.WriteLine($"Expensive & active: {string.Join(",
", expensiveActive)}}");

    // GROUPBY
    var byCategory = products.GroupBy(p => p.Category);
    foreach (var group in byCategory)
    {
        Console.WriteLine($"Category: {group.Key}");
        foreach (var p in group)
            Console.WriteLine($"    - {p.Name} ({p.Price:C})");
    }

    // AGGREGATE functions
    decimal totalValue = products.Where(p =>
p.IsActive).Sum(p => p.Price);
    decimal avgPrice    = products.Average(p => p.Price);
    decimal maxPrice    = products.Max(p => p.Price);
    Product cheapest    = products.OrderBy(p =>
p.Price).First();

    Console.WriteLine($"Total active value :
{totalValue:C}");
    Console.WriteLine($"Average price      : {avgPrice:C}");
    Console.WriteLine($"Most expensive    : {maxPrice:C}");
    Console.WriteLine($"Cheapest product   :
{cheapest.Name}");

```

```

// FIRSTORDEFAULT – safe way to find one item
var phone = products.FirstOrDefault(p => p.Name ==
"Phone");
Console.WriteLine($"Found: {phone?.Name ?? "Not
found"}");

var tablet = products.FirstOrDefault(p => p.Name ==
"Tablet");
Console.WriteLine($"Found: {tablet?.Name ?? "Not
found"}");

// ANY / ALL
bool anyInactive = products.Any(p => !p.IsActive);
bool allActive = products.All(p => p.IsActive);
Console.WriteLine($"Any inactive: {anyInactive}");
Console.WriteLine($"All active : {allActive}");

// DISTINCT
var categories = products.Select(p =>
p.Category).Distinct();
Console.WriteLine($"Categories: {string.Join(", ",
categories)}");
}
}

```

Output:

```

Active products: 4
All names: Laptop, Phone, Desk, Chair, Monitor, Keyboard
By price: Laptop:$999.00, Phone:$599.00, Monitor:$399.00,
Desk:$299.00, Chair:$199.00, Keyboard:$79.00
Expensive & active: Laptop, Phone, Monitor

Category: Electronics
- Laptop ($999.00)

```


- Phone (\$599.00)
- Monitor (\$399.00)
- Keyboard (\$79.00)

Category: Furniture

- Desk (\$299.00)
- Chair (\$199.00)

Total active value : \$2,276.00

Average price : \$429.17

Most expensive : \$999.00

Cheapest product : Keyboard

Found: Phone

Found: Not found

Any inactive: True

All active : False

Categories: Electronics, Furniture

Async / Await / Tasks

What Is It?

"Run operations without blocking the main thread. Essential for APIs, DB calls, HTTP requests."

Without async — thread **waits** (frozen) while waiting for database, API, file:

Thread: ———[waiting for DB 2 seconds]————▶ response
❌ Thread is blocked, can't handle other requests

With async — thread is **free** while waiting:

Thread: —[starts DB call]—[handles other requests]—[DB done, continues]—▶
✅ Thread free to do other work

Working Examples

```
using System;
using System.Net.Http;
using System.Threading.Tasks;

class Program
{
    // async method must return Task or Task<T>
    static async Task<string> FetchDataAsync(string url)
    {
        using var client = new HttpClient();

        Console.WriteLine("Starting HTTP request...");

        // await = "pause here, free the thread, come back when done"
        string result = await client.GetStringAsync(url);

        Console.WriteLine("HTTP request completed.");
        return result;
    }
}
```

```

static async Task SimulateDatabaseCallAsync(int userId)
{
    Console.WriteLine($"Fetching user {userId} from DB...");

    // Simulating DB delay
    await Task.Delay(1000); // non-blocking wait

    Console.WriteLine($"User {userId} fetched.");
}

static async Task Main()
{
    // Sequential async calls
    await SimulateDatabaseCallAsync(1);
    await SimulateDatabaseCallAsync(2);

    Console.WriteLine("\n--- Parallel async calls ---");

    // Parallel async calls – both start simultaneously
    Task task1 = SimulateDatabaseCallAsync(10);
    Task task2 = SimulateDatabaseCallAsync(20);
    Task task3 = SimulateDatabaseCallAsync(30);

    // Wait for ALL to finish
    await Task.WhenAll(task1, task2, task3);

    Console.WriteLine("All parallel tasks done.");

    // Wait for FIRST to finish
    Task<int> t1 = Task.Run(async () => { await
Task.Delay(500); return 1; });
    Task<int> t2 = Task.Run(async () => { await
Task.Delay(200); return 2; });

```

```
Task<int> winner = await Task.WhenAny(t1, t2);
Console.WriteLine($"First completed returned:
{winner.Result}");
}
}
```

Output:

```
Fetching user 1 from DB...
User 1 fetched.
Fetching user 2 from DB...
User 2 fetched.

--- Parallel async calls ---
Fetching user 10 from DB...
Fetching user 20 from DB...
Fetching user 30 from DB...
User 10 fetched.
User 20 fetched.
User 30 fetched.
All parallel tasks done.
First completed returned: 2
```

Real ASP.NET Controller Pattern

```
// Every real ASP.NET endpoint looks like this
[HttpGet("{id}")]
public async Task<IActionResult> GetProduct(int id)
{
    // Non-blocking DB call
    var product = await _dbContext.Products
```

```
        .FirstOrDefaultAsync(p => p.Id == id);

    if (product == null)
        return NotFound();

    return Ok(product);
}
```

You will write `async/await` in **every single controller method** in real projects.

Exception Handling

What Is It?

"Handle runtime errors gracefully instead of crashing the application."

Working Example — All Concepts Together

```
using System;
using System.IO;

class DatabaseException : Exception // Custom exception
{
    public int ErrorCode { get; }

    public DatabaseException(string message, int errorCode)
        : base(message)
    {
    }
}
```

```

        ErrorCode = errorCode;
    }
}

class Program
{
    static int Divide(int a, int b)
    {
        if (b == 0)
            throw new DivideByZeroException("Cannot divide by
zero.");
        return a / b;
    }

    static void ReadFile(string path)
    {
        if (!File.Exists(path))
            throw new FileNotFoundException($"File not found:
{path}");

        var content = File.ReadAllText(path);
        Console.WriteLine(content);
    }

    static void ConnectToDatabase(bool shouldFail)
    {
        if (shouldFail)
            throw new DatabaseException("Connection timeout.",
errorCode: 1001);
    }

    static void Main()
    {
        // Basic try/catch/finally
        try

```

```

{
    int result = Divide(10, 0);
}
catch (DivideByZeroException ex)
{
    Console.WriteLine($"Math error: {ex.Message}");
}
finally
{
    // ALWAYS runs – cleanup code goes here
    Console.WriteLine("Finally block always runs.\n");
}

// Multiple catch blocks – specific to general
try
{
    ReadFile("nonexistent.txt");
}
catch (FileNotFoundException ex)
{
    Console.WriteLine($"File error: {ex.Message}");
}
catch (UnauthorizedAccessException ex)
{
    Console.WriteLine($"Permission error: {ex.Message}");
}
catch (Exception ex) // catches anything else
{
    Console.WriteLine($"Unexpected error: {ex.Message}");
}

// Custom exception
try
{
    ConnectToDatabase(shouldFail: true);
}

```

```

    }
    catch (DatabaseException ex)
    {
        Console.WriteLine($"DB Error [{ex.ErrorCode}]:
{ex.Message}");
    }

    // when – conditional catch filter
    try
    {
        throw new InvalidOperationException("Operation failed
with code 500");
    }
    catch (InvalidOperationException ex) when
(ex.Message.Contains("500"))
    {
        Console.WriteLine($"Caught specific 500 error:
{ex.Message}");
    }

    Console.WriteLine("\nApplication continues normally.");
}
}

```

Output:

Math error: Cannot divide by zero.

Finally block always runs.

File error: File not found: nonexistent.txt

DB Error [1001]: Connection timeout.

Caught specific 500 error: Operation failed with code 500

Application continues normally.

Real Project — Global Exception Middleware (ASP.NET)

*// In real projects, you catch ALL unhandled exceptions globally
// so individual controllers don't need try/catch everywhere*

```
public class GlobalExceptionMiddleware
{
    private readonly RequestDelegate _next;
    private readonly ILogger<GlobalExceptionMiddleware> _logger;

    public GlobalExceptionMiddleware(RequestDelegate next,
        ILogger<GlobalExceptionMiddleware> logger)
    {
        _next = next;
        _logger = logger;
    }

    public async Task InvokeAsync(HttpContext context)
    {
        try
        {
            await _next(context); // run the next  
middleware/controller
        }
        catch (Exception ex)
        {
            _logger.LogError(ex, "Unhandled exception  
occurred.");

            context.Response.StatusCode = 500;
            await context.Response.WriteAsJsonAsync(new
            {
                error = "An unexpected error occurred.",
            });
        }
    }
}
```

```
        traceId = context.TraceIdentifier
    });
}
}
}
```

This is **production-level** exception handling. Instead of try

20/02/2026, 00:54:29

👤 Start stage 2 in proper guide in depth and proper simple good way explain don't miss flow i given and do it all one by one proper with easy proepr example also

20/02/2026, 00:57:08

Thought process: Orchestrated comprehensive ASP.NET Core curriculum spanning request lifecycle through dependency injection.

The user wants to start Stage 2 - ASP.NET Core Web Development. Let me cover it properly, one by one, in depth but simple way. Stage 2 covers:

1. ASP.NET Core - full request lifecycle
2. MVC Pattern (Model, View, Controller)
3. Web API (REST principles, HTTP verbs, Status codes, Model validation, DTO pattern)
4. Middleware
5. Dependency Injection
6. Configuration

Let me start with the first topic - understanding ASP.NET Core and the full request lifecycle, then MVC.

Stage 2 — ASP.NET Core Web Development

Lesson 1: What Is ASP.NET Core + Full Request Lifecycle

What Is ASP.NET Core?

Before writing any code, understand **what it actually is**.

ASP.NET Core is a **framework** built on top of C# and .NET that lets you build:

- Web APIs (backend for mobile/web apps)
- Web Applications
- Microservices

It is **Microsoft's main tool** for backend development. Every Microsoft enterprise project uses it.

Think of it like this:

```
Your C# Code alone = just logic, no networking
```

```
ASP.NET Core = C# + ability to receive HTTP requests
```

```
+ routing
```

```
+ middleware pipeline
```

```
+ dependency injection
```

```
+ configuration
```

```
= complete backend framework
```

How The Internet Works (Quick Basics)

Before request lifecycle, understand the basics:

BROWSER / APP

YOUR SERVER

```
|
| — HTTP Request —————> |
|   GET /api/products         |
|   Headers: { Authorization: ... } |
|   Body: { ... }             |
```

```
← HTTP Response
Status: 200 OK
Body: [{ id:1, name:"Laptop" }]
```

Every web app works this way. Request goes in, response comes out.

Full Request Lifecycle in ASP.NET Core

This is the **most important diagram** to understand:

HTTP Request Arrives



Kestrel Server

← Receives raw HTTP request



Middleware 1
(runs first)

← e.g. Exception Handler



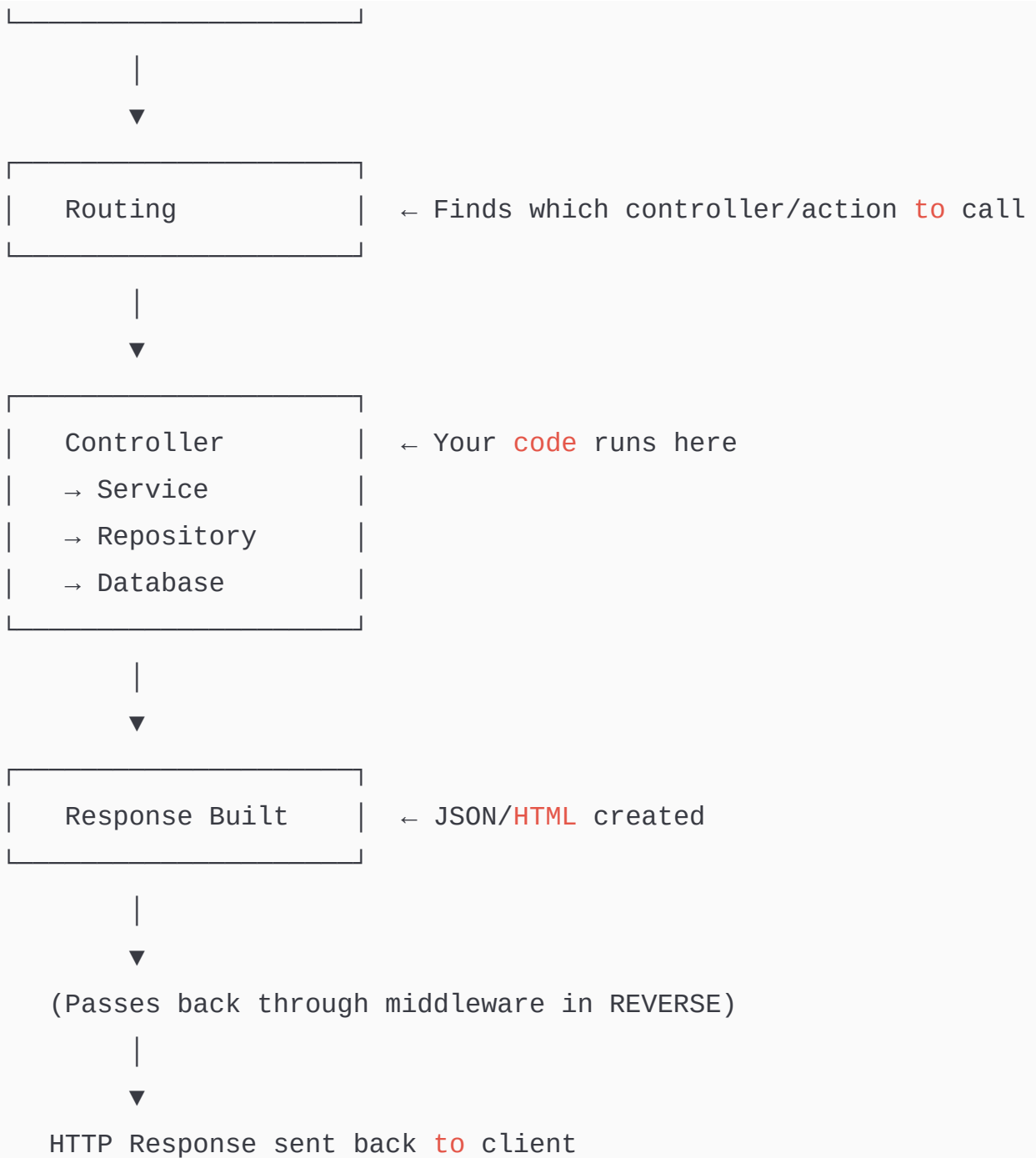
Middleware 2

← e.g. Authentication



Middleware 3

← e.g. Authorization



Every single request follows this exact path. Understanding this = understanding ASP.NET Core.

Create Your First Project

Open terminal and run:

```
dotnet new webapi -n MyFirstApi
cd MyFirstApi
dotnet run
```

Your project structure will look like:

```
MyFirstApi/
|
|— Controllers/
|   └─ WeatherForecastController.cs ← sample controller
|
|— Properties/
|   └─ launchSettings.json          ← dev settings, ports
|
|— appsettings.json                 ← configuration file
|— appsettings.Development.json     ← dev-specific config
|— Program.cs                       ← entry point (most
important)
└─ MyFirstApi.csproj                ← project file
```

Program.cs — The Heart of Everything

This file **controls everything** — what middleware runs, what services are registered, how the app starts.

```
// Program.cs – Modern .NET 6+ style (no Startup.cs anymore)

var builder = WebApplication.CreateBuilder(args);

// =====
// SECTION 1 – Register Services (DI Container)
```

```

// =====
builder.Services.AddControllers();           // registers MVC
controllers

builder.Services.AddEndpointsApiExplorer(); // for swagger
builder.Services.AddSwaggerGen();           // API documentation
UI

// Your own services registered here (we learn this in DI lesson)
// builder.Services.AddScoped<IProductService, ProductService>();

// =====
// SECTION 2 – Build the app
// =====

var app = builder.Build();

// =====
// SECTION 3 – Configure Middleware Pipeline
// (ORDER MATTERS – runs top to bottom)
// =====

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection(); // redirect HTTP → HTTPS
app.UseAuthentication();   // who are you?
app.UseAuthorization();    // what can you do?
app.MapControllers();      // connect routes to controllers

// =====
// SECTION 4 – Start the app
// =====

app.Run();

```


Two sections you must always remember:

- `builder.Services.Add...` → register things (before build)
- `app.Use...` → configure pipeline (after build)

Stage 2 — Lesson 2: MVC Pattern

What is MVC?

MVC = **Model + View + Controller**

It is a way to **organize your code** so each part has one job.

MVC	
Model	→ What the data looks like
View	→ What the user sees (HTML)
Controller	→ Who handles the request

Real world analogy:

Restaurant:

Waiter (Controller) → Takes your order, coordinates
Kitchen (Model/Logic) → Prepares the food (data/business rules)
Plate presented (View) → What you actually receive

Model — The Data Shape

Model is a simple C# class that represents your data:

```
// Models/Product.cs
namespace MyFirstApi.Models
{
    public class Product
    {
        public int Id { get; set; }
        public string Name { get; set; }
        public decimal Price { get; set; }
        public int Stock { get; set; }
        public bool IsActive { get; set; }
        public DateTime CreatedAt { get; set; }
    }
}
```

That's it. A model is just a **data shape**. No logic, no HTTP, nothing complex.

Controller — Handles the Request

Controller receives the request, calls the right logic, returns response:

```
// Controllers/ProductController.cs
using Microsoft.AspNetCore.Mvc;
using MyFirstApi.Models;

namespace MyFirstApi.Controllers
{

```

```

[ApiController]           // marks this as API controller
[Route("api/[controller]")] // URL = api/product
public class ProductController : ControllerBase
{
    // Fake data – pretend this is from database
    private static List<Product> _products = new()
    {
        new Product { Id=1, Name="Laptop", Price=999m,
Stock=10, IsActive=true, CreatedAt=DateTime.Now },
        new Product { Id=2, Name="Phone", Price=599m,
Stock=25, IsActive=true, CreatedAt=DateTime.Now },
        new Product { Id=3, Name="Monitor", Price=399m,
Stock=5, IsActive=false, CreatedAt=DateTime.Now }
    };

    // GET api/product
    [HttpGet]
    public IActionResult GetAll()
    {
        return Ok(_products);
    }

    // GET api/product/1
    [HttpGet("{id}")]
    public IActionResult GetById(int id)
    {
        var product = _products.FirstOrDefault(p => p.Id ==
id);

        if (product == null)
            return NotFound($"Product {id} not found.");

        return Ok(product);
    }
}

```

```
}  
}
```

Run the app and visit: `https://localhost:PORT/api/product`

You will see your JSON response.

View — In Web API Projects

Important clarification for beginners:

In **Web API** (which is what backend developers build), there is **no View** — you return **JSON** instead of HTML. The View is handled by the frontend (React, Angular, etc.)

```
MVC Web App = Controller + Model + View (HTML/Razor)  
Web API     = Controller + Model + JSON Response
```

In your career as a backend developer, you will mostly build **Web APIs**, not full MVC apps.

Stage 2 — Lesson 3: Web API Deep Dive

REST Principles — What Is REST?

REST = **RE**presentational **St**ate **T**ransfer

It is a set of **rules/conventions** for designing APIs that the whole industry follows.

6 REST Rules (simplified):

1. Client-Server → Frontend and backend are separate
2. Stateless → Each request is independent, server stores no session
3. Uniform Interface → Standard URLs and HTTP verbs
4. Resource-based → URLs represent resources (things), not actions

Good REST URL design:

❌ BAD — action-based URLs (not REST)

GET /getProducts

GET /getProductById?id=1

POST /createProduct

POST /deleteProduct?id=1

✅ GOOD — resource-based URLs (REST)

GET /api/products → get all products

GET /api/products/1 → get product with id 1

POST /api/products → create new product

PUT /api/products/1 → update product with id 1

DELETE /api/products/1 → delete product with id 1

The **URL is the resource (noun)**, the **HTTP verb is the action (verb)**.

HTTP Verbs — Complete Understanding

Verb	Purpose	Has Request Body?	Example
GET	Read data	No	GET /api/products

POST	Create data	Yes		POST /api/products
PUT	Full update	Yes		PUT /api/products/1
PATCH	Partial update	Yes		PATCH /api/products/1
DELETE	Delete data	No		DELETE /api/products/1

HTTP Status Codes — What You Must Know

2xx — Success

- 200 OK → request succeeded, data returned
- 201 Created → new resource created successfully
- 204 No Content → success but nothing to return (e.g. delete)

4xx — Client Error (their fault)

- 400 Bad Request → invalid data sent by client
- 401 Unauthorized → not logged in / no token
- 403 Forbidden → logged in but no permission
- 404 Not Found → resource doesn't exist
- 409 Conflict → duplicate / already exists
- 422 Unprocessable → validation failed

5xx — Server Error (our fault)

500 Internal Server Error → something crashed in our code
503 Service Unavailable → server is down/overloaded

Complete CRUD Web API — Full Working Example

Let's build a **complete Product API** with all operations:

```
// Models/Product.cs
namespace MyFirstApi.Models
{
    public class Product
    {
        public int Id { get; set; }
        public string Name { get; set; }
        public decimal Price { get; set; }
        public int Stock { get; set; }
        public bool IsActive { get; set; }
        public DateTime CreatedAt { get; set; }
    }
}
```

```
// Controllers/ProductController.cs
using Microsoft.AspNetCore.Mvc;
using MyFirstApi.Models;

namespace MyFirstApi.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class ProductController : ControllerBase
    {
        // Simulating a database with static list
```

```

private static List<Product> _products = new()
{
    new Product
    {
        Id = 1, Name = "Laptop", Price = 999m,
        Stock = 10, IsActive = true, CreatedAt =
DateTime.UtcNow
    },
    new Product
    {
        Id = 2, Name = "Phone", Price = 599m,
        Stock = 25, IsActive = true, CreatedAt =
DateTime.UtcNow
    },
    new Product
    {
        Id = 3, Name = "Monitor", Price = 399m,
        Stock = 5, IsActive = false, CreatedAt =
DateTime.UtcNow
    }
};

private static int _nextId = 4; // auto-increment id

// _____
// GET api/product
// Returns all products
// _____
[HttpGet]
public IActionResult GetAll()
{
    return Ok(_products);
}

// _____

```



```

// GET api/product/1
// Returns single product
// -----
[HttpGet("{id}")]
public IActionResult GetById(int id)
{
    var product = _products.FirstOrDefault(p => p.Id ==
id);

    if (product == null)
        return NotFound(new { message = $"Product with id
{id} not found." });

    return Ok(product);
}

// -----
// GET api/product/search?name=laptop&minPrice=100
// Search with query parameters
// -----
[HttpGet("search")]
public IActionResult Search([FromQuery] string? name,
                           [FromQuery] decimal? minPrice,
                           [FromQuery] decimal? maxPrice)
{
    var result = _products.AsQueryable();

    if (!string.IsNullOrEmpty(name))
        result = result.Where(p =>
            p.Name.Contains(name,
StringComparison.OrdinalIgnoreCase));

    if (minPrice.HasValue)
        result = result.Where(p => p.Price >=
minPrice.Value);

```

```

        if (maxPrice.HasValue)
            result = result.Where(p => p.Price <=
maxPrice.Value);

        return Ok(result.ToList());
    }

    // -----
    // POST api/product
    // Create new product
    // -----
    [HttpPost]
    public IActionResult Create([FromBody] Product product)
    {
        if (product == null)
            return BadRequest(new { message = "Product data
is required." });

        // Basic validation
        if (string.IsNullOrEmpty(product.Name))
            return BadRequest(new { message = "Product name
is required." });

        if (product.Price < 0)
            return BadRequest(new { message = "Price cannot
be negative." });

        // Assign system values
        product.Id = _nextId++;
        product.CreatedAt = DateTime.UtcNow;
        product.IsActive = true;

        _products.Add(product);
    }

```

```

        // 201 Created – returns location of new resource
        return CreatedAtAction(
            nameof(GetById),
            new { id = product.Id },
            product
        );
    }

    // -----
    // PUT api/product/1
    // Full update – replace entire product
    // -----
    [HttpPut("{id}")]
    public IActionResult Update(int id, [FromBody] Product
updatedProduct)
    {
        var existing = _products.FirstOrDefault(p => p.Id ==
id);

        if (existing == null)
            return NotFound(new { message = $"Product {id}
not found." });

        if (string.IsNullOrEmpty(updatedProduct.Name))
            return BadRequest(new { message = "Name is
required." });

        if (updatedProduct.Price < 0)
            return BadRequest(new { message = "Price cannot
be negative." });

        // Full replace
        existing.Name = updatedProduct.Name;
        existing.Price = updatedProduct.Price;
        existing.Stock = updatedProduct.Stock;
    }
}

```

```

        existing.IsActive = updatedProduct.IsActive;

        return Ok(existing);
    }

    // -----
    // DELETE api/product/1
    // Delete product
    // -----
    [HttpDelete("{id}")]
    public IActionResult Delete(int id)
    {
        var product = _products.FirstOrDefault(p => p.Id ==
id);

        if (product == null)
            return NotFound(new { message = $"Product {id}
not found." });

        _products.Remove(product);

        return NoContent(); // 204 – success, nothing to
return
    }
}

```

Test with these URLs:

```

GET    https://localhost:PORT/api/product
GET    https://localhost:PORT/api/product/1
GET    https://localhost:PORT/api/product/search?name=lap
GET    https://localhost:PORT/api/product/search?
minPrice=400&maxPrice=1000
POST   https://localhost:PORT/api/product

```

```
PUT      https://localhost:PORT/api/product/1
DELETE  https://localhost:PORT/api/product/1
```

DTO Pattern — Very Important in Real Projects

What is a DTO?

DTO = **D**ata **T**ransfer **O**bject

"A separate class used to control what data comes IN and goes OUT of your API."

Why not just use the Model directly?

```
// Your database model has EVERYTHING
public class User
{
    public int Id { get; set; }
    public string Name { get; set; }
    public string Email { get; set; }
    public string PasswordHash { get; set; } // ← NEVER send
this to client!
    public string Salt { get; set; } // ← NEVER send
this!
    public bool IsAdmin { get; set; } // ← client
shouldn't set this
    public DateTime LastLoginAt { get; set; }
    public DateTime CreatedAt { get; set; }
}
```

If you return this model directly → **password hash exposed to client = security breach.**

If client sends this model → **they can set IsAdmin = true = security breach.**

DTOs solve this:

```
// DTOs/UserDtos.cs

// What client SENDS when creating user
public class CreateUserDto
{
    public string Name { get; set; }
    public string Email { get; set; }
    public string Password { get; set; } // raw password, we
    hash it ourselves
}

// What client SENDS when logging in
public class LoginDto
{
    public string Email { get; set; }
    public string Password { get; set; }
}

// What we RETURN to client – safe subset only
public class UserResponseDto
{
    public int Id { get; set; }
    public string Name { get; set; }
    public string Email { get; set; }
    public DateTime CreatedAt { get; set; }
    // NO password, NO hash, NO salt
}
```

Full DTO example with Product API:

```
// DTOs/ProductDtos.cs
namespace MyFirstApi.DTos
{
    // Client sends this when creating
    public class CreateProductDto
    {
        public string Name { get; set; }
        public decimal Price { get; set; }
        public int Stock { get; set; }
    }

    // Client sends this when updating
    public class UpdateProductDto
    {
        public string Name { get; set; }
        public decimal Price { get; set; }
        public int Stock { get; set; }
        public bool IsActive { get; set; }
    }

    // We return this to client
    public class ProductResponseDto
    {
        public int Id { get; set; }
        public string Name { get; set; }
        public decimal Price { get; set; }
        public int Stock { get; set; }
        public bool IsActive { get; set; }
        public string StockStatus { get; set; } // "In Stock" /
        "Low Stock" / "Out of Stock"
        public DateTime CreatedAt { get; set; }
    }
}
```

```

// Updated Controller using DTOs
using MyFirstApi.DTOS;
using MyFirstApi.Models;

[HttpPost]
public IActionResult Create([FromBody] CreateProductDto dto)
{
    // Map DTO → Model (you control the mapping)
    var product = new Product
    {
        Id = _nextId++,
        Name = dto.Name,
        Price = dto.Price,
        Stock = dto.Stock,
        IsActive = true,           // system sets this
        CreatedAt = DateTime.UtcNow // system sets this
    };

    _products.Add(product);

    // Map Model → Response DTO
    var response = MapToResponseDto(product);

    return CreatedAtAction(nameof(GetById), new { id = product.Id }, response);
}

[HttpGet("{id}")]
public IActionResult GetById(int id)
{
    var product = _products.FirstOrDefault(p => p.Id == id);
    if (product == null)
        return NotFound();

    return Ok(MapToResponseDto(product)); // return DTO, not raw

```



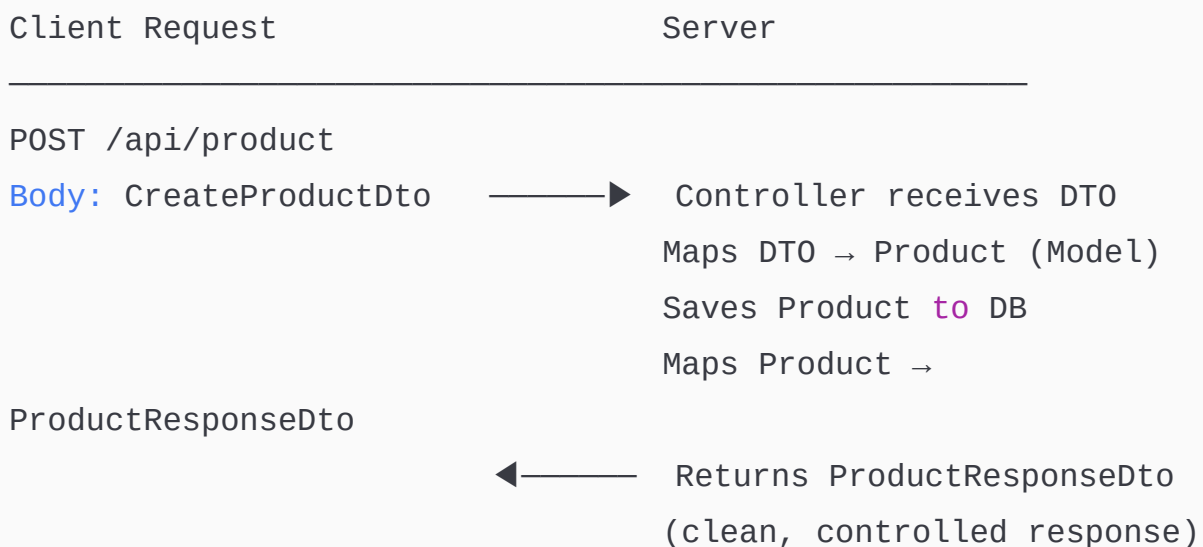
```

model
}

// Helper mapping method
private ProductResponseDto MapToResponseDto(Product product)
{
    return new ProductResponseDto
    {
        Id = product.Id,
        Name = product.Name,
        Price = product.Price,
        Stock = product.Stock,
        IsActive = product.IsActive,
        CreatedAt = product.CreatedAt,
        StockStatus = product.Stock == 0 ? "Out of Stock"
                    : product.Stock <= 5 ? "Low Stock"
                    : "In Stock"
    };
}

```

DTO Flow:



Model Validation — With Data Annotations

Instead of writing manual `if` checks in every controller, use **Data Annotations**:

```
// DTOs/CreateProductDto.cs
using System.ComponentModel.DataAnnotations;

public class CreateProductDto
{
    [Required(ErrorMessage = "Product name is required.")]
    [StringLength(100, MinimumLength = 2,
        ErrorMessage = "Name must be between 2 and 100
characters.")]
    public string Name { get; set; }

    [Required]
    [Range(0.01, 999999.99,
        ErrorMessage = "Price must be between 0.01 and
999,999.99")]
    public decimal Price { get; set; }

    [Range(0, 10000, ErrorMessage = "Stock must be between 0 and
10,000.")]
    public int Stock { get; set; }

    [EmailAddress(ErrorMessage = "Invalid email format.")]
    public string? SupplierEmail { get; set; }

    [Url(ErrorMessage = "Invalid URL format.")]
    public string? ProductImageUrl { get; set; }
}
```

Now your controller becomes clean:

```

[HttpPost]
public IActionResult Create([FromBody] CreateProductDto dto)
{
    // [ApiController] automatically checks ModelState
    // If validation fails → 400 Bad Request is returned
    automatically
    // You DON'T need to write: if (!ModelState.IsValid) manually

    var product = new Product
    {
        Id = _nextId++,
        Name = dto.Name,
        Price = dto.Price,
        Stock = dto.Stock,
        IsActive = true,
        CreatedAt = DateTime.UtcNow
    };

    _products.Add(product);
    return CreatedAtAction(nameof(GetById), new { id = product.Id
}, product);
}

```

If client sends invalid data, they automatically get:

```

{
  "errors": {
    "Name": ["Product name is required."],
    "Price": ["Price must be between 0.01 and 999,999.99"]
  },
  "status": 400,
  "title": "One or more validation errors occurred."
}

```



Stage 2 — Lesson 4: Middleware



What Is Middleware?

"Code that runs on every request before it reaches your controller, and on every response before it leaves."

Think of it like airport security:

```
You (Request) → Check-in → Security → Passport Control → Gate → Plane
                ↑           ↑           ↑           ↑
            (Middleware 1)(Middleware 2)(Middleware 3)
(Middleware 4)
```

Each step can:

- Let you pass (call next middleware)
- Stop you (return response immediately)
- Modify your luggage (modify request/response)

How Middleware Flows

```
Request  → [MW1 before] → [MW2 before] → [Controller]
                                     |
Response ← [MW1 after]  ← [MW2 after]  ←
```



The middleware pipeline is **bidirectional** — runs before AND after your controller.

Built-In Middleware Order (Very Important)

```
// Program.cs — ORDER MATTERS!
app.UseExceptionHandler();    // 1st — catch any unhandled
                                exceptions
app.UseHttpsRedirection();     // 2nd — force HTTPS
app.UseStaticFiles();          // 3rd — serve CSS/JS/images
app.UseRouting();              // 4th — figure out which controller
app.UseAuthentication();        // 5th — who are you?
app.UseAuthorization();         // 6th — what can you do?
app.MapControllers();           // 7th — run the controller
```

If you put `UseAuthorization` before `UseAuthentication` — it breaks. Order is critical.

Custom Middleware — Two Ways

Way 1 — Quick Inline Middleware (for simple things)

```
// In Program.cs
app.Use(async (context, next) =>
{
    // Code BEFORE controller
    Console.WriteLine($"[{DateTime.Now}] Incoming:
{context.Request.Method} {context.Request.Path}");

    await next(); // call next middleware / controller
```

```
// Code AFTER controller (response is being built)
Console.WriteLine($"[{DateTime.Now}] Outgoing:
{context.Response.StatusCode}");
});
```

Way 2 — Proper Class Middleware (for real projects)

```
// Middleware/RequestLoggingMiddleware.cs
namespace MyFirstApi.Middleware
{
    public class RequestLoggingMiddleware
    {
        private readonly RequestDelegate _next;
        private readonly ILogger<RequestLoggingMiddleware>
_logger;

        public RequestLoggingMiddleware(RequestDelegate next,
            ILogger<RequestLoggingMiddleware> logger)
        {
            _next = next;
            _logger = logger;
        }

        public async Task InvokeAsync(HttpContext context)
        {
            // — BEFORE controller —————
            var startTime = DateTime.UtcNow;

            _logger.LogInformation(
                "Request started: {Method} {Path} at {Time}",
                context.Request.Method,
                context.Request.Path,
                startTime);
```

```

        // — Call next middleware/controller —
        await _next(context);

        // — AFTER controller —————
        var duration = DateTime.UtcNow - startTime;

        _logger.LogInformation(
            "Request finished: {Method} {Path} | Status:
{StatusCode} | Duration: {Duration}ms",
            context.Request.Method,
            context.Request.Path,
            context.Response.StatusCode,
            duration.TotalMilliseconds);
    }
}

// Extension method – clean way to register middleware
public static class RequestLoggingMiddlewareExtensions
{
    public static IApplicationBuilder UseRequestLogging(
        this IApplicationBuilder app)
    {
        return app.UseMiddleware<RequestLoggingMiddleware>();
    }
}
}

```

Register in Program.cs:

```

// Program.cs
app.UseRequestLogging(); // your custom middleware

```

Real Project Middleware — Global Exception Handler

```
// Middleware/GlobalExceptionMiddleware.cs
using System.Net;
using System.Text.Json;

namespace MyFirstApi.Middleware
{
    public class GlobalExceptionMiddleware
    {
        private readonly RequestDelegate _next;
        private readonly ILogger<GlobalExceptionMiddleware>
_logger;

        public GlobalExceptionMiddleware(RequestDelegate next,
            ILogger<GlobalExceptionMiddleware> logger)
        {
            _next = next;
            _logger = logger;
        }

        public async Task InvokeAsync(HttpContext context)
        {
            try
            {
                await _next(context); // try to run everything
normally
            }
            catch (Exception ex)
            {
                // Something crashed – handle it here instead of
showing ugly error
                _logger.LogError(ex, "Unhandled exception
occurred.");
                await HandleExceptionAsync(context, ex);
            }
        }
    }
}
```



```

    }
}

private static async Task
HandleExceptionAsync(HttpContext context, Exception ex)
{
    context.Response.ContentType = "application/json";

    var (statusCode, message) = ex switch
    {
        ArgumentException => (400, ex.Message),
        UnauthorizedAccessException => (401,
"Unauthorized."),
        KeyNotFoundException => (404, ex.Message),
        _ => (500, "An unexpected error
occurred.");
    };

    context.Response.StatusCode = statusCode;

    var errorResponse = new
    {
        statusCode,
        message,
        traceId = context.TraceIdentifier
    };

    await context.Response.WriteAsync(
        JsonSerializer.Serialize(errorResponse));
}

public static class GlobalExceptionMiddlewareExtensions
{
    public static IApplicationBuilder

```

```
UseGlobalExceptionHandler(  
    this IApplicationBuilder app)  
{  
    return app.UseMiddleware<GlobalExceptionHandler>  
(  
);  
}  
}
```

```
// Program.cs – register FIRST so it catches everything  
app.UseGlobalExceptionHandler();  
// ... rest of middleware
```

Stage 2 — Lesson 5: Dependency Injection

What Is Dependency Injection?

This is **one of the most important concepts** in enterprise development.

First, understand the problem:

```
// ❌ WITHOUT Dependency Injection  
public class OrderController  
{  
    public IActionResult PlaceOrder()  
    {  
        // Creating dependencies manually – very bad  
        var emailService = new SmtplibEmailService();  
    }  
}
```

```

tightly coupled
    var paymentService = new StripePaymentService(); //
tightly coupled
    var inventoryService = new InventoryService(); //
tightly coupled

    // Problems:
    // 1. Hard to test – can't swap with fake services
    // 2. Hard to change – must edit controller code to
change implementation
    // 3. Violates Single Responsibility
}
}

```

DI solution — inject dependencies from outside:

```

// ✅ WITH Dependency Injection
public class OrderController
{
    private readonly IEmailService _emailService;
    private readonly IPaymentService _paymentService;

    // Dependencies are INJECTED – controller doesn't create them
    public OrderController(IEmailService emailService,
                           IPaymentService paymentService)
    {
        _emailService = emailService;
        _paymentService = paymentService;
    }

    public IActionResult PlaceOrder()
    {
        _paymentService.Process(amount: 100m);
        _emailService.Send("user@email.com", "Order placed!");
        return Ok();
    }
}

```

```
}  
  
}
```

The controller doesn't know OR care **which** EmailService or PaymentService is used. The DI container provides it.

Service Lifetimes — Critical Concept

When you register a service, you choose its **lifetime**:

Lifetime	When is a new instance created?	Use for
Transient	Every time it's requested	Stateless services
Scoped	Once per HTTP request	DB context, repos
Singleton	Once for entire app lifetime	Cache, Config

Visual explanation:

REQUEST 1:
Transient A → new instance
Transient B → new instance (different from A)
Scoped C → new instance
Scoped D → SAME instance as C (same request)
Singleton E → instance created once, never again

REQUEST 2:

Transient A → new instance (different from request 1)
Scoped C → new instance (different from request 1)
Singleton E → SAME instance from request 1

Complete DI Example — Real Project Structure

```
// Services/Interfaces/IProductService.cs
namespace MyFirstApi.Services
{
    public interface IProductService
    {
        List<ProductResponseDto> GetAll();
        ProductResponseDto? GetById(int id);
        ProductResponseDto Create(CreateProductDto dto);
        ProductResponseDto? Update(int id, UpdateProductDto dto);
        bool Delete(int id);
    }
}
```

```
// Services/ProductService.cs
using MyFirstApi.DTOS;
using MyFirstApi.Models;

namespace MyFirstApi.Services
{
    public class ProductService : IProductService
    {
        // In real project this would be DbContext
        private static List<Product> _products = new()
        {
            new Product { Id=1, Name="Laptop", Price=999m,

```

```

Stock=10,
                IsActive=true,
CreatedAt=DateTime.UtcNow },
        new Product { Id=2, Name="Phone", Price=599m,
Stock=25,
                IsActive=true,
CreatedAt=DateTime.UtcNow }
    };

    private static int _nextId = 3;

    public List<ProductResponseDto> GetAll()
    {
        return _products.Select(MapToDto).ToList();
    }

    public ProductResponseDto? GetById(int id)
    {
        var product = _products.FirstOrDefault(p => p.Id ==
id);

        return product == null ? null : MapToDto(product);
    }

    public ProductResponseDto Create(CreateProductDto dto)
    {
        var product = new Product
        {
            Id = _nextId++,
            Name = dto.Name,
            Price = dto.Price,
            Stock = dto.Stock,
            IsActive = true,
            CreatedAt = DateTime.UtcNow
        };

```

```

        _products.Add(product);
        return MapToDto(product);
    }

    public ProductResponseDto? Update(int id,
UpdateProductDto dto)
    {
        var product = _products.FirstOrDefault(p => p.Id ==
id);

        if (product == null) return null;

        product.Name = dto.Name;
        product.Price = dto.Price;
        product.Stock = dto.Stock;
        product.IsActive = dto.IsActive;

        return MapToDto(product);
    }

    public bool Delete(int id)
    {
        var product = _products.FirstOrDefault(p => p.Id ==
id);

        if (product == null) return false;

        _products.Remove(product);
        return true;
    }

    private ProductResponseDto MapToDto(Product p) => new()
    {
        Id = p.Id,
        Name = p.Name,
        Price = p.Price,
        Stock = p.Stock,

```

```

        IsActive = p.IsActive,
        CreatedAt = p.CreatedAt,
        StockStatus = p.Stock == 0 ? "Out of Stock"
                    : p.Stock <= 5 ? "Low Stock"
                    : "In Stock"
    };
}
}

```

```

// Controllers/ProductController.cs – clean with DI
using Microsoft.AspNetCore.Mvc;
using MyFirstApi.DTOS;
using MyFirstApi.Services;

namespace MyFirstApi.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class ProductController : ControllerBase
    {
        private readonly IProductService _productService;

        // DI container injects IProductService automatically
        public ProductController(IProductService productService)
        {
            _productService = productService;
        }

        [HttpGet]
        public IActionResult GetAll()
        {
            var products = _productService.GetAll();
            return Ok(products);
        }
    }
}

```



```

[HttpGet("{id}")]
public IActionResult GetById(int id)
{
    var product = _productService.GetById(id);
    if (product == null)
        return NotFound(new { message = $"Product {id}
not found." });
    return Ok(product);
}

[HttpPost]
public IActionResult Create([FromBody] CreateProductDto
dto)
{
    var product = _productService.Create(dto);
    return CreatedAtAction(nameof(GetById), new { id =
product.Id }, product);
}

[HttpPut("{id}")]
public IActionResult Update(int id, [FromBody]
UpdateProductDto dto)
{
    var product = _productService.Update(id, dto);
    if (product == null)
        return NotFound(new { message = $"Product {id}
not found." });
    return Ok(product);
}

[HttpDelete("{id}")]
public IActionResult Delete(int id)
{
    var deleted = _productService.Delete(id);
    if (!deleted)

```

```
        return NotFound(new { message = $"Product {id}
not found." });
        return NoContent();
    }
}
}
```

```
// Program.cs – Register service
builder.Services.AddScoped<IProductService, ProductService>();
// Scoped = new instance per HTTP request (correct for services)
```

DI Registration — All Three Lifetimes

```
// Program.cs – registering different lifetimes

// Scoped – new per HTTP request (most common for services +
repos)
builder.Services.AddScoped<IProductService, ProductService>();
builder.Services.AddScoped<IOrderService, OrderService>();

// Transient – new every time requested (for lightweight,
stateless)
builder.Services.AddTransient<IEmailValidator, EmailValidator>();

// Singleton – one instance forever (for shared state/config)
builder.Services.AddSingleton<ICacheService, MemoryCacheService>
();
builder.Services.AddSingleton<IConfiguration>
(builder.Configuration);
```



Stage 2 — Lesson 6: Configuration

appsettings.json — How It Works

```
// appsettings.json
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "ConnectionStrings": {
    "DefaultConnection":
"Server=myserver;Database=mydb;User=sa;Password=mypass;"
  },
  "AppSettings": {
    "AppName": "My Product API",
    "MaxItemsPerPage": 50,
    "EnableEmailNotifications": true
  },
  "JwtSettings": {
    "SecretKey": "my-super-secret-key-min-32-chars!!",
    "Issuer": "MyProductApi",
    "Audience": "MyProductApiUsers",
    "ExpiryMinutes": 60
  }
}
```

```
// appsettings.Development.json – overrides for local dev only
{
  "ConnectionStrings": {
    "DefaultConnection":
"Server=localhost;Database=mydb_dev;Trusted_Connection=True;"
  },
  "AppSettings": {
    "EnableEmailNotifications": false
  }
}
```

When running in development, ASP.NET Core **merges** both files — Development file wins on conflicts.

Reading Configuration — Three Ways

Way 1 — Direct reading (quick)

```
// Program.cs or anywhere with IConfiguration
var connectionString = builder.Configuration
    .GetConnectionString("DefaultConnection");

var appName = builder.Configuration["AppSettings:AppName"];
var maxItems = builder.Configuration.GetValue<int>
("AppSettings:MaxItemsPerPage");
```

Way 2 — Options Pattern (recommended for real projects)

```
// Configuration/AppSettings.cs
namespace MyFirstApi.Configuration
{
```

```

public class AppSettings
{
    public string AppName { get; set; }
    public int MaxItemsPerPage { get; set; }
    public bool EnableEmailNotifications { get; set; }
}

public class JwtSettings
{
    public string SecretKey { get; set; }
    public string Issuer { get; set; }
    public string Audience { get; set; }
    public int ExpiryMinutes { get; set; }
}
}

```

```

// Program.cs – bind config to classes
builder.Services.Configure<AppSettings>(
    builder.Configuration.GetSection("AppSettings"));

builder.Services.Configure<JwtSettings>(
    builder.Configuration.GetSection("JwtSettings"));

```

```

// In a service – inject and use
using Microsoft.Extensions.Options;

public class ProductService : IProductService
{
    private readonly AppSettings _settings;
    private readonly ILogger<ProductService> _logger;

    public ProductService(IOptions<AppSettings> settings,
        ILogger<ProductService> logger)
    {

```

```

        _settings = settings.Value;
        _logger = logger;
    }

    public List<ProductResponseDto> GetAll()
    {
        _logger.LogInformation("GetAll called from {AppName}",
            _settings.AppName);

        // Use MaxItemsPerPage from config
        return _products
            .Take(_settings.MaxItemsPerPage)
            .Select(MapToDto)
            .ToList();
    }
}

```

Way 3 — Environment Variables (Production)

Never store real passwords in appsettings.json. Use environment variables:

```

# Set environment variable (Linux/Mac)
export ConnectionStrings__DefaultConnection="Server=prod;..."
export JwtSettings__SecretKey="production-secret-key"

# Windows
set ConnectionStrings__DefaultConnection=Server=prod;...

```

ASP.NET Core automatically picks these up. Double underscore  maps to the nested JSON structure.

Complete Final Project Structure

After all lessons, your project should look like:

```
MyFirstApi/
|
├─ Configuration/
|   ├─ AppSettings.cs
|   └─ JwtSettings.cs
|
├─ Controllers/
|   └─ ProductController.cs
|
├─ DTOs/
|   ├─ CreateProductDto.cs
|   ├─ UpdateProductDto.cs
|   └─ ProductResponseDto.cs
|
├─ Middleware/
|   ├─ GlobalExceptionMiddleware.cs
|   └─ RequestLoggingMiddleware.cs
|
├─ Models/
|   └─ Product.cs
|
├─ Services/
|   ├─ Interfaces/
|   │   └─ IProductService.cs
|   └─ ProductService.cs
|
├─ appsettings.json
├─ appsettings.Development.json
└─ Program.cs
```

✓ Complete Program.cs — Everything Together

```
// Program.cs – production-style setup
using MyFirstApi.Configuration;
using MyFirstApi.Middleware;
using MyFirstApi.Services;

var builder = WebApplication.CreateBuilder(args);

// — Services —————
builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

// Configuration binding
builder.Services.Configure<AppSettings>(
    builder.Configuration.GetSection("AppSettings"));
builder.Services.Configure<JwtSettings>(
    builder.Configuration.GetSection("JwtSettings"));

// Your app services
builder.Services.AddScoped<IProductService, ProductService>();

// Logging
builder.Services.AddLogging();

// — Build —————
var app = builder.Build();

// — Middleware Pipeline —————
app.UseGlobalExceptionHandler(); // custom – first!
app.UseRequestLogging();           // custom logging

if (app.Environment.IsDevelopment())
{
```



```
app.UseSwagger();
app.UseSwaggerUI();
}

app.UseHttpsRedirection();
app.UseAuthentication();
app.UseAuthorization();
app.MapControllers();

// — Run —————
app.Run();
```

Stage 2 Interview Questions

Q1: What is the difference between MVC and Web API?

MVC returns HTML views for browsers. Web API returns JSON for clients (mobile apps, frontend frameworks). Web API uses ControllerBase, MVC uses Controller.

Q2: What is a DTO and why use it?

Data Transfer Object — a separate class for what comes in and goes out of API. Prevents exposing sensitive DB fields, allows input/output shaping independently.

Q3: What is middleware and give an example?

Code that runs on every request/response. Examples: authentication, logging, exception handling, HTTPS redirection.

Q4: What is the difference between Scoped, Transient, and Singleton?

Transient = new every time. Scoped = new per HTTP request. Singleton = one instance for app lifetime.

Q5: Why use Dependency Injection?

Loose coupling, testability, easy to swap implementations, and follows SOLID principles — specifically Dependency Inversion.

Q6: What is the order of middleware important?

Yes — authentication must come before authorization. Exception handler must be first to catch all errors. Wrong order breaks security or functionality.



Stage 2 Complete — What You Learned

- ✓ ASP.NET Core — what it **is**, request lifecycle
- ✓ Program.cs — services + middleware pipeline
- ✓ MVC **Pattern** — Model, Controller, JSON response
- ✓ REST Principles — proper URL design
- ✓ HTTP Verbs — **GET** **POST** **PUT** **DELETE**
- ✓ Status Codes — **200** **201** **204** **400** **401** **404** **500**
- ✓ DTOs — input/output separation
- ✓ Model Validation — Data Annotations
- ✓ Middleware — custom logging + **global** exception handler
- ✓ Dependency Injection — interfaces + lifetimes
- ✓ Configuration — appsettings + Options **pattern**

Reply **"next"** and we go into **Stage 3 — SQL Server Database** — queries, joins, indexes, stored procedures, transactions, and ACID properties. 🚀

20/02/2026, 00:59:55

 Start stage 4 proper same but much better good way

20/02/2026, 01:02:25

Thought process: Architected comprehensive Entity Framework Core learning guide.

The user wants to start Stage 4 which is ORM & Data Access (Entity Framework Core). Let me check the roadmap:

STAGE 4 — ORM & Data Access

- Entity Framework Core
- DbContext
- Migrations
- Code First
- Lazy vs Eager loading
- Performance tuning
- Tracking vs No Tracking

Let me create a comprehensive, well-structured guide for Stage 4.

Stage 4 — ORM & Data Access

Entity Framework Core — Complete Guide

What Is ORM First?

Before EF Core, understand **why it exists**.

Without ORM, you write raw SQL inside C# like this:

```
// ❌ OLD WAY – Raw ADO.NET (painful)
using var connection = new SqlConnection(connectionString);
connection.Open();

var command = new SqlCommand(
    "SELECT Id, Name, Price FROM Products WHERE IsActive = 1",
    connection);

var reader = command.ExecuteReader();

var products = new List<Product>();

while (reader.Read())
{
    products.Add(new Product
    {
        Id    = (int)reader["Id"],
        Name  = (string)reader["Name"],
        Price = (decimal)reader["Price"]
    });
}
```

Problems with this approach:

- Write SQL manually for every operation
- Manually map every column to every property
- SQL errors only found at runtime, not compile time
- Changing a column name = searching and fixing everywhere
- No type safety whatsoever

ORM solves all of this:

```
// ✅ EF CORE WAY – clean, type-safe, no raw SQL needed
var products = await _context.Products
```

```
.Where(p => p.IsActive)
.ToListAsync();
```

EF Core automatically generates the SQL, maps results to your classes, handles connections, and gives you compile-time safety.

What Is Entity Framework Core?

"EF Core is Microsoft's official ORM (Object-Relational Mapper) that lets you work with a database using C# objects instead of SQL."

YOUR C# CLASS	↔	DATABASE TABLE
Product class	↔	Products table
Property: Id	↔	Column: Id (int)
Property: Name	↔	Column: Name (nvarchar)
Property: Price	↔	Column: Price (decimal)
List<Product>	↔	Multiple rows
LINQ query	↔	SQL query (auto-generated)

EF Core translates your C# code into SQL and your SQL results back into C# objects.

Setting Up EF Core — Step by Step

Step 1 — Install NuGet Packages

```
# For SQL Server (most common in Microsoft projects)
dotnet add package Microsoft.EntityFrameworkCore.SqlServer
```

```
dotnet add package Microsoft.EntityFrameworkCore.Tools
dotnet add package Microsoft.EntityFrameworkCore.Design
```

Step 2 — Create Your Entity Classes

Entities are C# classes that **map to database tables**:

```
// Models/Product.cs
namespace MyApi.Models
{
    public class Product
    {
        public int Id { get; set; }           // Primary Key
        (auto-detected)
        public string Name { get; set; }
        public string Description { get; set; }
        public decimal Price { get; set; }
        public int Stock { get; set; }
        public bool IsActive { get; set; }
        public DateTime CreatedAt { get; set; }

        // Navigation property – relationship to Category
        public int CategoryId { get; set; }
        public Category Category { get; set; }

        // Navigation property – relationship to OrderItems
        public List<OrderItem> OrderItems { get; set; } = new();
    }
}
```

```
// Models/Category.cs
namespace MyApi.Models
{
    public class Category
```

```

{
    public int Id { get; set; }
    public string Name { get; set; }
    public string Description { get; set; }

    // Navigation – one category has many products
    public List<Product> Products { get; set; } = new();
}

```

```

// Models/Order.cs
namespace MyApi.Models
{
    public class Order
    {
        public int Id { get; set; }
        public string CustomerName { get; set; }
        public string CustomerEmail { get; set; }
        public decimal TotalAmount { get; set; }
        public string Status { get; set; } // Pending,
Confirmed, Shipped
        public DateTime OrderDate { get; set; }

        // Navigation – one order has many items
        public List<OrderItem> OrderItems { get; set; } = new();
    }
}

```

```

// Models/OrderItem.cs
namespace MyApi.Models
{
    public class OrderItem
    {
        public int Id { get; set; }
    }
}

```

```

    public int OrderId { get; set; }
    public int ProductId { get; set; }
    public int Quantity { get; set; }
    public decimal UnitPrice { get; set; }
    public decimal TotalPrice => Quantity * UnitPrice; //
    computed

    // Navigation properties
    public Order Order { get; set; }
    public Product Product { get; set; }
}
}

```

Lesson 1 — DbContext (The Core of EF Core)

What Is DbContext?

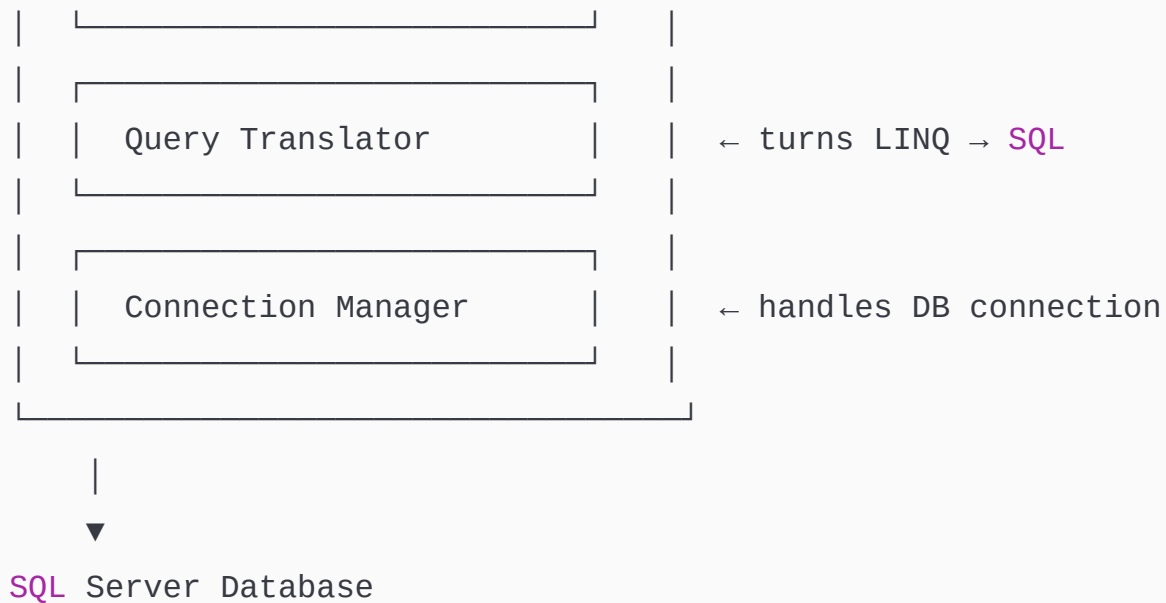
"DbContext is the bridge between your C# application and the database. It manages database connections, tracks changes, and executes queries."

Think of it as:

Your Code



changed



Creating Your DbContext

```
// Data/AppDbContext.cs
using Microsoft.EntityFrameworkCore;
using MyApi.Models;

namespace MyApi.Data
{
    public class AppDbContext : DbContext
    {
        // Constructor – receives options from DI
        public AppDbContext(DbContextOptions<AppDbContext>
options)
        : base(options)
        {
        }

        // — DbSet = Tables —————
        // Each DbSet represents one table in database
    }
}
```

```

public DbSet<Product>    Products    { get; set; }
public DbSet<Category>  Categories { get; set; }
public DbSet<Order>     Orders      { get; set; }
public DbSet<OrderItem> OrderItems  { get; set; }

// — OnModelCreating = Fluent API Configuration —
protected override void OnModelCreating(ModelBuilder
modelBuilder)
{
    base.OnModelCreating(modelBuilder);

    // — Product Configuration —
    modelBuilder.Entity<Product>(entity =>
    {
        entity.HasKey(p => p.Id); // Primary Key

        entity.Property(p => p.Name)
            .IsRequired()          // NOT NULL in DB
            .HasMaxLength(100);    // nvarchar(100)

        entity.Property(p => p.Description)
            .HasMaxLength(500);

        entity.Property(p => p.Price)
            .HasColumnType("decimal(18,2)"); // proper
money type

        entity.Property(p => p.IsActive)
            .HasDefaultValue(true); // default value

        entity.Property(p => p.CreatedAt)
            .HasDefaultValueSql("GETUTCDATE()"); // DB
sets this

        // Index for faster search by name

```

```

        entity.HasIndex(p => p.Name);

        // Relationship: Product belongs to one Category
        entity.HasOne(p => p.Category)
            .WithMany(c => c.Products)
            .HasForeignKey(p => p.CategoryId)
            .OnDelete(DeleteBehavior.Restrict); // don't
cascade delete
    });

    // — Category Configuration —————
    modelBuilder.Entity<Category>(entity =>
    {
        entity.HasKey(c => c.Id);

        entity.Property(c => c.Name)
            .IsRequired()
            .HasMaxLength(50);
    });

    // — Order Configuration —————
    modelBuilder.Entity<Order>(entity =>
    {
        entity.HasKey(o => o.Id);

        entity.Property(o => o.CustomerName)
            .IsRequired()
            .HasMaxLength(100);

        entity.Property(o => o.CustomerEmail)
            .IsRequired()
            .HasMaxLength(150);

        entity.Property(o => o.TotalAmount)
            .HasColumnType("decimal(18,2)");
    });

```

```

        entity.Property(o => o.Status)
            .HasMaxLength(20)
            .HasDefaultValue("Pending");

        entity.Property(o => o.OrderDate)
            .HasDefaultValueSql("GETUTCDATE()");
    });

    // — OrderItem Configuration —————
    modelBuilder.Entity<OrderItem>(entity =>
    {
        entity.HasKey(oi => oi.Id);

        entity.Property(oi => oi.UnitPrice)
            .HasColumnType("decimal(18,2)");

        // Ignore computed property – don't map to DB
        entity.Ignore(oi => oi.TotalPrice);

        // Relationship: OrderItem belongs to Order
        entity.HasOne(oi => oi.Order)
            .WithMany(o => o.OrderItems)
            .HasForeignKey(oi => oi.OrderId)
            .OnDelete(DeleteBehavior.Cascade); // delete
items when order deleted

        // Relationship: OrderItem references Product
        entity.HasOne(oi => oi.Product)
            .WithMany(p => p.OrderItems)
            .HasForeignKey(oi => oi.ProductId)
            .OnDelete(DeleteBehavior.Restrict);
    });

    // — Seed Data – initial data in database —————

```

```

        modelBuilder.Entity<Category>().HasData(
            new Category { Id = 1, Name = "Electronics",
                Description = "Electronic devices"
            },
            new Category { Id = 2, Name = "Furniture",
                Description = "Home and office
furniture" }
        );

        modelBuilder.Entity<Product>().HasData(
            new Product { Id=1, Name="Laptop", Price=999m,
                Stock=10, IsActive=true,
                CategoryId=1,
                CreatedAt=DateTime.UtcNow },
            new Product { Id=2, Name="Phone", Price=599m,
                Stock=25, IsActive=true,
                CategoryId=1,
                CreatedAt=DateTime.UtcNow },
            new Product { Id=3, Name="Desk", Price=299m,
                Stock=8, IsActive=true,
                CategoryId=2,
                CreatedAt=DateTime.UtcNow }
        );
    }
}

```

Register DbContext in Program.cs

```

// Program.cs
using MyApi.Data;
using Microsoft.EntityFrameworkCore;

```

```
var builder = WebApplication.CreateBuilder(args);

// Register EF Core with SQL Server
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(

builder.Configuration.GetConnectionString("DefaultConnection")
    )
);

// rest of your services...
```

```
// appsettings.json
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=
(localdb)\mssqllocaldb;Database=MyApiDb;Trusted_Connection=True;"
  }
}
```

Lesson 2 — Code First & Migrations

What Is Code First?

There are two approaches in EF Core:

DATABASE **FIRST** → You design DB tables **first**, EF generates C# classes

(legacy projects, existing databases)

CODE **FIRST** → You write C# classes **first**, EF creates/updates DB

(modern approach, what you should learn)

In **Code First**, your C# classes ARE your database schema. EF Core generates all the SQL to create tables from your classes.

What Are Migrations?

"Migrations are a version control system for your database schema."

C# Class Change → Migration Created → Migration Applied → DB Updated

Example:

1. You **add** a **new** property "Brand" **to** Product class
2. You **create** a migration
3. EF Core generates **ALTER TABLE SQL**
4. You apply migration → DB **column is** added

Migration Commands — Step by Step

```
# Step 1 – Create first migration
# This reads your DbContext and generates SQL to create all
tables
dotnet ef migrations add InitialCreate
```

```
# Step 2 – Apply migration to database
# This actually creates the tables in your DB
dotnet ef database update

# After that – adding a new property to a class:
# Step 3 – Create another migration for the change
dotnet ef migrations add AddBrandToProduct

# Step 4 – Apply the new migration
dotnet ef database update

# Other useful commands:
dotnet ef migrations list           # see all migrations
dotnet ef migrations remove        # remove last migration
dotnet ef database update 0        # revert ALL migrations
dotnet ef database update InitialCreate # revert to specific
migration
```

What Migrations Generate

When you run `dotnet ef migrations add InitialCreate`, EF Core creates a file like:

```
// Migrations/20260220_InitialCreate.cs (auto-generated – don't
edit)
public partial class InitialCreate : Migration
{
    protected override void Up(MigrationBuilder migrationBuilder)
    {
        // Creates Categories table first (Products depends on
it)
        migrationBuilder.CreateTable(
```



```

        name: "Categories",
        columns: table => new
        {
            Id = table.Column<int>(nullable: false)
                .Annotation("SqlServer:Identity", "1, 1"),
            Name = table.Column<string>(maxLength: 50,
nullable: false),
            Description = table.Column<string>(maxLength:
200, nullable: true)
        },
        constraints: table =>
        {
            table.PrimaryKey("PK_Categories", x => x.Id);
        });

// Creates Products table with FK to Categories
migrationBuilder.CreateTable(
    name: "Products",
    columns: table => new
    {
        Id = table.Column<int>(nullable: false)

.Annotation("SqlServer:Identity", "1, 1"),
        Name = table.Column<string>(maxLength:
100, nullable: false),
        Price = table.Column<decimal>(type:
"decimal(18,2)", nullable: false),
        Stock = table.Column<int>(nullable: false),
        IsActive = table.Column<bool>(defaultValue:
true),
        CreatedAt = table.Column<DateTime>(
            defaultValueSql:
"GETUTCDATE()"),
        CategoryId = table.Column<int>(nullable: false)
    },

```

```

        constraints: table =>
        {
            table.PrimaryKey("PK_Products", x => x.Id);
            table.ForeignKey(
                name: "FK_Products_Categories_CategoryId",
                column: x => x.CategoryId,
                principalTable: "Categories",
                principalColumn: "Id",
                onDelete: ReferentialAction.Restrict);
        });
    }

    protected override void Down(MigrationBuilder
migrationBuilder)
    {
        // Down = how to UNDO this migration
        migrationBuilder.DropTable(name: "Products");
        migrationBuilder.DropTable(name: "Categories");
    }
}

```

EF Core generates all this SQL automatically from your C# classes. You never write this by hand.

Apply Migrations Automatically On App Start (Production Pattern)

```

// Program.cs – auto-apply pending migrations when app starts
var app = builder.Build();

// Apply any pending migrations automatically
using (var scope = app.Services.CreateScope())

```

```
{  
    var db =  
scope.ServiceProvider.GetRequiredService<AppDbContext>();  
    db.Database.Migrate(); // applies any unapplied migrations  
}  
  
app.Run();
```

Lesson 3 — CRUD Operations with EF Core

Complete Repository Pattern — Real Project Style

```
// Repositories/Interfaces/IProductRepository.cs  
namespace MyApi.Repositories  
{  
    public interface IProductRepository  
    {  
        Task<List<Product>> GetAllAsync();  
        Task<Product?> GetByIdAsync(int id);  
        Task<Product?> GetByIdWithCategoryAsync(int id);  
        Task<List<Product>> GetByCategoryAsync(int categoryId);  
        Task<Product> CreateAsync(Product product);  
        Task<Product> UpdateAsync(Product product);  
        Task<bool> DeleteAsync(int id);  
        Task<bool> ExistsAsync(int id);  
    }  
}
```

```

// Repositories/ProductRepository.cs
using Microsoft.EntityFrameworkCore;
using MyApi.Data;
using MyApi.Models;

namespace MyApi.Repositories
{
    public class ProductRepository : IProductRepository
    {
        private readonly AppDbContext _context;

        public ProductRepository(AppDbContext context)
        {
            _context = context;
        }

        // — GET ALL —————
        public async Task<List<Product>> GetAllAsync()
        {
            return await _context.Products
                .Where(p => p.IsActive)
                .OrderBy(p => p.Name)
                .ToListAsync();
        }

        // — GET BY ID —————
        public async Task<Product?> GetByIdAsync(int id)
        {
            return await _context.Products
                .FirstOrDefaultAsync(p => p.Id == id);
        }

        // — GET BY ID WITH RELATED DATA —————
        public async Task<Product?> GetByIdWithCategoryAsync(int
id)

```

```

    {
        return await _context.Products
            .Include(p => p.Category)           // eager load
Category
            .FirstOrDefaultAsync(p => p.Id == id);
    }

// — GET BY CATEGORY —————
public async Task<List<Product>> GetByCategoryAsync(int
categoryId)
{
    return await _context.Products
        .Where(p => p.CategoryId == categoryId &&
p.IsActive)
        .Include(p => p.Category)
        .OrderBy(p => p.Price)
        .ToListAsync();
}

// — CREATE —————
public async Task<Product> CreateAsync(Product product)
{
    product.CreatedAt = DateTime.UtcNow;
    product.IsActive = true;

    _context.Products.Add(product);    // track this new
object

    await _context.SaveChangesAsync(); // execute INSERT
SQL

    return product; // now has Id from database
}

// — UPDATE —————
public async Task<Product> UpdateAsync(Product product)

```

```

        {
            _context.Products.Update(product); // mark as
modified

            await _context.SaveChangesAsync(); // execute UPDATE
SQL

            return product;
        }

// — DELETE —————
public async Task<bool> DeleteAsync(int id)
{
    var product = await _context.Products.FindAsync(id);

    if (product == null) return false;

    _context.Products.Remove(product); // mark for
deletion

    await _context.SaveChangesAsync(); // execute DELETE
SQL

    return true;
}

// — EXISTS —————
public async Task<bool> ExistsAsync(int id)
{
    return await _context.Products.AnyAsync(p => p.Id ==
id);
}
}
}

```

Understanding SaveChangesAsync — Very Important

```
// EF Core uses Unit of Work pattern internally  
// Nothing is saved to DB until SaveChangesAsync() is called  
  
// Example – multiple changes in one transaction:  
public async Task ProcessOrderAsync(Order order, List<OrderItem>  
items)  
{  
    // Step 1 – add order (not saved yet)  
    _context.Orders.Add(order);  
  
    // Step 2 – add all items (not saved yet)  
    foreach (var item in items)  
    {  
        _context.OrderItems.Add(item);  
    }  
  
    // Step 3 – update product stock (not saved yet)  
    foreach (var item in items)  
    {  
        var product = await  
_context.Products.FindAsync(item.ProductId);  
        if (product != null)  
            product.Stock -= item.Quantity;  
    }  
  
    // Step 4 – save ALL changes in ONE database transaction  
    await _context.SaveChangesAsync();  
  
    // If any step fails → ALL changes are rolled back  
automatically  
    // Either ALL succeed or NONE succeed  
}
```

Lesson 4 — Eager vs Lazy Loading

This is one of the **most important performance topics** in EF Core.

The N+1 Problem First (What We're Solving)

Imagine you have:

- 100 Products
- Each Product has a Category

WRONG approach (N+1 queries):

Query 1: `SELECT all 100 products` → 1 query

Query 2: `SELECT category for product 1` → 1 query

Query 3: `SELECT category for product 2` → 1 query

... 100 more queries...

Query 101: `SELECT category for product 100`

TOTAL: 101 queries! Terrible for performance.

CORRECT approach (1 query):

`SELECT products JOIN categories` → 1 query gets everything

Three Loading Strategies

1. Eager Loading — Load Everything Together (Recommended)


```

// Eager Loading – uses Include() to join tables in ONE query
// SQL: SELECT * FROM Products JOIN Categories...

// Single level include
var products = await _context.Products
    .Include(p => p.Category)
    .ToListAsync();

// Multi-level include
var orders = await _context.Orders
    .Include(o => o.OrderItems)           // include items
    .ThenInclude(oi => oi.Product)       // include each item's
product
    .Include(o => o.OrderItems)
    .ThenInclude(oi => oi.Product)
    // don't need to go deeper here
    .ToListAsync();

// Include with filter (EF Core 5+)
var categories = await _context.Categories
    .Include(c => c.Products.Where(p => p.IsActive)) // only
active products
    .ToListAsync();

// Multiple includes
var product = await _context.Products
    .Include(p => p.Category)
    .Include(p => p.OrderItems)
    .FirstOrDefaultAsync(p => p.Id == id);

```

2. Explicit Loading — Load on Demand

```

// First load the entity
var product = await _context.Products.FindAsync(id);

```

```
// Then load specific related data only when needed
await _context.Entry(product)
    .Reference(p => p.Category)    // load single navigation
property
    .LoadAsync();

await _context.Entry(product)
    .Collection(p => p.OrderItems) // load collection navigation
property
    .LoadAsync();

// Useful when you don't always need related data
// Example: get product, then only load category if user asks for
it
```

3. Lazy Loading — Auto-Load When Accessed (Use With Caution)

```
// Step 1 – install lazy loading package
// dotnet add package Microsoft.EntityFrameworkCore.Proxies

// Step 2 – enable in Program.cs
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(connectionString)
        .UseLazyLoadingProxies()    // enable lazy loading
);

// Step 3 – mark navigation properties as virtual
public class Product
{
    public int Id { get; set; }
    public string Name { get; set; }

    public virtual Category Category { get; set; } // virtual =
```

```

lazy loadable
    public virtual List<OrderItem> OrderItems { get; set; }
}

// Now when you access Category, EF Core auto-loads it:
var product = await _context.Products.FindAsync(id);
var categoryName = product.Category.Name; // ← triggers another
DB query here

// ⚠ WARNING – Lazy Loading causes N+1 problem easily
// Use Eager Loading (Include) in most cases
// Lazy Loading is only okay if you truly load one record at a
time

```

Comparison Table

Strategy	When loads	Queries	Best
Use Case			
Eager (.Include)	Always upfront	1 JOIN query	Most
cases 			
Explicit	When you call Load	Separate query	
Conditional loads			
Lazy (virtual)	When property used	Many queries ⚠	Rare,
be careful			

Rule: Always prefer Eager Loading. Only use Lazy Loading if you fully understand the query impact.

Lesson 5 — Tracking vs No-Tracking

What Is Change Tracking?

By default, EF Core **tracks every entity** you query. It watches for changes so

`SaveChanges()` knows what to update.

```
// TRACKING (default behavior)
var product = await _context.Products.FindAsync(1);
// EF Core now WATCHES this object in memory

product.Price = 1099m; // EF Core detects this change

await _context.SaveChangesAsync();
// EF Core knows price changed → generates UPDATE SQL
automatically
```

The Problem With Tracking (Memory + Speed)

If you only **read** data and never modify it, tracking is wasted overhead:

```
// GET endpoint – just reading, not modifying
public async Task<List<Product>> GetAllProductsForDisplay()
{
    // ❌ BAD – tracking 1000 products when we only read them
    return await _context.Products.ToListAsync();
    // EF Core is tracking all 1000 objects in memory – wasteful!
}
```

No-Tracking Queries — Better Performance for Read-Only

```
//  AsNoTracking() – skip change tracking for read-only queries  
// Faster, uses less memory
```

```
public async Task<List<Product>> GetAllForDisplayAsync()  
{  
    return await _context.Products  
        .AsNoTracking()           // tell EF Core: don't track  
these  
        .Where(p => p.IsActive)  
        .Include(p => p.Category)  
        .OrderBy(p => p.Name)  
        .ToListAsync();  
}
```

```
// For single item reads  
public async Task<Product?> GetForDisplayAsync(int id)  
{  
    return await _context.Products  
        .AsNoTracking()  
        .Include(p => p.Category)  
        .FirstOrDefaultAsync(p => p.Id == id);  
}
```

AsNoTrackingWithIdentityResolution — Best of Both

```
// When you Include() related data, same object may appear multiple times  
// (e.g. same Category on 10 Products)  
// AsNoTracking creates duplicate objects in memory  
// AsNoTrackingWithIdentityResolution avoids duplicates but still no tracking  
  
var products = await _context.Products  
    .AsNoTrackingWithIdentityResolution()  
    .Include(p => p.Category)  
    .ToListAsync();  
  
// Same category object is reused across products – memory efficient
```

When To Use Each

```
// — READ (GET endpoints) —————  
// Use AsNoTracking() – faster, less memory  
var products = await _context.Products  
    .AsNoTracking()  
    .ToListAsync();  
  
// — UPDATE —————  
// Use tracked query OR attach manually  
// Option 1: tracked query, then modify  
var product = await _context.Products.FindAsync(id);  
product.Price = 1099m;  
await _context.SaveChangesAsync();  
  
// Option 2: fetch with AsNoTracking, then manually update  
var product = await _context.Products.AsNoTracking()
```

```
.FirstOrDefaultAsync(p => p.Id == id);

product.Price = 1099m;
_context.Products.Update(product); // manually tell EF Core to update
await _context.SaveChangesAsync();

// — DELETE —————
// Need tracked entity OR attach first
var product = await _context.Products.FindAsync(id);
_context.Products.Remove(product);
await _context.SaveChangesAsync();
```

Lesson 6 — Performance Tuning

Most Important EF Core Performance Tips

1. Only Select What You Need

```
// ❌ BAD – loads entire Product object (all columns) from DB
var products = await _context.Products.ToListAsync();
Console.WriteLine(products[0].Name); // only needed Name!

// ✅ GOOD – only select columns you actually need
var productNames = await _context.Products
    .Where(p => p.IsActive)
    .Select(p => new
    {
        p.Id,
        p.Name,
        p.Price
```

```

    })
    .ToListAsync();

// Even better – project directly into your DTO
var productDtos = await _context.Products
    .Where(p => p.IsActive)
    .Select(p => new ProductResponseDto
    {
        Id      = p.Id,
        Name    = p.Name,
        Price   = p.Price,
        CategoryName = p.Category.Name // auto-joins!
    })
    .AsNoTracking()
    .ToListAsync();

```

2. Pagination — Never Load All Records

```

// ❌ BAD – loads ALL products from DB
var allProducts = await _context.Products.ToListAsync();
var page = allProducts.Skip(20).Take(10).ToList(); // filtering in memory!

// ✅ GOOD – pagination done IN THE DATABASE
public async Task<PagedResult<ProductResponseDto>> GetPagedAsync(
    int pageNumber, int pageSize)
{
    // pageNumber starts from 1
    var skip = (pageNumber - 1) * pageSize;

    // Get total count for pagination metadata
    var totalCount = await _context.Products
        .Where(p => p.IsActive)
        .CountAsync();

```



```

// Get only this page's data
var products = await _context.Products
    .Where(p => p.IsActive)
    .OrderBy(p => p.Name)
    .Skip(skip)           // SQL: OFFSET
    .Take(pageSize)       // SQL: FETCH NEXT
    .Select(p => new ProductResponseDto
    {
        Id      = p.Id,
        Name    = p.Name,
        Price   = p.Price
    })
    .AsNoTracking()
    .ToListAsync();

return new PagedResult<ProductResponseDto>
{
    Data          = products,
    TotalCount    = totalCount,
    PageNumber    = pageNumber,
    PageSize      = pageSize,
    TotalPages    = (int)Math.Ceiling(totalCount /
(double)pageSize)
};
}

// Supporting class
public class PagedResult<T>
{
    public List<T> Data { get; set; }
    public int TotalCount { get; set; }
    public int PageNumber { get; set; }
    public int PageSize { get; set; }
    public int TotalPages { get; set; }
}

```

```
public bool HasNextPage => PageNumber < TotalPages;
public bool HasPreviousPage => PageNumber > 1;
}
```

3. Use FindAsync for Primary Key Lookups

```
//  FindAsync – checks memory cache first, then DB
// Fastest way to get by primary key
var product = await _context.Products.FindAsync(id);

// vs FirstOrDefaultAsync – always hits DB
var product = await _context.Products
    .FirstOrDefaultAsync(p => p.Id == id);
```

4. Avoid Loading Data You'll Throw Away

```
//  BAD – loads product just to check if it exists
var product = await _context.Products.FindAsync(id);
if (product != null) { ... }

//  GOOD – just check existence without loading data
var exists = await _context.Products.AnyAsync(p => p.Id == id);
if (exists) { ... }

//  BAD – loads all to count
var count = (await _context.Products.ToListAsync()).Count;

//  GOOD – count in database
var count = await _context.Products.CountAsync();
```

5. Bulk Operations (EF Core 7+)

```
//  Bulk update without loading entities into memory
await _context.Products
    .Where(p => p.CategoryId == 1)
    .ExecuteUpdateAsync(setters => setters
        .SetProperty(p => p.IsActive, false));
// SQL: UPDATE Products SET IsActive = 0 WHERE CategoryId = 1

//  Bulk delete without loading entities
await _context.Products
    .Where(p => !p.IsActive)
    .ExecuteDeleteAsync();
// SQL: DELETE FROM Products WHERE IsActive = 0

// These do NOT load any data into memory – very fast for large
// datasets
```

6. Use IQueryable — Defer Execution

```
// IQueryable builds the SQL query – nothing runs yet
// ToListAsync() / FirstOrDefaultAsync() triggers the actual SQL

public async Task<List<Product>> SearchAsync(
    string? name, decimal? minPrice, bool? isActive)
{
    // Start with base query – no SQL executed yet
    IQueryable<Product> query = _context.Products.AsNoTracking();

    // Conditionally add filters – still no SQL
    if (!string.IsNullOrEmpty(name))
        query = query.Where(p => p.Name.Contains(name));

    if (minPrice.HasValue)
        query = query.Where(p => p.Price >= minPrice.Value);
}
```

```
if (isActive.HasValue)
    query = query.Where(p => p.IsActive == isActive.Value);

// NOW execute – ONE SQL query with all filters combined
return await query
    .OrderBy(p => p.Name)
    .ToListAsync();
}
```

Lesson 7 — Complete Real-World Service Layer

Putting everything together with proper repository + service:

```
// Services/ProductService.cs – complete production-style
using MyApi.Data;
using MyApi.DTOS;
using MyApi.Models;
using Microsoft.EntityFrameworkCore;

namespace MyApi.Services
{
    public class ProductService : IProductService
    {
        private readonly AppDbContext _context;
        private readonly ILogger<ProductService> _logger;

        public ProductService(AppDbContext context,
                               ILogger<ProductService> logger)
        {
            _context = context;
        }
    }
}
```

```

        _logger = logger;
    }

    // — GET ALL with Pagination —————
    public async Task<PagedResult<ProductResponseDto>>
    GetAllAsync(
        int page = 1, int pageSize = 10)
    {
        var skip = (page - 1) * pageSize;

        var total = await _context.Products
            .Where(p => p.IsActive)
            .CountAsync();

        var products = await _context.Products
            .Where(p => p.IsActive)
            .Include(p => p.Category)
            .OrderBy(p => p.Name)
            .Skip(skip)
            .Take(pageSize)
            .Select(p => MapToDto(p))
            .AsNoTracking()
            .ToListAsync();

        return new PagedResult<ProductResponseDto>
        {
            Data = products,
            TotalCount = total,
            PageNumber = page,
            PageSize = pageSize,
            TotalPages = (int)Math.Ceiling(total /
(double)pageSize)
        };
    }

```

```

// — GET BY ID —————
public async Task<ProductResponseDto?> GetByIdAsync(int
id)
{
    var product = await _context.Products
        .Include(p => p.Category)
        .AsNoTracking()
        .FirstOrDefaultAsync(p => p.Id == id);

    return product == null ? null : MapToDto(product);
}

// — CREATE —————
public async Task<ProductResponseDto>
CreateAsync(CreateProductDto dto)
{
    // Check category exists
    var categoryExists = await _context.Categories
        .AnyAsync(c => c.Id == dto.CategoryId);

    if (!categoryExists)
        throw new KeyNotFoundException(
            $"Category {dto.CategoryId} not found.");

    var product = new Product
    {
        Name          = dto.Name,
        Description    = dto.Description,
        Price          = dto.Price,
        Stock          = dto.Stock,
        CategoryId     = dto.CategoryId,
        IsActive       = true,
        CreatedAt      = DateTime.UtcNow
    };
}

```

```

        _context.Products.Add(product);
        await _context.SaveChangesAsync();

        _logger.LogInformation("Product created: {Id} -
{Name}",
                                product.Id, product.Name);

        // Reload with category included
        await _context.Entry(product)
            .Reference(p => p.Category)
            .LoadAsync();

        return MapToDto(product);
    }

    // — UPDATE —————
    public async Task<ProductResponseDto?> UpdateAsync(
        int id, UpdateProductDto dto)
    {
        var product = await _context.Products
            .FirstOrDefaultAsync(p => p.Id == id);

        if (product == null) return null;

        // Update only provided fields
        product.Name = dto.Name;
        product.Description = dto.Description;
        product.Price = dto.Price;
        product.Stock = dto.Stock;
        product.IsActive = dto.IsActive;

        // EF Core detects changes automatically (tracking)
        await _context.SaveChangesAsync();

        _logger.LogInformation("Product updated: {Id}", id);
    }

```

```

        await _context.Entry(product)
            .Reference(p => p.Category)
            .LoadAsync();

        return MapToDto(product);
    }

    // — DELETE (Soft Delete) —————
    public async Task<bool> DeleteAsync(int id)
    {
        var product = await _context.Products.FindAsync(id);
        if (product == null) return false;

        // Soft delete – don't actually remove from DB
        // Just mark as inactive (common in enterprise apps)
        product.IsActive = false;
        await _context.SaveChangesAsync();

        _logger.LogInformation("Product soft-deleted: {Id}",
id);

        return true;
    }

    // — SEARCH —————
    public async Task<List<ProductResponseDto>> SearchAsync(
        string? name, decimal? minPrice, decimal? maxPrice,
        int? categoryId)
    {
        var query = _context.Products
            .Include(p => p.Category)
            .AsNoTracking()
            .AsQueryable();

        if (!string.IsNullOrEmpty(name))

```



```

        query = query.Where(p =>
            p.Name.Contains(name) ||
            p.Description.Contains(name));

        if (minPrice.HasValue)
            query = query.Where(p => p.Price >=
minPrice.Value);

        if (maxPrice.HasValue)
            query = query.Where(p => p.Price <=
maxPrice.Value);

        if (categoryId.HasValue)
            query = query.Where(p => p.CategoryId ==
categoryId.Value);

        query = query.Where(p => p.IsActive);

        return await query
            .OrderBy(p => p.Price)
            .Select(p => MapToDto(p))
            .ToListAsync();
    }

    // — MAPPING —————
    private static ProductResponseDto MapToDto(Product p) =>
new()
    {
        Id            = p.Id,
        Name          = p.Name,
        Description    = p.Description,
        Price         = p.Price,
        Stock         = p.Stock,
        IsActive      = p.IsActive,
        CategoryId    = p.CategoryId,
    }
}

```

```

        CategoryName = p.Category?.Name ?? "Unknown",
        StockStatus   = p.Stock == 0 ? "Out of Stock"
                      : p.Stock <= 5 ? "Low Stock"
                      : "In Stock",
        CreatedAt     = p.CreatedAt
    };
}
}

```

Register Everything in Program.cs

```

// Program.cs – complete setup
using MyApi.Data;
using MyApi.Middleware;
using MyApi.Repositories;
using MyApi.Services;
using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);

// — Services —
builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

// Database
builder.Services.AddDbContext<AppDbContext>(options =>
{
    options.UseSqlServer(

builder.Configuration.GetConnectionString("DefaultConnection"));

```

```

    // Show generated SQL in development logs
    if (builder.Environment.IsDevelopment())
        options.EnableSensitiveDataLogging()
            .LogTo(Console.WriteLine, LogLevel.Information);
});

// Your services – Scoped (per request) is correct for DB-related
// services
builder.Services.AddScoped<IProductRepository, ProductRepository>
();
builder.Services.AddScoped<IProductService, ProductService>();

// — Build —————
var app = builder.Build();

// — Auto-migrate on startup —————
using (var scope = app.Services.CreateScope())
{
    var db =
scope.ServiceProvider.GetRequiredService<AppDbContext>();
    db.Database.Migrate();
}

// — Middleware —————
app.UseGlobalExceptionHandler();

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();

```

```
app.Run();
```

Complete Project Structure for Stage 4

```
MyApi/
|
|— Controllers/
|   └─ ProductController.cs
|
|
|— Data/
|   └─ ApplicationDbContext.cs           ← DbContext lives here
|
|
|— DTOs/
|   └─ CreateProductDto.cs
|   └─ UpdateProductDto.cs
|   └─ ProductResponseDto.cs
|
|
|— Middleware/
|   └─ GlobalExceptionMiddleware.cs
|
|
|— Migrations/                          ← Auto-generated, don't edit
|   └─ 20260220_InitialCreate.cs
|   └─ ApplicationDbContextModelSnapshot.cs
|
|
|— Models/
|   └─ Product.cs
|   └─ Category.cs
|   └─ Order.cs
|   └─ OrderItem.cs
|
|— Repositories/
```

```
|   |─ Interfaces/
|   |   └─ IRepository.cs
|   └─ ProductRepository.cs
|
|─ Services/
|   └─ Interfaces/
|       └─ IProductService.cs
|       └─ ProductService.cs
|
|─ Common/
|   └─ PagedResult.cs           ← shared result wrapper
|
|─ appsettings.json
|─ appsettings.Development.json
└─ Program.cs
```

Stage 4 Interview Questions

Q1: What is EF Core and what problem does it solve?

EF Core is Microsoft's ORM that lets you work with databases using C# instead of raw SQL. It eliminates manual SQL writing, handles DB connections, maps results to objects, and provides type safety.

Q2: What is DbContext?

It's the bridge between your app and database. It manages DB connections, tracks entity changes, translates LINQ to SQL, and executes queries through DbSet properties.

Q3: What is Code First approach?

You write C# entity classes first, then EF Core generates the database schema from them using migrations.

Q4: What are migrations?

Migrations are version-controlled database schema changes. Each migration captures what changed in your model and generates SQL to apply or reverse those changes.

Q5: What is the difference between eager and lazy loading?

Eager loading uses `Include()` to load related data in one JOIN query. Lazy loading auto-loads related data when accessed, causing multiple queries. Eager is preferred to avoid N+1 query problem.

Q6: What is `AsNoTracking` and when to use it?

It tells EF Core not to watch the returned objects for changes. Use it on all read-only GET operations for better performance and lower memory usage.

Q7: What is the N+1 query problem?

Loading a list of N entities then making 1 additional query per entity to load related data. Total = N+1 queries. Solved by using `Include()` for eager loading.

✅ Stage 4 Complete — What You Learned

- ✅ What ORM **is and** why it **exists**
- ✅ DbContext – purpose, setup, configuration
- ✅ Entity classes – mapping C# **to** DB tables
- ✅ Fluent API – configuring relationships **and** constraints
- ✅ Code **First** approach
- ✅ Migrations – **create**, apply, revert
- ✅ **Full** CRUD – **Add**, Find, **Update**, Remove, SaveChanges
- ✅ Eager Loading – `Include()` **and** `ThenInclude()`
- ✅ Lazy Loading – virtual properties (**and** why **to** avoid)
- ✅ Explicit Loading – `Reference()` **and** `Collection()`

- ✓ Tracking vs NoTracking – **when to use each**
- ✓ Performance – **Select** projection, pagination, bulk ops
- ✓ IQueryable – deferred execution **pattern**
- ✓ Repository **Pattern** – **real** project structure
- ✓ Service Layer – complete production-style implementation

Reply **"next"** and we go straight into **Stage 5 — Authentication & Security** — JWT tokens, OAuth 2.0, and Microsoft Entra ID (Azure AD) exactly how enterprise projects use it.



20/02/2026, 01:05:02

 Do it stage 3 in simple proper short but cover all

20/02/2026, 01:12:49

Thought process: Strategized comprehensive SQL Server Database coverage approach.

The user wants Stage 3 - SQL Server Database covered in a simple, proper, short but complete way.

Stage 3 — SQL Server & Database Knowledge

Complete Guide — Simple, Proper, All Covered

Why Does a Backend Developer Need SQL?

Even though EF Core generates SQL for you, you **must understand SQL** because:

- ✓ Debugging slow queries **in** production
- ✓ Writing stored procedures
- ✓ Understanding what EF Core generates
- ✓ Direct DB fixes **when** needed
- ✓ **Every** interview asks **SQL** questions

Setup — SQL Server Locally

1. Download: **SQL** Server Express (**free**)
<https://www.microsoft.com/sql-server/sql-server-downloads>
2. Download: **SQL** Server Management Studio (SSMS) – **free** GUI tool
<https://aka.ms/ssmsfullsetup>
3. **Connect using**: localhost **or** (localdb)\mssqllocaldb

Create Database & Tables First

```
-- Create database
CREATE DATABASE ShopDb;

-- Use it
USE ShopDb;

-- Create Categories table
CREATE TABLE Categories (
    Id          INT PRIMARY KEY IDENTITY(1,1),  -- auto increment
    Name        NVARCHAR(50) NOT NULL,
    Description NVARCHAR(200) NULL
);

-- Create Products table
CREATE TABLE Products (
    Id          INT PRIMARY KEY IDENTITY(1,1),
    Name        NVARCHAR(100) NOT NULL,
    Description NVARCHAR(500) NULL,
    Price       DECIMAL(18,2) NOT NULL,
    Stock       INT           NOT NULL DEFAULT 0,
    IsActive    BIT           NOT NULL DEFAULT 1,  -- 1=true,
```

```

0=false

    CreatedAt    DATETIME2      NOT NULL DEFAULT GETUTCDATE(),
    CategoryId   INT              NOT NULL,

    -- Foreign key
    CONSTRAINT FK_Products_Categories
        FOREIGN KEY (CategoryId)
        REFERENCES Categories(Id)
);

-- Create Orders table
CREATE TABLE Orders (
    Id            INT PRIMARY KEY IDENTITY(1,1),
    CustomerName  NVARCHAR(100)  NOT NULL,
    CustomerEmail NVARCHAR(150)  NOT NULL,
    TotalAmount   DECIMAL(18,2)  NOT NULL,
    Status        NVARCHAR(20)   NOT NULL DEFAULT 'Pending',
    OrderDate     DATETIME2      NOT NULL DEFAULT GETUTCDATE()
);

-- Create OrderItems table
CREATE TABLE OrderItems (
    Id            INT PRIMARY KEY IDENTITY(1,1),
    OrderId       INT              NOT NULL,
    ProductId     INT              NOT NULL,
    Quantity      INT              NOT NULL,
    UnitPrice     DECIMAL(18,2)  NOT NULL,

    CONSTRAINT FK_OrderItems_Orders
        FOREIGN KEY (OrderId) REFERENCES Orders(Id),

    CONSTRAINT FK_OrderItems_Products
        FOREIGN KEY (ProductId) REFERENCES Products(Id)
);

```

Insert Sample Data

```
-- Insert categories
INSERT INTO Categories (Name, Description) VALUES
('Electronics', 'Electronic devices and gadgets'),
('Furniture', 'Home and office furniture'),
('Clothing', 'Clothes and accessories');

-- Insert products
INSERT INTO Products (Name, Price, Stock, IsActive, CategoryId)
VALUES
('Laptop', 999.99, 10, 1, 1),
('Phone', 599.99, 25, 1, 1),
('Monitor', 399.99, 8, 1, 1),
('Keyboard', 79.99, 50, 1, 1),
('Desk', 299.99, 5, 1, 2),
('Chair', 199.99, 12, 1, 2),
('T-Shirt', 29.99, 100, 1, 3),
('Jacket', 89.99, 30, 0, 3); -- IsActive=0 means inactive

-- Insert orders
INSERT INTO Orders (CustomerName, CustomerEmail, TotalAmount,
Status) VALUES
('Ali Ahmed', 'ali@email.com', 1599.98, 'Confirmed'),
('Sara Khan', 'sara@email.com', 399.99, 'Pending'),
('John Smith', 'john@email.com', 279.98, 'Shipped');

-- Insert order items
INSERT INTO OrderItems (OrderId, ProductId, Quantity, UnitPrice)
VALUES
(1, 1, 1, 999.99), -- Order 1: 1 Laptop
(1, 2, 1, 599.99), -- Order 1: 1 Phone
(2, 3, 1, 399.99), -- Order 2: 1 Monitor
```

```
(3, 7, 2, 29.99), -- Order 3: 2 T-Shirts
(3, 4, 1, 79.99); -- Order 3: 1 Keyboard
```



Lesson 1 — Basic Queries

SELECT

```
-- Get everything
SELECT * FROM Products;

-- Get specific columns only (always prefer this over *)
SELECT Id, Name, Price, Stock FROM Products;

-- Calculated column
SELECT
  Id,
  Name,
  Price,
  Stock,
  Price * Stock AS TotalInventoryValue -- calculated column
FROM Products;

-- Rename columns with alias
SELECT
  Id AS ProductId,
  Name AS ProductName,
  Price AS SalePrice
FROM Products;
```

WHERE — Filtering

```
-- Basic filter
SELECT * FROM Products WHERE IsActive = 1;

-- Multiple conditions
SELECT * FROM Products
WHERE IsActive = 1
  AND Price > 100
  AND Stock > 0;

-- OR condition
SELECT * FROM Products
WHERE CategoryId = 1
  OR CategoryId = 2;

-- IN — cleaner way for multiple OR on same column
SELECT * FROM Products
WHERE CategoryId IN (1, 2);

-- NOT IN
SELECT * FROM Products
WHERE CategoryId NOT IN (3);

-- BETWEEN — inclusive on both ends
SELECT * FROM Products
WHERE Price BETWEEN 100 AND 500;

-- LIKE — pattern matching
SELECT * FROM Products WHERE Name LIKE 'L%';      -- starts with
L
SELECT * FROM Products WHERE Name LIKE '%op%';    -- contains
"op"
SELECT * FROM Products WHERE Name LIKE '%e';      -- ends with e
```

```
-- NULL checks
SELECT * FROM Products WHERE Description IS NULL;
SELECT * FROM Products WHERE Description IS NOT NULL;
```

ORDER BY — Sorting

```
-- Ascending (default)
SELECT * FROM Products ORDER BY Price ASC;

-- Descending
SELECT * FROM Products ORDER BY Price DESC;

-- Multiple columns – sort by category, then by price within
category
SELECT * FROM Products
ORDER BY CategoryId ASC, Price DESC;

-- Sort by calculated value
SELECT Id, Name, Price, Stock,
       Price * Stock AS InventoryValue
FROM Products
ORDER BY Price * Stock DESC;
```

TOP — Limit Results

```
-- Get top 5 most expensive products
SELECT TOP 5 Name, Price
FROM Products
```

```
ORDER BY Price DESC;
```

```
-- Get top 10 percent
```

```
SELECT TOP 10 PERCENT Name, Price  
FROM Products  
ORDER BY Price DESC;
```

DISTINCT — Remove Duplicates

```
-- Unique categories used by products
```

```
SELECT DISTINCT CategoryId FROM Products;
```

```
-- Unique statuses in orders
```

```
SELECT DISTINCT Status FROM Orders;
```

Lesson 2 — JOINS

JOINS are the **most important SQL concept** for a backend developer. They combine data from multiple tables.

INNER JOIN — Only Matching Rows

```
-- Get products WITH their category name
```

```
SELECT  
    p.Id,
```

```

    p.Name          AS ProductName,
    p.Price,
    c.Name          AS CategoryName
FROM Products p
INNER JOIN Categories c ON p.CategoryId = c.Id
WHERE p.IsActive = 1
ORDER BY c.Name, p.Price;

```

Result:

Id	ProductName	Price	CategoryName
1	Laptop	999.99	Electronics
2	Phone	599.99	Electronics
3	Monitor	399.99	Electronics
5	Desk	299.99	Furniture
6	Chair	199.99	Furniture
7	T-Shirt	29.99	Clothing

Only rows that have a matching CategoryId in both tables appear.

LEFT JOIN — All Left Rows + Matching Right

```

-- Get ALL categories, show their products (even if category has
no products)
SELECT
    c.Name          AS CategoryName,
    p.Name          AS ProductName,
    p.Price
FROM Categories c
LEFT JOIN Products p ON c.Id = p.CategoryId
ORDER BY c.Name;

```


If a category has no products, it still appears with NULL in product columns.

Multiple JOINS — Real Query

```
-- Full order details: order + customer + product names
```

```
SELECT
```

```
    o.Id           AS OrderId,  
    o.CustomerName,  
    o.Status,  
    p.Name         AS ProductName,  
    oi.Quantity,  
    oi.UnitPrice,  
    oi.Quantity * oi.UnitPrice AS LineTotal,  
    o.OrderDate
```

```
FROM Orders o
```

```
INNER JOIN OrderItems oi ON o.Id      = oi.OrderId
```

```
INNER JOIN Products  p  ON oi.ProductId = p.Id
```

```
ORDER BY o.Id, p.Name;
```

OrderId	CustomerName	Status	ProductName	Qty	UnitPrice	LineTotal
1	Ali Ahmed	Confirmed	Laptop	1	999.99	999.99
1	Ali Ahmed	Confirmed	Phone	1	599.99	599.99
2	Sara Khan	Pending	Monitor	1	399.99	399.99
3	John Smith	Shipped	Keyboard	1	79.99	79.99
3	John Smith	Shipped	T-Shirt	2	29.99	59.98

JOIN Types Summary

JOIN Type	What it returns
INNER JOIN	Only rows that match in BOTH tables
LEFT JOIN	All left rows + matching right (NULL if none)
RIGHT JOIN	All right rows + matching left (NULL if none)
FULL JOIN	All rows from both tables

Most commonly used: INNER JOIN and LEFT JOIN

Lesson 3 — GROUP BY & Aggregates

Aggregate Functions

-- COUNT — how many rows

```
SELECT COUNT(*) AS TotalProducts FROM Products;
```

```
SELECT COUNT(*) AS ActiveProducts FROM Products WHERE IsActive = 1;
```

-- SUM — add up values

```
SELECT SUM(TotalAmount) AS TotalRevenue FROM Orders;
```

-- AVG — average

```
SELECT AVG(Price) AS AveragePrice FROM Products;
```

```
-- MIN / MAX
SELECT MIN(Price) AS CheapestPrice,
       MAX(Price) AS MostExpensivePrice
FROM Products;
```

GROUP BY — Aggregate Per Group

```
-- Product count per category
SELECT
    c.Name          AS CategoryName,
    COUNT(p.Id)     AS ProductCount,
    AVG(p.Price)    AS AveragePrice,
    MIN(p.Price)    AS LowestPrice,
    MAX(p.Price)    AS HighestPrice
FROM Categories c
LEFT JOIN Products p ON c.Id = p.CategoryId
GROUP BY c.Id, c.Name
ORDER BY ProductCount DESC;
```

CategoryName	ProductCount	AveragePrice	LowestPrice	HighestPrice
Electronics	4	519.99	79.99	999.99
Clothing	2	59.99	29.99	89.99
Furniture	2	249.99	199.99	299.99

HAVING — Filter After GROUP BY

```
-- WHERE filters BEFORE grouping
-- HAVING filters AFTER grouping

-- Categories that have MORE than 2 products
SELECT
    c.Name          AS CategoryName,
    COUNT(p.Id)     AS ProductCount
FROM Categories c
LEFT JOIN Products p ON c.Id = p.CategoryId
GROUP BY c.Id, c.Name
HAVING COUNT(p.Id) > 2;

-- Categories where average price is above 200
SELECT
    c.Name          AS CategoryName,
    AVG(p.Price)    AS AvgPrice
FROM Categories c
INNER JOIN Products p ON c.Id = p.CategoryId
GROUP BY c.Id, c.Name
HAVING AVG(p.Price) > 200
ORDER BY AvgPrice DESC;
```



Lesson 4 — Advanced SQL

Indexes — Make Queries Fast

```
-- Without index: SQL Server scans EVERY row (slow for large tables)
-- With index: SQL Server jumps directly to the right rows (fast)
```



```

-- Create index on single column (most common)
CREATE INDEX IX_Products_CategoryId
ON Products(CategoryId);

-- Index on column often used in WHERE
CREATE INDEX IX_Products_IsActive
ON Products(IsActive);

-- Composite index – when you always filter by two columns
together
CREATE INDEX IX_Products_Category_Active
ON Products(CategoryId, IsActive);

-- Unique index – enforces uniqueness + faster lookups
CREATE UNIQUE INDEX IX_Products_Name
ON Products(Name);

-- View all indexes on a table
SELECT * FROM sys.indexes WHERE object_id =
OBJECT_ID('Products');

-- Drop an index
DROP INDEX IX_Products_CategoryId ON Products;

```

When to add indexes:

- Columns used in **WHERE** clauses often
- Columns used in **JOIN ON** conditions
- Columns used in **ORDER BY** often
- Foreign key columns

When NOT to add indexes:

- Small tables (full scan is fine)
- Columns rarely searched

- Tables with very frequent INSERT/UPDATE (indexes slow down writes)

Execution Plan — See How SQL Runs Your Query

```
-- Add this before your query to see execution plan
SET STATISTICS IO ON;
SET STATISTICS TIME ON;

SELECT p.Name, c.Name AS Category
FROM Products p
INNER JOIN Categories c ON p.CategoryId = c.Id
WHERE p.Price > 200;

-- In SSMS: press Ctrl+M before running
-- Shows you: Table Scan (bad) vs Index Seek (good)
```

Stored Procedures

"A saved SQL script stored in the database. Called by name from C# or SSMS."

```
-- Create stored procedure – get products by category
CREATE PROCEDURE sp_GetProductsByCategory
    @CategoryId INT,
    @MinPrice    DECIMAL(18,2) = 0 -- optional parameter with
default
AS
BEGIN
    SET NOCOUNT ON; -- don't return rows affected count
```

```

SELECT
    p.Id,
    p.Name,
    p.Price,
    p.Stock,
    c.Name AS CategoryName
FROM Products p
INNER JOIN Categories c ON p.CategoryId = c.Id
WHERE p.CategoryId = @CategoryId
    AND p.Price >= @MinPrice
    AND p.IsActive = 1
ORDER BY p.Price;
END;

-- Execute it
EXEC sp_GetProductsByCategory @CategoryId = 1;
EXEC sp_GetProductsByCategory @CategoryId = 1, @MinPrice = 100;

```

```

-- Stored procedure with INSERT + return new ID
CREATE PROCEDURE sp_CreateProduct
    @Name          NVARCHAR(100),
    @Price          DECIMAL(18,2),
    @Stock          INT,
    @CategoryId     INT,
    @NewId          INT OUTPUT -- output parameter
AS
BEGIN
    SET NOCOUNT ON;

    INSERT INTO Products (Name, Price, Stock, IsActive,
CategoryId, CreatedAt)
    VALUES (@Name, @Price, @Stock, 1, @CategoryId, GETUTCDATE());

    SET @NewId = SCOPE_IDENTITY(); -- get the auto-generated ID
END;

```

```
-- Execute with output
DECLARE @NewProductId INT;

EXEC sp_CreateProduct
    @Name          = 'New Laptop',
    @Price         = 1299.99,
    @Stock         = 5,
    @CategoryId    = 1,
    @NewId         = @NewProductId OUTPUT;

SELECT @NewProductId AS 'New Product ID';
```

Transactions — All or Nothing

```
-- Transaction ensures multiple operations succeed or all fail
together

BEGIN TRANSACTION;

BEGIN TRY

    -- Step 1: Create order
    INSERT INTO Orders (CustomerName, CustomerEmail, TotalAmount,
Status)
    VALUES ('New Customer', 'new@email.com', 999.99,
'Confirmed');

    DECLARE @OrderId INT = SCOPE_IDENTITY();

    -- Step 2: Add order item
    INSERT INTO OrderItems (OrderId, ProductId, Quantity,
```



```

UnitPrice)
    VALUES (@OrderId, 1, 1, 999.99);

-- Step 3: Reduce stock
UPDATE Products
SET Stock = Stock - 1
WHERE Id = 1;

-- Check stock didn't go negative
IF (SELECT Stock FROM Products WHERE Id = 1) < 0
BEGIN
    THROW 50001, 'Insufficient stock.', 1;
END

-- All steps succeeded – save everything
COMMIT TRANSACTION;
PRINT 'Order placed successfully!';

END TRY
BEGIN CATCH

    -- Something failed – undo EVERYTHING
    ROLLBACK TRANSACTION;
    PRINT 'Error: ' + ERROR_MESSAGE();

END CATCH;

```



Lesson 5 — Normalization

"Organizing database tables to reduce repetition and improve data quality."

Bad Design — No Normalization

Orders Table (bad):

	Id	Customer	CustomerEmail	Product	Category	Price
	1	Ali	ali@email.com	Laptop	Electronics	999.99
	2	Ali	ali@email.com	Phone	Electronics	599.99
	3	Sara	sara@email.com	Monitor	Electronics	399.99

Problems:

- Ali's email repeated – if it changes, update multiple rows
- "Electronics" repeated – inconsistency risk
- No good way to store multiple products per order

Good Design — Normalized

Categories:

Id	Name
1	Electronics
2	Furniture

Products:

Id	Name	CatId
1	Laptop	1
2	Phone	1

Orders:

	Id	Name	Email
	1	Ali	ali@
	2	Sara	sara@

OrderItems (links Orders ↔ Products):

Id	OrderId	ProductId	Qty
1	1	1	1
2	1	2	2
3	2	1	1

No repetition. Change Ali's email in one place. Clean.

Normalization Forms (Simple Explanation)

1NF (First Normal Form):

- Each column has ONE value (no comma-separated lists in a cell)
- Each row is unique (has a primary key)

2NF (Second Normal Form):

- Meet 1NF
- Every column depends on the WHOLE primary key (not part of it)

3NF (Third Normal Form):

- Meet 2NF
- Columns depend ONLY on the primary key, not on other columns
- Example: if you store CategoryId, don't ALSO store CategoryName (CategoryId depends on CategoryId, not on the row's PK)

In practice: Follow 3NF for most tables. That's what we did with our Products/Categories design above.



Lesson 6 — ACID Properties

"The rules that guarantee database transactions are reliable."

ACID Explained Simply

A — Atomicity

"All or nothing"

Either ALL operations **in** a transaction succeed,
or NONE **of** them are saved.

Example: Transfer \$100 **from** Ali **to** Sara

→ Deduct \$100 **from** Ali 

→ Add \$100 **to** Sara  (fails)

→ Atomic: Ali's \$100 *is returned too. No money lost.*

C — Consistency

"Data always stays valid"

A transaction can only bring database **from** one valid
state **to** another valid state.

Example: Product stock cannot go below 0.

If a transaction would cause Stock = -1,
it **is** rejected. Constraint enforced.

I — Isolation

"Transactions don't interfere with each other"

Two transactions running at the same time don't



see *each* other's *incomplete changes*.

Example: Ali *and* Sara both *try to* buy the last item.
Only one succeeds. The other sees *"out of stock"*.
They don't *both think they got it*.

D – Durability

"Once committed, it's permanent"

After a transaction *is* committed (saved),
it survives crashes, power failures, restarts.

Example: *Order* confirmed at *3:00PM*.
Server crashes at *3:01PM*.
Order is still there after restart.

Quick SQL Practice Queries (All Concepts Together)

```
-- Q1: Show each category with product count and total stock
value
SELECT
    c.Name                                AS Category,
    COUNT(p.Id)                          AS ProductCount,
    SUM(p.Stock)                          AS TotalStock,
    SUM(p.Price * p.Stock)                AS TotalInventoryValue,
    AVG(p.Price)                          AS AvgPrice
FROM Categories c
LEFT JOIN Products p ON c.Id = p.CategoryId AND p.IsActive = 1
GROUP BY c.Id, c.Name
ORDER BY TotalInventoryValue DESC;
```

-- Q2: Find customers who spent more than 500 total

```
SELECT
    o.CustomerName,
    o.CustomerEmail,
    COUNT(o.Id)           AS OrderCount,
    SUM(o.TotalAmount)    AS TotalSpent
FROM Orders o
GROUP BY o.CustomerName, o.CustomerEmail
HAVING SUM(o.TotalAmount) > 500
ORDER BY TotalSpent DESC;
```

-- Q3: Products never ordered

```
SELECT p.Id, p.Name, p.Price
FROM Products p
LEFT JOIN OrderItems oi ON p.Id = oi.ProductId
WHERE oi.Id IS NULL;
```

-- Q4: Top 3 best-selling products by quantity

```
SELECT TOP 3
    p.Name,
    SUM(oi.Quantity)           AS TotalSold,
    SUM(oi.Quantity * oi.UnitPrice) AS Revenue
FROM Products p
INNER JOIN OrderItems oi ON p.Id = oi.ProductId
GROUP BY p.Id, p.Name
ORDER BY TotalSold DESC;
```

-- Q5: Orders with full details in one query

```
SELECT
    o.Id           AS OrderId,
    o.CustomerName,
    o.Status,
    o.OrderDate,
    COUNT(oi.Id)           AS ItemCount,
    SUM(oi.Quantity * oi.UnitPrice) AS OrderTotal
```

```
FROM Orders o
INNER JOIN OrderItems oi ON o.Id = oi.OrderId
GROUP BY o.Id, o.CustomerName, o.Status, o.OrderDate
ORDER BY o.OrderDate DESC;
```

Interview Questions

Q1: What is the difference between WHERE and HAVING?

WHERE filters rows before grouping. HAVING filters groups after GROUP BY. You can't use aggregate functions in WHERE.

Q2: What is a JOIN and name the types?

JOIN combines rows from two tables. Types: INNER JOIN (only matches), LEFT JOIN (all left + matches), RIGHT JOIN (all right + matches), FULL JOIN (all rows both sides).

Q3: What is an index and why use it?

An index is a data structure that speeds up data retrieval. Like a book index — instead of scanning every page, you jump directly to the right page. Use on frequently searched/joined columns.

Q4: What is a stored procedure?

A named, saved SQL script stored in the database. Reusable, faster (pre-compiled), and keeps SQL logic in the database layer.

Q5: Explain ACID with a real example?

Bank transfer — Atomicity: both debit and credit succeed or both fail. Consistency: balance never goes negative. Isolation: two simultaneous transfers don't interfere. Durability: confirmed transfer survives server crash.

Q6: What is normalization?

Organizing database tables to eliminate data repetition and ensure data integrity. 1NF: atomic values. 2NF: full dependency on PK. 3NF: no transitive dependencies.

✅ Stage 3 Complete — All Covered

- ✅ Database setup — `CREATE TABLE`, constraints, FK
- ✅ `INSERT` — seed data
- ✅ `SELECT` — columns, aliases, calculated columns
- ✅ `WHERE` — `AND`, `OR`, `IN`, `BETWEEN`, `LIKE`, `IS NULL`
- ✅ `ORDER BY` — `ASC`, `DESC`, multiple columns
- ✅ `TOP`, `DISTINCT`
- ✅ `INNER JOIN`, `LEFT JOIN`, multiple JOINS
- ✅ Aggregate functions — `COUNT`, `SUM`, `AVG`, `MIN`, `MAX`
- ✅ `GROUP BY` — grouping results
- ✅ `HAVING` — filtering groups
- ✅ Indexes — create, when to use
- ✅ Execution Plan — how to check query performance
- ✅ Stored Procedures — create, parameters, output
- ✅ Transactions — `BEGIN`, `COMMIT`, `ROLLBACK`, `TRY/CATCH`
- ✅ Normalization — 1NF, 2NF, 3NF
- ✅ ACID — Atomicity, Consistency, Isolation, Durability

Reply **"next"** and we go to **Stage 5 — Authentication & Security** — JWT, OAuth 2.0, and Microsoft Entra ID the way real enterprise projects implement it. 🚀

20/02/2026, 01:14:27

🧑 Gives stage 5 proper in deep cover proper good way in simple terms proepr

20/02/2026, 01:15:43

Thought process: Architected comprehensive security stage covering authentication frameworks.

The user wants Stage 5 - Authentication & Security covered properly, in depth, simple terms, covering JWT, OAuth 2.0, and Microsoft Entra ID.

Stage 5 — Authentication & Security

Complete Guide — JWT, OAuth 2.0, Microsoft Entra ID

Authentication vs Authorization — Know The Difference First

These two words are used every day in enterprise projects. They mean different things:

Authentication = WHO ARE YOU?

"Prove your identity"

Like showing your passport at the airport.

Authorization = WHAT CAN YOU DO?

"What are you allowed to access?"

Like showing your boarding pass for your specific flight.

Example:

User logs in with email + password → Authentication

User tries to access Admin Panel → Authorization

→ Are they an Admin? Yes → Allow

→ Are they a User? Yes → Deny



How Security Works in a Real API

Without security:

Anyone → GET /api/orders → Gets all orders ← DANGEROUS

Anyone → DELETE /api/users/1 → Deletes user ← DANGEROUS

With security:

Request arrives

|



Who are you?
Check token/cookie

← Authentication Middleware

|

— No token → 401 Unauthorized

|



What can you do?
Check roles/claims

← Authorization Middleware



|
|
| — Wrong role → 403 Forbidden
|
▼

| Your Controller | ← Now runs safely

Lesson 1 — JWT Authentication

What Is JWT?

JWT = **J**SON **W**eb **T**oken

"A self-contained token that proves who you are. Sent with every request."

JWT Token Structure

A JWT looks like this:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
.
eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpXVCJ9
.
SflKxwRJSMeKKF2QT4fwpMeJf36P0k6yJV_adQssw5c
```

It has **3 parts separated by dots:**

HEADER.PAYLOAD.SIGNATURE

```
| HEADER (Base64 encoded) |
| { |
|   "alg": "HS256", ← signing algorithm |
|   "typ": "JWT" |
| } |
|-----|
| PAYLOAD (Base64 encoded) – the claims |
| { |
|   "sub": "123", ← user ID (subject) |
|   "name": "Ali Ahmed", ← user name |
|   "email": "ali@x.com", ← user email |
|   "role": "Admin", ← user role |
|   "iat": 1700000000, ← issued at (timestamp) |
|   "exp": 1700003600 ← expires at (timestamp) |
| } |
|-----|
| SIGNATURE |
| HMACSHA256( |
|   base64(header) + "." + base64(payload), |
|   SECRET_KEY |
| ) |
| → Proves token was not tampered with |
```

Important:

- Header + Payload are **encoded, NOT encrypted** — anyone can decode and read them
- NEVER put passwords or sensitive data in JWT payload
- The Signature is what makes it secure — only your server can create/verify it

How JWT Works in a Real API

Step 1 – Login:

Client —POST /api/auth/login—► Server
 { email, password }

Server checks credentials **in** DB

If valid → creates JWT token → returns it

Client ◀— { token: "eyJhbG..." } — Server

Step 2 – Use Token:

Client —GET /api/orders—► Server
 Header: Authorization: Bearer eyJhbG...

Server reads token → validates signature → reads claims
→ knows **who** the user is → allows/denies request

Client ◀— [orders data] — Server

Step 3 – Token Expires:

After 60 minutes (or whatever expiry you **set**)

Token is invalid → client must login again

OR use Refresh Token to get a new access token

Implement JWT From Scratch

Step 1 — Install Package

```
dotnet add package Microsoft.AspNetCore.Authentication.JwtBearer
dotnet add package System.IdentityModel.Tokens.Jwt
```

Step 2 — Configuration

```
// appsettings.json
{
  "JwtSettings": {
    "SecretKey": "my-super-secret-key-minimum-32-characters-
long!!",
    "Issuer": "MyApi",
    "Audience": "MyApiUsers",
    "ExpiryMinutes": 60,
    "RefreshTokenExpiryDays": 7
  }
}
```

```
// Configuration/JwtSettings.cs
namespace MyApi.Configuration
{
  public class JwtSettings
  {
    public string SecretKey { get; set; }
    public string Issuer { get; set; }
    public string Audience { get; set; }
    public int ExpiryMinutes { get; set; }
    public int RefreshTokenExpiryDays { get; set; }
  }
}
```

Step 3 — User & Auth Models

```
// Models/User.cs
namespace MyApi.Models
{
    public class User
    {
        public int Id { get; set; }
        public string FullName { get; set; }
        public string Email { get; set; }
        public string PasswordHash { get; set; } // never store
        plain password
        public string Role { get; set; } // Admin,
        Manager, User
        public bool IsActive { get; set; }
        public DateTime CreatedAt { get; set; }
        public string? RefreshToken { get; set; }
        public DateTime? RefreshTokenExpiry { get; set; }
    }
}
```

```
// DTOs/AuthDtos.cs
using System.ComponentModel.DataAnnotations;

namespace MyApi.DTOs
{
    public class RegisterDto
    {
        [Required]
        [StringLength(100, MinimumLength = 2)]
        public string FullName { get; set; }

        [Required]
        [EmailAddress]
        public string Email { get; set; }
    }
}
```

```

        [Required]
        [StringLength(100, MinimumLength = 6)]
        public string Password { get; set; }
    }

    public class LoginDto
    {
        [Required]
        [EmailAddress]
        public string Email { get; set; }

        [Required]
        public string Password { get; set; }
    }

    public class AuthResponseDto
    {
        public string AccessToken { get; set; }
        public string RefreshToken { get; set; }
        public DateTime ExpiresAt { get; set; }
        public UserInfoDto User { get; set; }
    }

    public class UserInfoDto
    {
        public int Id { get; set; }
        public string FullName { get; set; }
        public string Email { get; set; }
        public string Role { get; set; }
    }

    public class RefreshTokenDto
    {
        [Required]
        public string RefreshToken { get; set; }
    }

```



```
}  
}
```

Step 4 — Token Service

```
// Services/TokenService.cs  
using System.IdentityModel.Tokens.Jwt;  
using System.Security.Claims;  
using System.Security.Cryptography;  
using System.Text;  
using Microsoft.Extensions.Options;  
using Microsoft.IdentityModel.Tokens;  
using MyApi.Configuration;  
using MyApi.Models;  
  
namespace MyApi.Services  
{  
    public interface ITokenService  
    {  
        string GenerateAccessToken(User user);  
        string GenerateRefreshToken();  
        ClaimsPrincipal? ValidateToken(string token);  
    }  
  
    public class TokenService : ITokenService  
    {  
        private readonly JwtSettings _jwtSettings;  
  
        public TokenService(IOptions<JwtSettings> jwtSettings)  
        {  
            _jwtSettings = jwtSettings.Value;  
        }  
  
        // — Generate Access Token —————
```

```

public string GenerateAccessToken(User user)
{
    // Claims = data stored inside the token
    var claims = new List<Claim>
    {
        new Claim(ClaimTypes.NameIdentifier,
user.Id.ToString()),
        new Claim(ClaimTypes.Name,
user.FullName),
        new Claim(ClaimTypes.Email,          user.Email),
        new Claim(ClaimTypes.Role,          user.Role),
        new Claim(JwtRegisteredClaimNames.Jti,
Guid.NewGuid().ToString()),
        new Claim(JwtRegisteredClaimNames.Iat,

DateTimeOffset.UtcNow.ToUnixTimeSeconds().ToString())
    };

    // Create signing key from secret
    var key = new SymmetricSecurityKey(
        Encoding.UTF8.GetBytes(_jwtSettings.SecretKey));

    var credentials = new SigningCredentials(
        key, SecurityAlgorithms.HmacSha256);

    // Build the token
    var token = new JwtSecurityToken(
        issuer:          _jwtSettings.Issuer,
        audience:        _jwtSettings.Audience,
        claims:          claims,
        notBefore:       DateTime.UtcNow,
        expires:         DateTime.UtcNow.AddMinutes(
_jwtSettings.ExpiryMinutes),
        signingCredentials: credentials

```

```

    );

    // Serialize to string
    return new
JwtSecurityTokenHandler().WriteToken(token);
}

// — Generate Refresh Token —————
// Refresh token is just a random secure string
// Stored in DB alongside expiry
public string GenerateRefreshToken()
{
    var randomBytes = new byte[64];
    using var rng = RandomNumberGenerator.Create();
    rng.GetBytes(randomBytes);
    return Convert.ToBase64String(randomBytes);
}

// — Validate Token (for refresh flow) —————
public ClaimsPrincipal? ValidateToken(string token)
{
    var tokenHandler = new JwtSecurityTokenHandler();
    var key =
Encoding.UTF8.GetBytes(_jwtSettings.SecretKey);

    try
    {
        var principal = tokenHandler.ValidateToken(token,
            new TokenValidationParameters
            {
                ValidateIssuerSigningKey = true,
                IssuerSigningKey         = new
SymmetricSecurityKey(key),
                ValidateIssuer           = true,
                ValidIssuer               =

```

```

_jwtSettings.Issuer,
                ValidateAudience            = true,
                ValidAudience                =
_jwtSettings.Audience,
                ValidateLifetime              = false // allow
expired for refresh
            },
            out SecurityToken validatedToken);

        return principal;
    }
    catch
    {
        return null; // invalid token
    }
}
}
}

```

Step 5 — Auth Service (Register + Login + Refresh)

```

// Services/AuthService.cs
using Microsoft.EntityFrameworkCore;
using MyApi.Data;
using MyApi.DTOS;
using MyApi.Models;

namespace MyApi.Services
{
    public interface IAuthService
    {
        Task<AuthResponseDto> RegisterAsync(RegisterDto dto);
        Task<AuthResponseDto> LoginAsync(LoginDto dto);
        Task<AuthResponseDto> RefreshTokenAsync(string

```

```

refreshToken);
    Task RevokeTokenAsync(string refreshToken);
}

public class AuthService : IAuthService
{
    private readonly AppDbContext _context;
    private readonly ITokenService _tokenService;
    private readonly IConfiguration _config;

    public AuthService(AppDbContext context,
                      ITokenService tokenService,
                      IConfiguration config)
    {
        _context      = context;
        _tokenService = tokenService;
        _config       = config;
    }

    // — REGISTER —————
    public async Task<AuthResponseDto>
RegisterAsync(RegisterDto dto)
    {
        // Check email not already taken
        var existingUser = await _context.Users
            .AnyAsync(u => u.Email == dto.Email);

        if (existingUser)
            throw new InvalidOperationException(
                "Email is already registered.");

        // Hash the password – NEVER store plain text
        var passwordHash =
BCrypt.Net.BCrypt.HashPassword(dto.Password);

```

```

        var user = new User
        {
            FullName      = dto.FullName,
            Email          = dto.Email,
            PasswordHash   = passwordHash,
            Role           = "User",           // default role
            IsActive       = true,
            CreatedAt      = DateTime.UtcNow
        };

        _context.Users.Add(user);
        await _context.SaveChangesAsync();

        return await GenerateAuthResponse(user);
    }

    // — LOGIN —
    public async Task<AuthResponseDto> LoginAsync(LoginDto
dto)
    {
        // Find user by email
        var user = await _context.Users
            .FirstOrDefaultAsync(u => u.Email == dto.Email
                                && u.IsActive);

        // Verify password hash
        // Use same error for wrong email OR wrong password
        // → prevents user enumeration attack
        if (user == null ||
            !BCrypt.Net.BCrypt.Verify(dto.Password,
user.PasswordHash))
        {
            throw new UnauthorizedAccessException(
                "Invalid email or password.");
        }
    }

```

```

        return await GenerateAuthResponse(user);
    }

    // — REFRESH TOKEN —————
    public async Task<AuthResponseDto> RefreshTokenAsync(
        string refreshToken)
    {
        // Find user with this refresh token
        var user = await _context.Users
            .FirstOrDefaultAsync(u =>
                u.RefreshToken == refreshToken &&
                u.IsActive);

        // Validate refresh token exists and not expired
        if (user == null ||
            user.RefreshTokenExpiry < DateTime.UtcNow)
        {
            throw new UnauthorizedAccessException(
                "Invalid or expired refresh token.");
        }

        return await GenerateAuthResponse(user);
    }

    // — REVOKE (Logout) —————
    public async Task RevokeTokenAsync(string refreshToken)
    {
        var user = await _context.Users
            .FirstOrDefaultAsync(u => u.RefreshToken ==
refreshToken);

        if (user == null) return;

        // Clear refresh token = logout

```

```

        user.RefreshToken          = null;
        user.RefreshTokenExpiry = null;

        await _context.SaveChangesAsync();
    }

    // — PRIVATE HELPER —
    private async Task<AuthResponseDto>
GenerateAuthResponse(User user)
    {
        var accessToken =
_tokenService.GenerateAccessToken(user);
        var refreshToken =
_tokenService.GenerateRefreshToken();

        var refreshDays = _config
            .GetValue<int>
("JwtSettings:RefreshTokenExpiryDays");

        // Save refresh token to DB
        user.RefreshToken          = refreshToken;
        user.RefreshTokenExpiry =
DateTime.UtcNow.AddDays(refreshDays);

        await _context.SaveChangesAsync();

        return new AuthResponseDto
        {
            AccessToken = accessToken,
            RefreshToken = refreshToken,
            ExpiresAt    = DateTime.UtcNow.AddMinutes(
                _config.GetValue<int>
("JwtSettings:ExpiryMinutes")),
            User = new UserInfoDto
            {

```



```

        Id      = user.Id,
        FullName = user.FullName,
        Email    = user.Email,
        Role     = user.Role
    }
};
}
}
}

```

Step 6 — Auth Controller

```

// Controllers/AuthController.cs
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using MyApi.DTOS;
using MyApi.Services;

namespace MyApi.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class AuthController : ControllerBase
    {
        private readonly IAuthService _authService;

        public AuthController(IAuthService authService)
        {
            _authService = authService;
        }

        // POST api/auth/register
        [HttpPost("register")]
        [AllowAnonymous] // no token needed to register
    }
}

```

```

        public async Task<IActionResult> Register([FromBody]
RegisterDto dto)
        {
            var result = await _authService.RegisterAsync(dto);
            return Ok(result);
        }

        // POST api/auth/login
        [HttpPost("login")]
        [AllowAnonymous]
        public async Task<IActionResult> Login([FromBody]
LoginDto dto)
        {
            var result = await _authService.LoginAsync(dto);
            return Ok(result);
        }

        // POST api/auth/refresh
        [HttpPost("refresh")]
        [AllowAnonymous]
        public async Task<IActionResult> RefreshToken(
            [FromBody] RefreshTokenDto dto)
        {
            var result = await
_authService.RefreshTokenAsync(dto.RefreshToken);
            return Ok(result);
        }

        // POST api/auth/logout
        [HttpPost("logout")]
        [Authorize] // must be logged in to logout
        public async Task<IActionResult> Logout(
            [FromBody] RefreshTokenDto dto)
        {
            await

```

```

_authService.RevokeTokenAsync(dto.RefreshToken);
    return NoContent();
}

// GET api/auth/me – get current user info
[HttpGet("me")]
[Authorize]
public IActionResult GetCurrentUser()
{
    // Claims from JWT token are automatically available
    var userId = User.FindFirst(
System.Security.Claims.ClaimTypes.NameIdentifier)?.Value;
    var email = User.FindFirst(
        System.Security.Claims.ClaimTypes.Email)?.Value;
    var fullName = User.FindFirst(
        System.Security.Claims.ClaimTypes.Name)?.Value;
    var role = User.FindFirst(
        System.Security.Claims.ClaimTypes.Role)?.Value;

    return Ok(new
    {
        Id = userId,
        Email = email,
        FullName = fullName,
        Role = role
    });
}
}
}

```

Step 7 — Protect Your Controllers

```

// Controllers/ProductController.cs
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;

[ApiController]
[Route("api/[controller]")]
[Authorize] // entire controller requires authentication
public class ProductController : ControllerBase
{
    // GET – any authenticated user can read
    [HttpGet]
    [AllowAnonymous] // override controller-level [Authorize] for this one
    public async Task<IActionResult> GetAll()
    {
        // anyone can see products – no login needed
        return Ok(await _productService.GetAllAsync());
    }

    // GET by ID – authenticated only
    [HttpGet("{id}")]
    [Authorize] // must be logged in
    public async Task<IActionResult> GetById(int id)
    {
        var product = await _productService.GetByIdAsync(id);
        if (product == null) return NotFound();
        return Ok(product);
    }

    // POST – only Admins and Managers can create
    [HttpPost]
    [Authorize(Roles = "Admin,Manager")]
    public async Task<IActionResult> Create([FromBody]
CreateProductDto dto)
    {

```

```

        var product = await _productService.CreateAsync(dto);
        return CreatedAtAction(nameof(GetById), new { id =
product.Id }, product);
    }

    // PUT – only Admins and Managers
    [HttpPut("{id}")]
    [Authorize(Roles = "Admin,Manager")]
    public async Task<IActionResult> Update(
        int id, [FromBody] UpdateProductDto dto)
    {
        var product = await _productService.UpdateAsync(id, dto);
        if (product == null) return NotFound();
        return Ok(product);
    }

    // DELETE – only Admins
    [HttpDelete("{id}")]
    [Authorize(Roles = "Admin")]
    public async Task<IActionResult> Delete(int id)
    {
        var deleted = await _productService.DeleteAsync(id);
        if (!deleted) return NotFound();
        return NoContent();
    }
}

```

Step 8 — Register Everything in Program.cs

```

// Program.cs – complete auth setup
using System.Text;
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.IdentityModel.Tokens;
using MyApi.Configuration;

```

```

using MyApi.Services;

var builder = WebApplication.CreateBuilder(args);

// — Bind JWT Settings —————
var jwtSettings = builder.Configuration
    .GetSection("JwtSettings")
    .Get<JwtSettings>();

builder.Services.Configure<JwtSettings>(
    builder.Configuration.GetSection("JwtSettings"));

// — Register Services —————
builder.Services.AddScoped<IAuthService, AuthService>();
builder.Services.AddScoped<ITokenService, TokenService>();

// — JWT Authentication Setup —————
builder.Services.AddAuthentication(options =>
{
    options.DefaultAuthenticateScheme =
JwtBearerDefaults.AuthenticationScheme;
    options.DefaultChallengeScheme =
JwtBearerDefaults.AuthenticationScheme;
})
.AddJwtBearer(options =>
{
    options.TokenValidationParameters = new
TokenValidationParameters
    {
        // Validate that token was signed with our secret key
        ValidateIssuerSigningKey = true,
        IssuerSigningKey = new SymmetricSecurityKey(
            Encoding.UTF8.GetBytes(jwtSettings.SecretKey)),

        // Validate issuer (who created the token)

```

```

ValidateIssuer = true,
ValidIssuer    = jwtSettings.Issuer,

// Validate audience (who the token is for)
ValidateAudience = true,
ValidAudience    = jwtSettings.Audience,

// Validate expiry
ValidateLifetime    = true,
ClockSkew           = TimeSpan.Zero // no grace period
on expiry
};

// Return proper 401 responses
options.Events = new JwtBearerEvents
{
    OnChallenge = context =>
    {
        context.HandleResponse();
        context.Response.StatusCode = 401;
        context.Response.ContentType = "application/json";
        return context.Response.WriteAsync(
            "{ \"error\": \"Unauthorized. Please login.\" }");
    },
    OnForbidden = context =>
    {
        context.Response.StatusCode = 403;
        context.Response.ContentType = "application/json";
        return context.Response.WriteAsync(
            "{ \"error\": \"Forbidden. Insufficient
permissions.\" }");
    }
};
});

```

```
builder.Services.AddAuthorization();
builder.Services.AddControllers();

// — Build & Middleware —————
var app = builder.Build();

app.UseAuthentication(); // who are you? ← must be before
Authorization
app.UseAuthorization(); // what can you do?

app.MapControllers();
app.Run();
```

Refresh Token Flow — Visual

ACCESS TOKEN (short-lived: 60 min):

Used for every API request

Stored in memory on client (not localStorage)

REFRESH TOKEN (long-lived: 7 days):

Used ONLY to get new access token

Stored in HttpOnly cookie or secure storage

Saved in DB so we can revoke it

Normal flow:

1. Login → get accessToken + refreshToken
2. Use accessToken for requests
3. After 60 min → accessToken expires → 401
4. Client sends refreshToken to /api/auth/refresh
5. Server validates refreshToken in DB

6. Server returns **NEW** accessToken + **NEW** refreshToken
7. Client continues normally

Logout flow:

1. Client sends refreshToken **to** /api/auth/logout
2. Server clears refreshToken **from** DB
3. Old refreshToken now invalid forever

Lesson 2 — OAuth 2.0

What Is OAuth 2.0?

"A standard protocol that lets users give your app permission to access their data on another service — without sharing their password."

Real example everyone knows:

You visit a website: "Login with Google"

You click it:

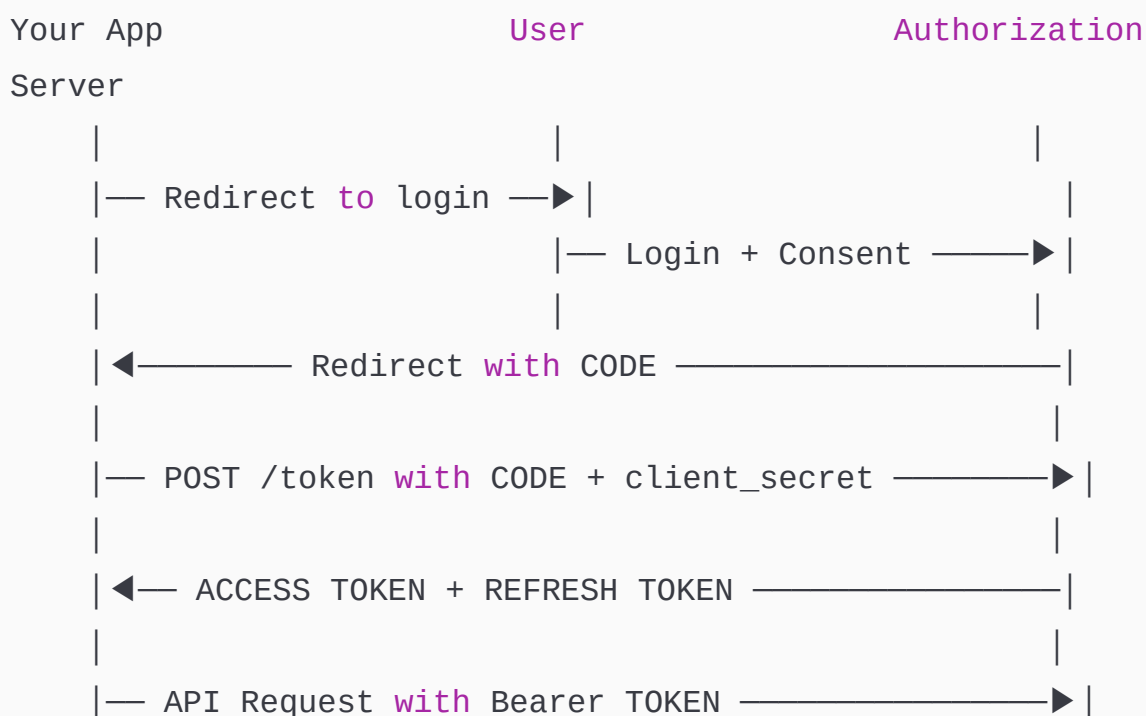
1. You are redirected **to** Google's login page
2. You login **with** YOUR Google credentials
3. Google asks: "This app wants to read your profile. Allow?"
4. You click Allow
5. Google redirects back **to** the app **with** a CODE
6. App exchanges CODE **for** an ACCESS TOKEN **from** Google
7. App uses token **to** read your Google profile
8. App creates your account — you never gave your password **to** the app

The key point: The third-party app never sees your Google password. Google handles authentication. The app just gets permission (authorization).

OAuth 2.0 Key Terms

Resource Owner → The **user** (you)
Client → The app requesting access
Authorization Server → The server that **authenticates** (Google, Microsoft)
Resource Server → The server that holds the **data** (Google APIs)
Access Token → What the app gets to access your data
Scope → What access is **requested** (read email, read profile, etc.)

OAuth 2.0 — Authorization Code Flow (Most Common)



(Resource Server)

| ◀ — Protected Resource Data ————— |

OAuth 2.0 Scopes — Requesting Specific Permissions

Scopes **define** what your app can access:

<code>openid</code>	→ Use OpenID Connect (get user identity)
<code>profile</code>	→ Read user's name , picture
<code>email</code>	→ Read user's email address
<code>offline_access</code>	→ Get refresh token (for long sessions)

Microsoft-**specific**:

<code>User.Read</code>	→ Read user profile from Microsoft
<code>Mail.Read</code>	→ Read user emails
<code>Calendars.Read</code>	→ Read user calendar

Lesson 3 — Microsoft Entra ID (Azure Active Directory)

What Is Microsoft Entra ID?

"Microsoft's cloud-based identity and access management service. Used in almost every enterprise Microsoft project."

Previously called: **Azure Active Directory (Azure AD)** New name (2023): **Microsoft Entra ID**

In real enterprise projects:

- Employees log in with their company Microsoft account
- Your API trusts Microsoft Entra ID as the authority
- No need to manage passwords yourself
- Single Sign-On (SSO) — one login for all company apps

Key Concepts You Must Know

Tenant	→ Your company's Entra ID organization Example: "contoso.onmicrosoft.com" Each company has its own tenant with its own
tenant ID	
App Registration	→ You register your API in Entra ID portal Like telling Entra ID "my app exists" Gets a Client ID (unique app identifier)
Client ID	→ Unique ID for your registered app Like a username for your app
Client Secret	→ Password for your app App uses this to prove its identity to Entra ID
Tenant ID	→ ID of your Entra ID organization
Scope	→ What access your app needs Example: api://your-app-id/Products.Read
RBAC	→ Role Based Access Control

Assign roles to users **in** Entra ID

Your API checks these roles **for** authorization

Step-by-Step: Register App in Azure Portal

1. Go to: `portal.azure.com`
2. Search for: "Microsoft Entra ID"
3. Click "App registrations" → "New registration"
4. Fill in:
 - Name: `My Product API`
 - Supported accounts: `Single tenant (your company only)`
or `Multi-tenant (anyone with Microsoft account)`
 - Redirect URI: `https://localhost:PORT/signin-oidc`
5. Click Register
 - You get: `Application (client) ID`
 - You get: `Directory (tenant) ID`
6. Go to "Certificates & secrets" → New client secret
 - Copy the VALUE immediately (you can't see it again)
7. Go to "Expose an API"
 - Set App ID URI: `api://your-client-id`
 - Add a scope: `Products.Read`
 - Add a scope: `Products.Write`
8. Go to "App roles" → Create a role
 - Value: `Admin`

- Value: Manager
- Value: User

Implement Entra ID Auth in ASP.NET Core

Install Package

```
dotnet add package Microsoft.Identity.Web  
dotnet add package Microsoft.Identity.Web.MicrosoftGraph
```

appsettings.json

```
{  
  "AzureAd": {  
    "Instance": "https://login.microsoftonline.com/",  
    "TenantId": "your-tenant-id-here",  
    "ClientId": "your-client-id-here",  
    "ClientSecret": "your-client-secret-here",  
    "Audience": "api://your-client-id-here"  
  }  
}
```

Program.cs — Entra ID Setup

```
// Program.cs — Microsoft Entra ID Authentication  
using Microsoft.Identity.Web;  
using Microsoft.AspNetCore.Authentication.JwtBearer;  
  
var builder = WebApplication.CreateBuilder(args);
```

```
// — Add Entra ID Authentication —————
// Microsoft handles ALL the JWT validation for you
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationS

.AddMicrosoftIdentityWebApi(builder.Configuration.GetSection("AzureA

builder.Services.AddAuthorization();
builder.Services.AddControllers();

var app = builder.Build();

app.UseAuthentication();
app.UseAuthorization();
app.MapControllers();
app.Run();
```

Protect Controllers with Entra ID

```
// Controllers/ProductController.cs – Entra ID version
using Microsoft.AspNetCore.Authorization;
using Microsoft.Identity.Web.Resource;
using Microsoft.AspNetCore.Mvc;

[ApiController]
[Route("api/[controller]")]
[Authorize] // must have valid Entra ID token
public class ProductController : ControllerBase
{
    // Require specific scope – caller must have Products.Read
    permission

    private static readonly string[] ReadScope = {
        "Products.Read" };

    private static readonly string[] WriteScope = {
        "Products.Write" };
```

```

// GET – requires Products.Read scope
[HttpGet]
[RequiredScope(RequiredScopesConfigurationKey =
"AzureAd:Scopes")]
public async Task<IActionResult> GetAll()
{
    // Verify the scope
    HttpContext.VerifyUserHasAnyAcceptedScope(ReadScope);

    return Ok(await _productService.GetAllAsync());
}

// POST – requires Admin or Manager role
[HttpPost]
[Authorize(Roles = "Admin,Manager")]
public async Task<IActionResult> Create([FromBody]
CreateProductDto dto)
{
    HttpContext.VerifyUserHasAnyAcceptedScope(WriteScope);

    var product = await _productService.CreateAsync(dto);
    return CreatedAtAction(nameof(GetAll), new { id =
product.Id }, product);
}

// DELETE – Admin role only
[HttpDelete("{id}")]
[Authorize(Roles = "Admin")]
public async Task<IActionResult> Delete(int id)
{
    HttpContext.VerifyUserHasAnyAcceptedScope(WriteScope);

    var deleted = await _productService.DeleteAsync(id);
    if (!deleted) return NotFound();
}

```



```
        return NoContent();
    }
}
```

Reading User Info from Entra ID Token

```
// In any controller – reading claims from Entra ID token
[HttpGet("my-info")]
[Authorize]
public IActionResult GetMyInfo()
{
    // Entra ID puts user info in specific claim types
    var userId =
User.FindFirst("http://schemas.microsoft.com/identity/claims/objectid")?.Value;
    var email =
User.FindFirst("preferred_username")?.Value;
    var displayName = User.FindFirst("name")?.Value;
    var tenantId =
User.FindFirst("http://schemas.microsoft.com/identity/claims/tenantid")?.Value;

    // Get all roles assigned in Entra ID
    var roles =
User.FindAll(System.Security.Claims.ClaimTypes.Role)
        .Select(c => c.Value)
        .ToList();

    return Ok(new
    {
        UserId = userId,
        Email = email,
        DisplayName = displayName,
        TenantId = tenantId,
        Roles = roles
    });
}
```

```
});  
}
```

Own JWT vs Entra ID — When To Use Which

Your Own JWT	Microsoft Entra ID
You manage users	Microsoft manages users
You manage passwords	No passwords in your app
Public apps	Enterprise/company internal apps
Startups, SaaS	Corporate projects, Microsoft ecosystem
Full control	SSO, company policies built-in
More code to write	Much less code, Microsoft does the work

Real world:

- Building a public e-commerce site? → Your own JWT
- Building an internal company tool? → Entra ID
- Microsoft outsourcing projects? → Almost always Entra ID

Lesson 4 — Security Best Practices

Password Hashing — Never Store Plain Passwords

```
dotnet add package BCrypt.Net-Next
```

```
// Hashing password on register
var hash = BCrypt.Net.BCrypt.HashPassword("user-plain-password");
// Result: "$2a$11$rT8..." – completely different every time
(salted)

// Verifying on login
bool isValid = BCrypt.Net.BCrypt.Verify("user-plain-password",
hash);
// Returns true if password matches the hash
```

HTTPS — Always Use It

```
// Program.cs
app.UseHttpsRedirection(); // redirects all HTTP to HTTPS

// Never send tokens over plain HTTP
// Tokens are visible in network traffic – HTTPS encrypts the
connection
```

CORS — Control Who Can Call Your API

```
// Program.cs – CORS setup
builder.Services.AddCors(options =>
{
    // Development – allow localhost
```

```

options.AddPolicy("Development", policy =>
{
    policy.WithOrigins(
        "http://localhost:3000",    // React app
        "http://localhost:4200",    // Angular app
        "https://localhost:7001")    // another local
service

        .AllowAnyHeader()
        .AllowAnyMethod()
        .AllowCredentials();
    });

    // Production – only your real frontend
options.AddPolicy("Production", policy =>
{
    policy.WithOrigins(
        "https://myapp.com",
        "https://www.myapp.com")
        .AllowAnyHeader()
        .WithMethods("GET", "POST", "PUT", "DELETE")
        .AllowCredentials();
    });
});

var app = builder.Build();

// Use correct policy per environment
if (app.Environment.IsDevelopment())
    app.UseCors("Development");
else
    app.UseCors("Production");

```

Security Headers — Protect Against Common Attacks

```
// Middleware/SecurityHeadersMiddleware.cs
public class SecurityHeadersMiddleware
{
    private readonly RequestDelegate _next;

    public SecurityHeadersMiddleware(RequestDelegate next) =>
        _next = next;

    public async Task InvokeAsync(HttpContext context)
    {
        // Prevent clickjacking
        context.Response.Headers.Add("X-Frame-Options", "DENY");

        // Prevent MIME type sniffing
        context.Response.Headers.Add("X-Content-Type-Options",
            "nosniff");

        // XSS protection
        context.Response.Headers.Add("X-XSS-Protection", "1;
            mode=block");

        // Don't send referrer info
        context.Response.Headers.Add("Referrer-Policy", "no-
            referrer");

        // Only allow HTTPS for 1 year
        context.Response.Headers.Add(
            "Strict-Transport-Security",
            "max-age=31536000; includeSubDomains");

        await _next(context);
    }
}
```

```
}  
  
}
```

Store Secrets Safely — Never In Code

```
// ❌ NEVER do this – secret in code, visible in git history  
var secretKey = "my-secret-key-12345";  
  
// ✅ In appsettings.json (for non-sensitive settings only)  
// appsettings.json committed to git – don't put secrets here  
  
// ✅ Environment variables (for production)  
// Set in Azure App Service / server config – not in code  
var secretKey =  
Environment.GetEnvironmentVariable("JWT_SECRET_KEY");  
  
// ✅ Azure Key Vault (for enterprise – best way)  
// Secrets stored in Azure, your app fetches them at runtime  
// Nothing sensitive ever in your code or config files
```

```
// Program.cs – Azure Key Vault integration  
using Azure.Identity;  
  
// This pulls secrets from Azure Key Vault automatically  
// Key Vault name set in appsettings.json – not the actual  
secrets  
var keyVaultUrl = builder.Configuration["KeyVaultUrl"];  
  
builder.Configuration.AddAzureKeyVault(  
    new Uri(keyVaultUrl),  
    new DefaultAzureCredential()); // uses managed identity – no
```

secret needed!

// Now secrets from Key Vault are available in IConfiguration as normal

```
var jwtSecret = builder.Configuration["JwtSecretKey"]; // comes from Key Vault
```

Rate Limiting — Prevent Abuse

// Program.cs – built-in rate limiting (ASP.NET Core 7+)

```
using Microsoft.AspNetCore.RateLimiting;
```

```
using System.Threading.RateLimiting;
```

```
builder.Services.AddRateLimiter(options =>
```

```
{
```

// Fixed window – 10 requests per minute per IP

```
options.AddFixedWindowLimiter("api", limiter =>
```

```
{
```

```
    limiter.Window = TimeSpan.FromMinutes(1);
```

```
    limiter.PermitLimit = 10;
```

```
    limiter.QueueLimit = 0; // reject immediately
```

when limit hit

```
    limiter.QueueProcessingOrder =
```

```
QueueProcessingOrder.OldestFirst;
```

```
});
```

// Stricter for login – 5 attempts per minute (brute force protection)

```
options.AddFixedWindowLimiter("auth", limiter =>
```

```
{
```

```
    limiter.Window = TimeSpan.FromMinutes(1);
```

```
    limiter.PermitLimit = 5;
```

```

        limiter.QueueLimit = 0;
    });

    // Response when rate limit hit
    options.OnRejected = async (context, token) =>
    {
        context.HttpContext.Response.StatusCode = 429; // Too
Many Requests
        await context.HttpContext.Response.WriteAsync(
            "Too many requests. Please try again later.", token);
    };
});

var app = builder.Build();
app.UseRateLimiter(); // add to pipeline

```

```

// Apply rate limiting to controller/endpoint
[HttpPost("login")]
[AllowAnonymous]
[EnableRateLimiting("auth")] // strict limit for login
public async Task<IActionResult> Login([FromBody] LoginDto dto)
{
    var result = await _authService.LoginAsync(dto);
    return Ok(result);
}

[HttpGet]
[EnableRateLimiting("api")] // standard limit for API
public async Task<IActionResult> GetAll()
{
    return Ok(await _productService.GetAllAsync());
}

```


Common Security Mistakes to Avoid

```
// ❌ MISTAKE 1: Returning different errors for invalid email vs wrong password
```

```
// Tells attacker which emails are registered!
```

```
if (user == null)
    return Unauthorized("Email not found."); // BAD
if (!ValidPassword)
    return Unauthorized("Wrong password."); // BAD
```

```
// ✅ CORRECT: Same error for both
```

```
return Unauthorized("Invalid email or password."); // can't tell which failed
```

```
// _____
```

```
// ❌ MISTAKE 2: Storing JWT in localStorage
```

```
// JavaScript can access it – XSS attack can steal it
localStorage.setItem('token', accessToken); // BAD
```

```
// ✅ CORRECT: Store in memory or HttpOnly cookie
```

```
// HttpOnly cookie: JavaScript cannot read it
```

```
// Only sent automatically to your domain
```

```
// _____
```

```
// ❌ MISTAKE 3: Long-lived access tokens
```

```
ExpiryMinutes = 43200; // 30 days – terrible
```

```
// ✅ CORRECT: Short access token + refresh token strategy
```

```
ExpiryMinutes = 60; // access token: 60 minutes
```

```
RefreshTokenExpiry = 7days; // refresh token: 7 days
```

```
// _____
```

```
// ❌ MISTAKE 4: Not validating token on every request
// Trusting anyone who sends any data

// ✅ CORRECT: [Authorize] on every protected endpoint
// JWT middleware validates on every single request automatically

// -----

// ❌ MISTAKE 5: SQL injection via string concatenation
var query = "SELECT * FROM Users WHERE Email = '" + email + "'";
// Attacker sends: email = "'; DROP TABLE Users; --"

// ✅ CORRECT: EF Core protects you automatically
// Or use parameterized queries in raw SQL
var user = await _context.Users
    .FirstOrDefaultAsync(u => u.Email == email); // safe
```

Complete Project Structure for Stage 5

```
MyApi/
|
├─ Configuration/
|   └─ JwtSettings.cs
|
├─ Controllers/
|   ├── AuthController.cs
|   └─ ProductController.cs
|
├─ DTOs/
|   └─ AuthDtos.cs
|
└─ Middleware/
```

```
|   |─ GlobalExceptionHandler.cs
|   |─ SecurityHeadersMiddleware.cs
|
|─ Models/
|   |─ User.cs
|   |─ Product.cs
|
|─ Services/
|   |─ Interfaces/
|   |   |─ IAuthService.cs
|   |   |─ ITokenService.cs
|   |─ AuthService.cs
|   |─ TokenService.cs
|
|─ appsettings.json
|─ Program.cs
```

Interview Questions

Q1: What is the difference between authentication and authorization?

Authentication verifies identity — who you are. Authorization verifies permissions — what you can do. Authentication happens first.

Q2: Explain JWT structure.

Three Base64 parts separated by dots: Header (algorithm), Payload (claims/data), Signature (proves token wasn't tampered with). Only signature is cryptographically secure — payload is just encoded, not encrypted.

Q3: Why use refresh tokens?

Access tokens are short-lived for security. If stolen, they expire quickly. Refresh tokens are long-lived but stored securely in DB. They get new access tokens without requiring re-login, balancing security and user experience.

Q4: What is OAuth 2.0?

A standard authorization protocol that lets users grant third-party apps limited access to their account on another service without sharing passwords. "Login with Google" is OAuth 2.0.

Q5: What is Microsoft Entra ID?

Microsoft's cloud identity service (formerly Azure Active Directory). Manages user identities, authentication, SSO, and role-based access for enterprise applications in the Microsoft ecosystem.

Q6: What is CORS and why is it needed?

Cross-Origin Resource Sharing. Browsers block requests from different domains by default for security. CORS headers tell the browser which origins are allowed to call your API.

Q7: Why should passwords be hashed and not encrypted?

Encryption is reversible — if someone gets the key, they get all passwords. Hashing is one-way — even if the database is stolen, passwords cannot be recovered. BCrypt adds a unique salt per password, making identical passwords hash differently.

Stage 5 Complete — All Covered

-  Authentication vs Authorization — **clear** difference
-  JWT — structure, claims, signing, validation
-  Access Token + Refresh Token strategy
-  Complete JWT implementation — register, login, refresh, logout

- ✓ `[Authorize]` and `[Authorize(Roles)]` on controllers
- ✓ OAuth 2.0 – what it is, authorization code flow, scopes
- ✓ Microsoft Entra ID – tenant, app registration, client ID/secret
- ✓ Entra ID in ASP.NET – `Microsoft.Identity.Web`
- ✓ RBAC – role-based access control
- ✓ Password hashing – BCrypt
- ✓ HTTPS – always use it
- ✓ CORS – control which origins call your API
- ✓ Security headers – protect against common attacks
- ✓ Azure Key Vault – storing secrets safely
- ✓ Rate limiting – prevent brute force and abuse
- ✓ Common security mistakes – and how to avoid them

Reply **"next"** and we go into **Stage 6 — Azure Cloud** — App Service, Azure SQL, Storage, Key Vault, Azure Functions, and monitoring with Application Insights exactly as used in real enterprise projects. 🚀

20/02/2026, 01:18:29

🧑 Continue stage 6 with proper and much good examples with good approach in cloud most experienced like but explain in simple like beginner understand

20/02/2026, 16:47:03

Thought process: Architected beginner-friendly Azure explanation with enterprise-depth examples.

The user wants Stage 6 - Azure Cloud explained in a beginner-friendly way but with experienced/enterprise-level depth and good examples.

Stage 6 — Azure Cloud (Enterprise Level)

Complete Guide — Simple Explanation, Real Enterprise Approach

🤔 What Is Cloud Computing First?

Before Azure, understand **why cloud exists**.

Old way — On-Premise:

Company buys physical servers
Company puts them **in** a room
Company hires people **to** maintain them
Company pays electricity, cooling, security
Server breaks → company fixes it
Need more power → buy more servers (takes weeks)

Problems:

- ✗ Expensive upfront cost
- ✗ Takes weeks to get new servers
- ✗ You manage everything yourself
- ✗ Pay for servers even when idle

Cloud way — Azure:

Microsoft owns thousands of servers in data centers

You RENT what you need

You pay only for what you use

Need more power → click a button → done in minutes

Server breaks → Microsoft fixes it

Benefits:

- ✓ No upfront cost – pay as you go
- ✓ Scale up in minutes
- ✓ Microsoft manages hardware
- ✓ Data centers worldwide
- ✓ Built-in security, backups, monitoring

Azure Global Infrastructure

Azure has data centers in:

Americas:

East US (Virginia)

West US (California)

Brazil South

Europe:

North **Europe** (Ireland)
West **Europe** (Netherlands)
UK **South** (London)

Asia:

Southeast **Asia** (Singapore)
East **Asia** (Hong Kong)
Australia **East** (Sydney)

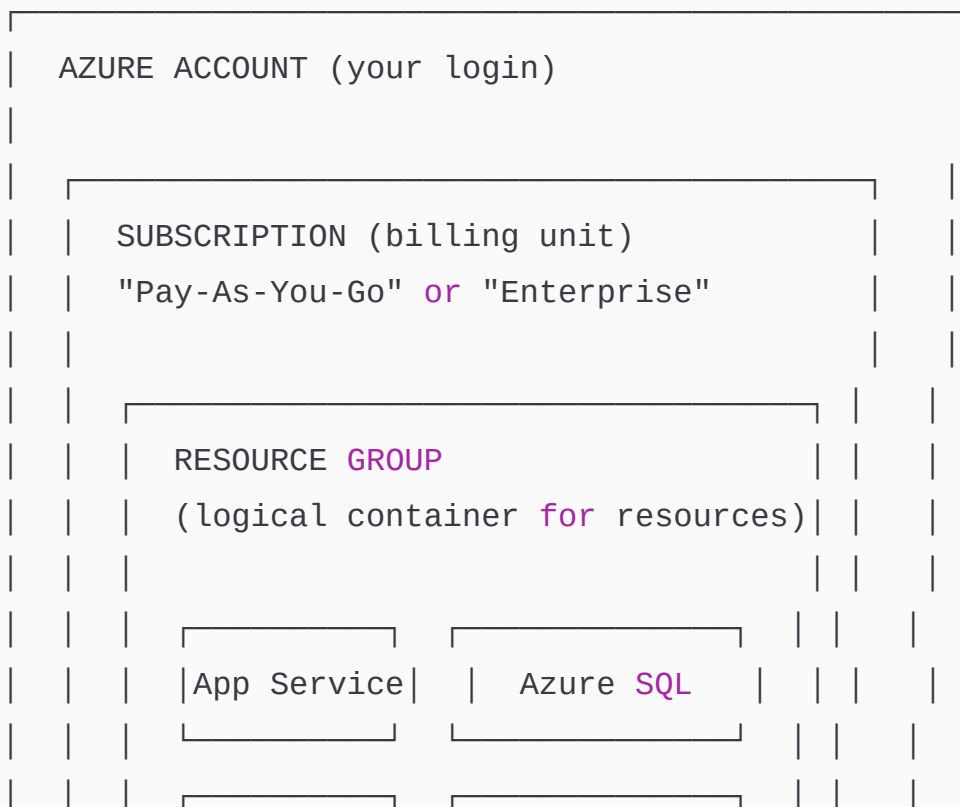
Each **location** = REGION

You choose which region your app runs in

Choose region CLOSEST to your users **for** low latency

How Azure Is Organized

Everything in Azure lives inside a hierarchy:





Lesson 1 — Resource Group

What Is a Resource Group?

"A logical container that holds all related Azure resources for one project."

Think of it like a **folder on your computer**:

My Computer:

- Project-ShopApp/
 - shopapp-database
 - shopapp-api
 - shopapp-keyvault
 - shopapp-storage

Azure:

- rg-shopapp-production/
 - sql-shopapp-prod
 - app-shopapp-prod
 - kv-shopapp-prod
 - st-shopapp-prod

Why Resource Groups matter:

1. Delete the group → deletes EVERYTHING inside (dangerous but useful)
2. See costs for the whole project in one place
3. Apply security policies to all resources at once
4. Deploy all resources together
5. Organize by project or environment

Best practice — name by environment:

```
rg-shopapp-development    ← dev resources
rg-shopapp-staging        ← testing resources
rg-shopapp-production     ← live resources
```

Create via Azure CLI:

```
# Install Azure CLI first:
https://docs.microsoft.com/cli/azure/install

# Login to Azure
az login

# Create resource group
az group create \
  --name rg-shopapp-production \
  --location eastus

# List all resource groups
az group list --output table

# Delete resource group (deletes EVERYTHING inside - be careful!)
az group delete --name rg-shopapp-development --yes
```



Lesson 2 — Azure App Service

What Is App Service?

"A fully managed hosting platform for your ASP.NET Core API. You deploy your code, Microsoft manages the server."

Without App Service:

You need to:

1. Set up Windows/Linux server
2. Install .NET runtime
3. Configure IIS or nginx
4. Set up SSL certificate
5. Configure firewall
6. Monitor the server
7. Apply security patches
8. Handle server crashes

With App Service:

You just:

1. Deploy your code
- Microsoft **handles** EVERYTHING **else**

App Service Plans — Choosing the Right Size

App Service Plan = the underlying virtual machine power:



Tier	Name	Cost	Use For
Free	F1	\$0/month	Learning, experiments
Shared	D1	~\$10/mo	Small hobby projects
Basic	B1-B3	~\$13-55	Dev/Test environments
Standard	S1-S3	~\$70-280	Production small/medium
Premium	P1v3-P3v3	~\$140+	Production high traffic
Isolated	I1-I3	~\$300+	Enterprise, dedicated VNet

For learning: Free (F1)

For real small project: B1 or S1

For enterprise production: P1v3 or above

Deploy ASP.NET Core API to App Service

Step 1 — Create App Service via CLI

```
# Create App Service Plan (the server)
az appservice plan create \
  --name plan-shopapp-prod \
  --resource-group rg-shopapp-production \
  --sku S1 \
  --is-linux

# Create the Web App on that plan
az webapp create \
  --name app-shopapp-prod \
  --resource-group rg-shopapp-production \
  --plan plan-shopapp-prod \
  --runtime "DOTNET|8.0"
```

```
# Your app is now live at:  
# https://app-shopapp-prod.azurewebsites.net
```

Step 2 — Configure App Settings (Environment Variables)

```
# Set connection string (never hardcode in appsettings.json for  
production)  
az webapp config connection-string set \  
  --name app-shopapp-prod \  
  --resource-group rg-shopapp-production \  
  --settings DefaultConnection="Server=sql-  
shopapp.database.windows.net;..." \  
  --connection-string-type SQLAzure  
  
# Set environment variables  
az webapp config appsettings set \  
  --name app-shopapp-prod \  
  --resource-group rg-shopapp-production \  
  --settings \  
    ASPNETCORE_ENVIRONMENT=Production \  
    JwtSettings__SecretKey=@Microsoft.KeyVault(...) \  
    JwtSettings__ExpiryMinutes=60
```

Step 3 — Deploy Your Code

Option A — Deploy via ZIP (simplest):

```
# Build your project  
dotnet publish -c Release -o ./publish  
  
# Zip the output  
Compress-Archive -Path ./publish/* -DestinationPath deploy.zip
```

```
# Deploy zip to Azure
az webapp deployment source config-zip \
  --name app-shopapp-prod \
  --resource-group rg-shopapp-production \
  --src deploy.zip
```

Option B — Deploy via GitHub Actions (enterprise way):

```
# .github/workflows/deploy.yml
name: Deploy to Azure App Service

on:
  push:
    branches: [ main ] # deploy when code pushed to main

jobs:
  deploy:
    runs-on: ubuntu-latest

    steps:
      # Step 1: Get the code
      - name: Checkout code
        uses: actions/checkout@v3

      # Step 2: Setup .NET
      - name: Setup .NET
        uses: actions/setup-dotnet@v3
        with:
          dotnet-version: '8.0.x'

      # Step 3: Build
      - name: Build
        run: dotnet build --configuration Release

      # Step 4: Test
```

```
- name: Test
  run: dotnet test --no-build --verbosity normal

# Step 5: Publish
- name: Publish
  run: dotnet publish -c Release -o ./publish

# Step 6: Deploy to Azure
- name: Deploy to Azure Web App
  uses: azure/webapps-deploy@v2
  with:
    app-name: app-shopapp-prod
    publish-profile: ${ secrets.AZURE_PUBLISH_PROFILE }
    package: ./publish
```

App Service — Important Features

Auto Scaling — Handle More Traffic

```
# Scale UP – bigger machine (more CPU/RAM)
az appservice plan update \
  --name plan-shopapp-prod \
  --resource-group rg-shopapp-production \
  --sku P2v3 # upgrade from P1 to P2

# Scale OUT – more instances (copies) of your app
az monitor autoscale create \
  --resource-group rg-shopapp-production \
  --resource plan-shopapp-prod \
  --resource-type Microsoft.Web/serverfarms \
  --name autoscale-shopapp \
  --min-count 1 \
```

```
--max-count 5 \  
--count 1  
  
# Auto-scale rule: add instance when CPU > 70%  
az monitor autoscale rule create \  
  --resource-group rg-shopapp-production \  
  --autoscale-name autoscale-shopapp \  
  --scale out 1 \  
  --condition "Percentage CPU > 70 avg 5m"
```

Deployment Slots — Zero Downtime Deployments

Production Slot	Staging Slot
app-shopapp-prod.azurewebsites	app-shopapp-prod-staging.azurewebsites
Current live version	New version being tested

Workflow:

1. Deploy **new** version **to** Staging slot
 2. Test it thoroughly
 3. SWAP slots → Staging becomes Production instantly
 4. **Old** Production becomes Staging (easy **rollback**)
- Zero downtime **for** users!

```
# Create staging slot  
az webapp deployment slot create \  
  --name app-shopapp-prod \  
  --resource-group rg-shopapp-production \  
  --slot staging  
  
# Deploy to staging slot
```



```
az webapp deployment source config-zip \  
  --name app-shopapp-prod \  
  --resource-group rg-shopapp-production \  
  --slot staging \  
  --src deploy.zip  
  
# Swap staging → production (zero downtime!)  
az webapp deployment slot swap \  
  --name app-shopapp-prod \  
  --resource-group rg-shopapp-production \  
  --slot staging \  
  --target-slot production
```

Lesson 3 — Azure SQL Database

What Is Azure SQL?

"Microsoft's fully managed SQL Server in the cloud. Same SQL Server you learned in Stage 3 — but Microsoft runs it."

Local SQL Server:

You install it
You **update** it
You backup manually
You **set** up HA
Server breaks = downtime
You manage security patches

Azure SQL:

Microsoft manages it
Automatic updates
Automatic backups (**35** days)
Built-**in** high availability
99.99% uptime SLA
Microsoft patches it

Create Azure SQL Database

```
# Create SQL Server (the server that hosts databases)
```

```
az sql server create \  
  --name sql-shopapp-prod \  
  --resource-group rg-shopapp-production \  
  --location eastus \  
  --admin-user sqladmin \  
  --admin-password "MyStr0ng@Password!"
```

```
# Create the actual database on that server
```

```
az sql db create \  
  --resource-group rg-shopapp-production \  
  --server sql-shopapp-prod \  
  --name ShopAppDb \  
  --edition GeneralPurpose \  
  --family Gen5 \  
  --capacity 2 \  
  --zone-redundant false
```

```
# Allow Azure services to connect (your App Service)
```

```
az sql server firewall-rule create \  
  --resource-group rg-shopapp-production \  
  --server sql-shopapp-prod \  
  --name AllowAzureServices \  
  --start-ip-address 0.0.0.0 \  
  --end-ip-address 0.0.0.0
```

```
# Allow YOUR IP for development access from SSMS
```

```
az sql server firewall-rule create \  
  --resource-group rg-shopapp-production \  
  --server sql-shopapp-prod \  
  --name MyDevMachine \  
  --start-ip-address 0.0.0.0
```

```
--start-ip-address YOUR.IP.ADDRESS.HERE \  
--end-ip-address YOUR.IP.ADDRESS.HERE
```

Connection String — Proper Setup

```
// appsettings.json – use placeholder, not real password  
{  
  "ConnectionStrings": {  
    "DefaultConnection": "Server=sql-shopapp-  
prod.database.windows.net;Database=ShopAppDb;User  
Id=sqladmin;Password=REPLACED_BY_KEYVAULT;"  
  }  
}
```

```
// Program.cs – connect EF Core to Azure SQL  
builder.Services.AddDbContext<AppDbContext>(options =>  
{  
  options.UseSqlServer(  
  
builder.Configuration.GetConnectionString("DefaultConnection"),  
  sqlOptions =>  
  {  
    // Retry logic for transient Azure SQL failures  
    sqlOptions.EnableRetryOnFailure(  
      maxRetryCount:      5,  
      maxRetryDelay:      TimeSpan.FromSeconds(30),  
      errorNumbersToAdd:  null);  
  });  
});
```

Azure SQL Pricing Tiers — Simple Guide

Tier	Best For
Basic	Dev/test, very small apps
Standard (S0-S3)	Small-medium production apps
Premium	High performance, high throughput
General Purpose	Most production workloads 
Business Critical	Mission critical, ultra low latency
Hyperscale	Massive databases (100TB+)

For learning: Basic (~\$5/month)

For real production: General Purpose (cheapest starts ~\$370/month)

For enterprise: Business Critical

Automatic Backups — How They Work

Azure SQL automatically backs up your database:

- Full backup: Every week
- Differential backup: Every 12-24 hours
- Transaction log backup: Every 5-10 minutes

Retention period:

- Basic tier: 7 days
- Standard/Premium: 7-35 days (you choose)

Restore to any point in time:

- "Restore to 3:27 PM yesterday" – Azure can do this!

```
# Restore database to specific point in time
az sql db restore \
  --resource-group rg-shopapp-production \
  --server sql-shopapp-prod \
  --name ShopAppDb-Restored \
  --dest-name ShopAppDb-Restored \
  --time "2026-02-19T15:27:00Z" # restore to this exact moment
```

Lesson 4 — Azure Key Vault

What Is Key Vault?

"A secure vault in Azure for storing secrets, connection strings, API keys, and certificates. Your app fetches them at runtime — they never live in your code."

The problem it solves:

✗ BAD — Secret in code (terrible):

```
var connectionString = "Server=sql;Password=MyPassword123;";
// This goes to git → everyone with repo access sees your password
```

✗ BAD — Secret in appsettings.json (still bad):

```
{
  "ConnectionStrings": {
    "DefaultConnection": "...Password=MyPassword123..."
  }
}
// appsettings.json often committed to git — same problem
```

✅ GOOD – Secret *in* Key Vault:

```
var connectionString =  
configuration["ConnectionStrings:DefaultConnection"];  
// Value fetched from Key Vault at runtime  
// Code has no actual secret → safe to push to git
```

Create Key Vault and Add Secrets

```
# Create Key Vault  
az keyvault create \  
  --name kv-shopapp-prod \  
  --resource-group rg-shopapp-production \  
  --location eastus \  
  --sku standard  
  
# Add secrets to Key Vault  
az keyvault secret set \  
  --vault-name kv-shopapp-prod \  
  --name "ConnectionStrings-DefaultConnection" \  
  --value "Server=sql-shopapp-  
prod.database.windows.net;Database=ShopAppDb;User  
Id=sqladmin;Password=ActualPassword123!"  
  
az keyvault secret set \  
  --vault-name kv-shopapp-prod \  
  --name "JwtSettings-SecretKey" \  
  --value "my-ultra-secure-jwt-secret-key-production-2026!!"  
  
az keyvault secret set \  
  --vault-name kv-shopapp-prod \  
  --name "ExternalApi-ApiKey" \  
  --value "sk-prod-abc123xyz789"
```

```
# List all secrets (shows names only, not values)
az keyvault secret list \
  --vault-name kv-shopapp-prod \
  --output table
```

Connect Your ASP.NET App to Key Vault

```
# Install packages
dotnet add package
Azure.Extensions.AspNetCore.Configuration.Secrets
dotnet add package Azure.Identity
```

```
// Program.cs – pull secrets from Key Vault automatically
using Azure.Identity;

var builder = WebApplication.CreateBuilder(args);

// — Add Key Vault to configuration —————
var keyVaultUrl = builder.Configuration["KeyVaultUrl"];
// KeyVaultUrl is safe to put in appsettings.json – it's not a secret

if (!string.IsNullOrEmpty(keyVaultUrl))
{
    builder.Configuration.AddAzureKeyVault(
        new Uri(keyVaultUrl),
        new DefaultAzureCredential()
        // DefaultAzureCredential tries multiple auth methods:
        // 1. Managed Identity (when running in Azure – best)
        // 2. Visual Studio credentials (when developing locally)
    );
}
```

```
// 3. Azure CLI credentials (when using az login locally)
// No client secret needed in code!

);
}

// After AddAzureKeyVault – use configuration normally
// Key Vault secrets automatically available as if they were in
appsettings!
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(

builder.Configuration.GetConnectionString("DefaultConnection")
    // ↑ This now comes from Key Vault!
));
```

```
// appsettings.json – only the KEY VAULT URL (not the secrets)
{
  "KeyVaultUrl": "https://kv-shopapp-prod.vault.azure.net/",
  "Logging": {
    "LogLevel": {
      "Default": "Information"
    }
  }
}
```

Enable Managed Identity — No Credentials Needed

Managed Identity = Azure gives your App Service an automatic identity. Key Vault trusts this identity. No username/password needed anywhere.

```
# Enable Managed Identity on your App Service
az webapp identity assign \
```



```
--name app-shopapp-prod \  
--resource-group rg-shopapp-production
```

*# The above command returns a principalId (object ID)
Copy the principalId from the output*

Give the App Service permission to READ Key Vault secrets
az keyvault set-policy \
--name kv-shopapp-prod \
--object-id YOUR_PRINCIPAL_ID_HERE \
--secret-permissions get list

*# Now your App Service can read Key Vault secrets
WITHOUT any username or password stored anywhere!*

Key Vault Best Practices

Secret Naming Convention (double dash = nested config):

Key Vault Secret Name	Maps to appsettings path
-----------------------	--------------------------

ConnectionStrings--DefaultConnection	→
--------------------------------------	---

ConnectionStrings:DefaultConnection	
-------------------------------------	--

JwtSettings--SecretKey	→ JwtSettings:SecretKey
------------------------	-------------------------

ExternalApi--ApiKey	→ ExternalApi:ApiKey
---------------------	----------------------

Never store in Key Vault:

- | | |
|----------------------|------------------------------------|
| ✓ Connection strings | ← YES, always Key Vault |
| ✓ JWT secret keys | ← YES, always Key Vault |
| ✓ External API keys | ← YES, always Key Vault |
| ✓ SSL certificates | ← YES, Key Vault handles certs too |

✗ App settings like ← NO, appsettings.json is fine
max page size,
feature flags,
log levels

Lesson 5 — Azure Storage Account

What Is Azure Storage?

"Cheap, massively scalable storage for files, images, videos, logs, and more."

Azure Storage has four types:

Storage Type	What It Stores
Blob Storage	Files, images, videos, documents, backups
Table Storage	NoSQL key-value data (cheaper than CosmosDB)
Queue Storage	Messages between services
File Storage	SMB file share, like a network drive

You will mostly use: Blob Storage (files/images)

Create Storage Account

```
# Create storage account
az storage account create \
  --name stshopappprod \
  --resource-group rg-shopapp-production \
  --location eastus \
  --sku Standard_LRS \
  --kind StorageV2

# Storage account names: lowercase only, no hyphens, 3-24 chars

# Create a container (like a folder) for product images
az storage container create \
  --name product-images \
  --account-name stshopappprod \
  --public-access blob # images are publicly readable

# Create private container for documents
az storage container create \
  --name private-documents \
  --account-name stshopappprod \
  --public-access off # private – requires auth to access
```

File Upload Service — Complete Working Code

```
# Install package
dotnet add package Azure.Storage.Blobs
```

```
// Services/StorageService.cs
using Azure.Storage.Blobs;
using Azure.Storage.Blobs.Models;
```

```

namespace MyApi.Services
{
    public interface IStorageService
    {
        Task<string> UploadFileAsync(
            IFormFile file, string containerName);

        Task<bool> DeleteFileAsync(
            string fileName, string containerName);

        Task<string> GetFileUrlAsync(
            string fileName, string containerName);
    }

    public class StorageService : IStorageService
    {
        private readonly BlobServiceClient _blobServiceClient;
        private readonly ILogger<StorageService> _logger;

        public StorageService(IConfiguration config,
                               ILogger<StorageService> logger)
        {
            _blobServiceClient = new BlobServiceClient(
                config["AzureStorage:ConnectionString"]);
            _logger = logger;
        }

        // — UPLOAD FILE —————
        public async Task<string> UploadFileAsync(
            IFormFile file, string containerName)
        {
            // Validate file
            if (file == null || file.Length == 0)
                throw new ArgumentException("File is empty.");
        }
    }
}

```

```

var allowedTypes = new[] {
    "image/jpeg", "image/png",
    "image/gif", "application/pdf" };

if
(!allowedTypes.Contains(file.ContentType.ToLower()))
    throw new ArgumentException(
        "Invalid file type. Only JPEG, PNG, GIF, PDF
allowed.");

const long maxSize = 5 * 1024 * 1024; // 5MB
if (file.Length > maxSize)
    throw new ArgumentException(
        "File too large. Maximum 5MB allowed.");

// Generate unique filename to avoid conflicts
var fileExtension = Path.GetExtension(file.FileName);
var uniqueFileName = $"{Guid.NewGuid()}
{fileExtension}";

// Get container reference
var containerClient = _blobServiceClient
    .GetBlobContainerClient(containerName);

// Create container if doesn't exist
await containerClient.CreateIfNotExistsAsync(
    PublicAccessType.Blob);

// Get blob reference
var blobClient = containerClient
    .GetBlobClient(uniqueFileName);

// Set metadata
var blobHttpHeaders = new BlobHttpHeaders
{

```

```

        ContentType = file.ContentType
    };

    // Upload file stream
    using var stream = file.OpenReadStream();
    await blobClient.UploadAsync(stream, new
BlobUploadOptions
    {
        HttpHeaders = blobHttpHeaders
    });

    _logger.LogInformation(
        "File uploaded: {FileName} to {Container}",
        uniqueFileName, containerName);

    // Return the public URL
    return blobClient.Uri.ToString();
}

// — DELETE FILE —————
public async Task<bool> DeleteFileAsync(
    string fileName, string containerName)
{
    var containerClient = _blobServiceClient
        .GetBlobContainerClient(containerName);

    var blobClient =
containerClient.GetBlobClient(fileName);

    var response = await
blobClient.DeleteIfExistsAsync();

    _logger.LogInformation(
        "File deleted: {FileName}", fileName);

```

```

        return response.Value;
    }

    // — GET FILE URL —————
    public async Task<string> GetFileUrlAsync(
        string fileName, string containerName)
    {
        var containerClient = _blobServiceClient
            .GetBlobContainerClient(containerName);

        var blobClient =
containerClient.GetBlobClient(fileName);

        var exists = await blobClient.ExistsAsync();
        if (!exists.Value)
            throw new FileNotFoundException(
                $"File {fileName} not found.");

        return blobClient.Uri.ToString();
    }
}

```

```

// Controllers/ProductImageController.cs
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using MyApi.Services;

[ApiController]
[Route("api/products/{productId}/images")]
[Authorize]
public class ProductImageController : ControllerBase
{
    private readonly IStorageService _storageService;
    private readonly IProductService _productService;
}

```

```

public ProductImageController(
    IStorageService storageService,
    IProductService productService)
{
    _storageService = storageService;
    _productService = productService;
}

// POST api/products/1/images
[HttpPost]
[Authorize(Roles = "Admin,Manager")]
public async Task<IActionResult> UploadImage(
    int productId, IFormFile image)
{
    var product = await
_productService.GetByIdAsync(productId);
    if (product == null)
        return NotFound($"Product {productId} not found.");

    var imageUrl = await _storageService
        .UploadFileAsync(image, "product-images");

    // Save URL to product in database
    await _productService.UpdateImageUrlAsync(productId,
imageUrl);

    return Ok(new { ImageUrl = imageUrl });
}

// DELETE api/products/1/images/filename.jpg
[HttpDelete("{fileName}")]
[Authorize(Roles = "Admin")]
public async Task<IActionResult> DeleteImage(
    int productId, string fileName)

```



```
{  
    var deleted = await _storageService  
        .DeleteFileAsync(fileName, "product-images");  
  
    if (!deleted)  
        return NotFound("Image not found.");  
  
    return NoContent();  
}  
}
```

Lesson 6 — Azure Functions

What Is Azure Functions?

"Serverless code that runs only when triggered. You pay per execution — not for a running server."

Normal App Service:

Server **running** 24/7
You pay: 24 hours × 30 days = 720 hours/month
Even **when** ZERO requests come **in** → still paying

Azure Functions:

Code sleeps until something triggers it
Wakes up → runs → sleeps again
You pay **ONLY for** the **time** it actually runs
First 1 million executions **per month** = **FREE**

Perfect **for**:

- Background jobs (send emails, process reports)
- Scheduled tasks (cleanup **old** data **every** night)
- Event-driven processing (**when** file uploaded, resize image)
- Webhooks **from external** services
- Message processing **from** queues

Types of Triggers

Trigger Type	When It Runs
HTTP Trigger	Someone calls an HTTP endpoint
Timer Trigger	On a schedule (like cron job)
Blob Trigger	When file uploaded to storage
Queue Trigger	When message added to queue
Service Bus	When message on Service Bus topic
Event Grid	When Azure event happens

Create Azure Functions Project

```
# Install Azure Functions tools
dotnet tool install -g Microsoft.Azure.WebJobs.Script.WebHost

# Create Functions project
func init ShopAppFunctions --worker-runtime dotnet
```

```
cd ShopAppFunctions
```

```
# Add packages
```

```
dotnet add package Microsoft.Azure.Functions.Worker
```

```
dotnet add package
```

```
Microsoft.Azure.Functions.Worker.Extensions.Http
```

```
dotnet add package
```

```
Microsoft.Azure.Functions.Worker.Extensions.Timer
```

```
dotnet add package
```

```
Microsoft.Azure.Functions.Worker.Extensions.Storage
```

Real Azure Functions Examples

Function 1 — Timer Trigger (Nightly Cleanup Job)

```
// Functions/NightlyCleanupFunction.cs
using Microsoft.Azure.Functions.Worker;
using Microsoft.Extensions.Logging;
using Microsoft.EntityFrameworkCore;
using MyApi.Data;

namespace ShopAppFunctions
{
    public class NightlyCleanupFunction
    {
        private readonly AppDbContext _context;
        private readonly ILogger<NightlyCleanupFunction> _logger;

        public NightlyCleanupFunction(
            AppDbContext context,
            ILogger<NightlyCleanupFunction> logger)
    }
}
```

```

{
    _context = context;
    _logger = logger;
}

// Runs every day at 2:00 AM UTC
// CRON format: second minute hour day month dayOfWeek
[Function("NightlyCleanup")]
public async Task Run(
    [TimerTrigger("0 0 2 * * *")] TimerInfo timerInfo)
{
    _logger.LogInformation(
        "Nightly cleanup started at: {Time}",
        DateTime.UtcNow);

    // Delete soft-deleted products older than 90 days
    var cutoffDate = DateTime.UtcNow.AddDays(-90);

    var oldProducts = await _context.Products
        .Where(p => !p.IsActive && p.CreatedAt <
cutoffDate)
        .ToListAsync();

    if (oldProducts.Any())
    {
        _context.Products.RemoveRange(oldProducts);
        await _context.SaveChangesAsync();

        _logger.LogInformation(
            "Cleaned up {Count} old products.",
oldProducts.Count);
    }

    // Archive old orders (older than 1 year)
    var archiveCutoff = DateTime.UtcNow.AddYears(-1);

```

```

        var oldOrders = await _context.Orders
            .Where(o => o.OrderDate < archiveCutoff
                && o.Status == "Delivered")
            .CountAsync();

        _logger.LogInformation(
            "Found {Count} orders to archive.", oldOrders);

        _logger.LogInformation(
            "Nightly cleanup completed at: {Time}",
            DateTime.UtcNow);
    }
}

```

Function 2 — HTTP Trigger (Lightweight Endpoint)

```

// Functions/ProductReportFunction.cs
using Microsoft.Azure.Functions.Worker;
using Microsoft.Azure.Functions.Worker.Http;
using Microsoft.Extensions.Logging;
using System.Net;
using MyApi.Data;
using Microsoft.EntityFrameworkCore;

namespace ShopAppFunctions
{
    public class ProductReportFunction
    {
        private readonly AppDbContext _context;
        private readonly ILogger<ProductReportFunction> _logger;
    }
}

```

```

public ProductReportFunction(
    AppDbContext context,
    ILogger<ProductReportFunction> logger)
{
    _context = context;
    _logger = logger;
}

// Generates a daily inventory report
[Function("GenerateInventoryReport")]
public async Task<HttpresponseData> Run(
    [HttpTrigger(AuthorizationLevel.Function,
        "get", Route = "reports/inventory")]
    HttpRequestData req)
{
    _logger.LogInformation("Generating inventory
report...");

    // Get low stock products
    var lowStockProducts = await _context.Products
        .Where(p => p.IsActive && p.Stock <= 5)
        .Select(p => new
        {
            p.Id,
            p.Name,
            p.Stock,
            Category = p.Category.Name
        })
        .OrderBy(p => p.Stock)
        .ToListAsync();

    // Get summary stats
    var stats = new
    {
        GeneratedAt = DateTime.UtcNow,

```

```

        TotalProducts      = await
_context.Products.CountAsync(p => p.IsActive),
        OutOfStock        = await
_context.Products.CountAsync(p => p.Stock == 0),
        LowStock          = lowStockProducts.Count,
        TotalInventoryValue = await _context.Products
            .Where(p => p.IsActive)
            .SumAsync(p => p.Price * p.Stock),
        LowStockProducts  = lowStockProducts
    };

    var response = req.CreateResponse(HttpStatusCode.OK);
    await response.WriteAsJsonAsync(stats);

    return response;
}
}
}

```

Function 3 — Blob Trigger (Process Uploaded File)

```

// Functions/ImageProcessingFunction.cs
using Microsoft.Azure.Functions.Worker;
using Microsoft.Extensions.Logging;

namespace ShopAppFunctions
{
    public class ImageProcessingFunction
    {
        private readonly ILogger<ImageProcessingFunction>
_logger;

```

```

public ImageProcessingFunction(
    ILogger<ImageProcessingFunction> logger)
{
    _logger = logger;
}

// Runs automatically when file uploaded to product-
images container
[Function("ProcessProductImage")]
public async Task Run(
    [BlobTrigger("product-images/{name}",
        Connection = "AzureWebJobsStorage")] Stream
imageStream,
    string name)
{
    _logger.LogInformation(
        "Processing uploaded image: {Name}, Size: {Size}
bytes",
        name, imageStream.Length);

    // In real project: resize image, create thumbnails
    // generate multiple sizes (small, medium, large)
    // save them back to storage

    // Example – just log for now:
    _logger.LogInformation(
        "Image {Name} processed successfully.", name);

    await Task.CompletedTask;
}
}
}

```


Deploy Azure Functions

```
# Create Function App in Azure
az functionapp create \
  --resource-group rg-shopapp-production \
  --consumption-plan-location eastus \
  --runtime dotnet-isolated \
  --functions-version 4 \
  --name func-shopapp-prod \
  --storage-account stshopappprod

# Deploy functions code
func azure functionapp publish func-shopapp-prod
```

Lesson 7 — Application Insights (Monitoring)

What Is Application Insights?

"Microsoft's monitoring service. It watches your running app 24/7 and tells you everything — errors, performance, who's using it, and where it's slow."

Without monitoring:

```
User: "The app is broken"
You: "Umm... let me check..."
You: searching through logs for hours
User: "Is it fixed yet?"
You: "I don't even know what's wrong yet..."
```

With Application Insights:

User: "The app is broken"

You: Opens Azure Portal → Application Insights

You: Sees exactly:

- Error spike at 3:47 PM
- NullReferenceException in ProductService.GetById
- Line 47
- Happened 243 times in last hour
- Average response time went from 200ms to 8000ms

You: "Fixed in 5 minutes"

What Application Insights Tracks Automatically

- ✓ Every HTTP request (URL, status code, duration)
- ✓ All exceptions with full stack trace
- ✓ Database query performance (slow queries flagged)
- ✓ External HTTP call performance
- ✓ Custom events you define
- ✓ User sessions and page views
- ✓ Server metrics (CPU, memory)
- ✓ Dependency tracking
- ✓ Real-time live stream of everything

Setup Application Insights in ASP.NET Core

```
# Create Application Insights resource  
az monitor app-insights component create \
```

```
--app insights-shopapp-prod \  
--resource-group rg-shopapp-production \  
--location eastus \  
--kind web  
  
# Get the connection string (copy this)  
az monitor app-insights component show \  
  --app insights-shopapp-prod \  
  --resource-group rg-shopapp-production \  
  --query connectionString  
  
# Install package  
dotnet add package Microsoft.ApplicationInsights.AspNetCore
```

```
// appsettings.json  
{  
  "ApplicationInsights": {  
    "ConnectionString":  
    "InstrumentationKey=abc123;IngestionEndpoint=https://..."  
  }  
}
```

```
// Program.cs – add Application Insights  
builder.Services.AddApplicationInsightsTelemetry(  
  
builder.Configuration["ApplicationInsights:ConnectionString"]);
```

Custom Telemetry — Track Your Own Events

```
// Services/ProductService.cs – with telemetry  
using Microsoft.ApplicationInsights;
```

```

using Microsoft.ApplicationInsights.DataContracts;

public class ProductService : IProductService
{
    private readonly AppDbContext _context;
    private readonly TelemetryClient _telemetry;
    private readonly ILogger<ProductService> _logger;

    public ProductService(
        AppDbContext context,
        TelemetryClient telemetry,
        ILogger<ProductService> logger)
    {
        _context    = context;
        _telemetry  = telemetry;
        _logger     = logger;
    }

    public async Task<ProductResponseDto>
CreateAsync(CreateProductDto dto)
    {
        var stopwatch = System.Diagnostics.Stopwatch.StartNew();

        try
        {
            var product = new Product
            {
                Name      = dto.Name,
                Price     = dto.Price,
                Stock     = dto.Stock,
                IsActive  = true,
                CreatedAt = DateTime.UtcNow
            };

            _context.Products.Add(product);

```

```

        await _context.SaveChangesAsync();

        stopwatch.Stop();

        // Track custom event – appears in Azure Portal
        _telemetry.TrackEvent("ProductCreated", new
Dictionary<string, string>
        {
            { "ProductId",    product.Id.ToString() },
            { "ProductName",  product.Name },
            { "Category",     dto.CategoryId.ToString() }
        },
        new Dictionary<string, double>
        {
            { "Price",        (double)product.Price },
            { "DurationMs",    stopwatch.ElapsedMilliseconds
}

        });

        return MapToDto(product);
    }
    catch (Exception ex)
    {
        stopwatch.Stop();

        // Track exception with context
        _telemetry.TrackException(ex, new Dictionary<string,
string>
        {
            { "Operation",    "CreateProduct" },
            { "ProductName",  dto.Name }
        });

        _logger.LogError(ex,
            "Failed to create product: {Name}", dto.Name);
    }
}

```

```

        throw;
    }
}

public async Task<List<ProductResponseDto>> GetAllAsync()
{
    // Track a custom metric – e.g. how many products
    returned
    var products = await _context.Products
        .Where(p => p.IsActive)
        .AsNoTracking()
        .ToListAsync();

    // Track metric
    _telemetry.TrackMetric("ProductsReturned",
products.Count);

    return products.Select(MapToDto).ToList();
}
}

```

Azure Monitor — Alerts That Wake You Up At Night

```

# Create alert: notify if app has more than 5 errors in 5 minutes
az monitor metrics alert create \
  --name alert-high-error-rate \
  --resource-group rg-shopapp-production \
  --scopes /subscriptions/YOUR_SUB/resourceGroups/rg-shopapp-
production/providers/Microsoft.Insights/components/insights-
shopapp-prod \
  --condition "count requests/failed > 5" \

```

```
--window-size 5m \  
--evaluation-frequency 1m \  
--action-group ag-shopapp-team \  
--severity 1 # 0=Critical, 1=Error, 2=Warning  
  
# Create alert: notify if response time > 3 seconds  
az monitor metrics alert create \  
  --name alert-slow-response \  
  --resource-group rg-shopapp-production \  
  --condition "avg requests/duration > 3000" \  
  --window-size 5m \  
  --severity 2
```

Log Analytics — Query Your Logs Like SQL

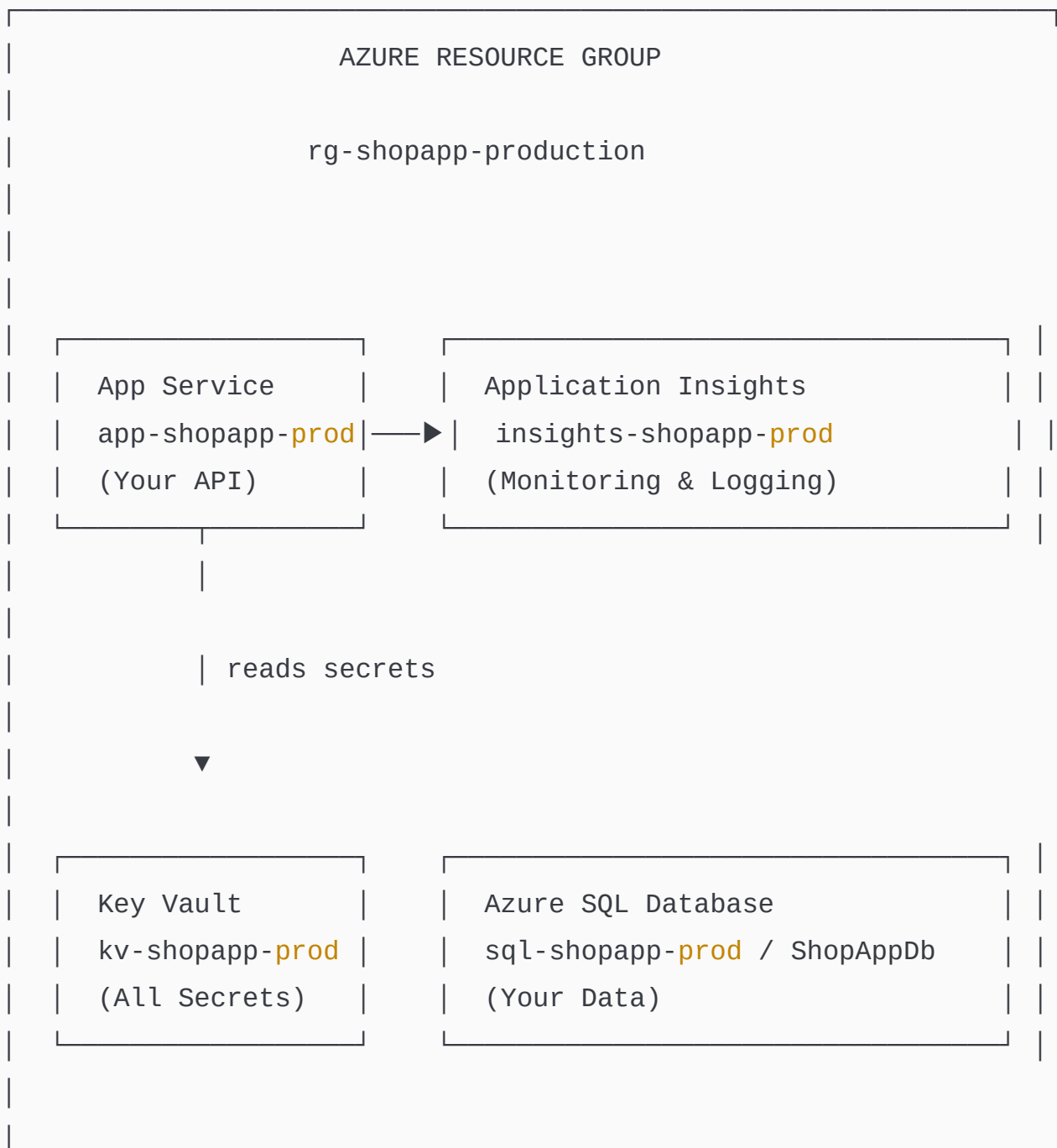
In Azure Portal → Application Insights → Logs
You can write Kusto Query [Language](#) (KQL) to analyze:

```
// Find all errors in last 24 hours  
exceptions  
| where timestamp > ago(24h)  
| summarize count() by type  
| order by count_ desc  
  
// Slowest API endpoints  
requests  
| where timestamp > ago(7d)  
| summarize avgDuration=avg(duration) by name  
| order by avgDuration desc  
| top 10 by avgDuration  
  
// Users who got errors
```

```
exceptions
```

```
| where timestamp > ago(1h)
| join requests on operation_Id
| project timestamp, user_Id, name, type, outerMessage
```

Complete Azure Architecture — All Pieces Together



Storage Account	Azure Functions
stshopappprod	func-shopapp- prod
(Files/Images)	(Background Jobs)

Complete Program.cs — Everything Wired Together

```
// Program.cs – production-ready with all Azure services
```

```
using Azure.Identity;  
using Microsoft.ApplicationInsights;  
using Microsoft.EntityFrameworkCore;  
using MyApi.Configuration;  
using MyApi.Data;  
using MyApi.Middleware;  
using MyApi.Services;
```

```
var builder = WebApplication.CreateBuilder(args);
```

```
// — 1. Key Vault – load secrets first —————
```

```
var keyVaultUrl = builder.Configuration["KeyVaultUrl"];  
if (!string.IsNullOrEmpty(keyVaultUrl))  
{  
    builder.Configuration.AddAzureKeyVault(  
        new Uri(keyVaultUrl),  
        new DefaultAzureCredential());  
}
```

```
// — 2. Application Insights —————
```

```
builder.Services.AddApplicationInsightsTelemetry()
```

```

builder.Configuration["ApplicationInsights:ConnectionString"]);

// — 3. Database —————
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(

builder.Configuration.GetConnectionString("DefaultConnection"),
    sql => sql.EnableRetryOnFailure(5,
    TimeSpan.FromSeconds(30), null)
));

// — 4. Authentication —————
builder.Services.Configure<JwtSettings>(
    builder.Configuration.GetSection("JwtSettings"));

builder.Services
    .AddAuthentication()
    .AddJwtBearer(/* JWT config */);

builder.Services.AddAuthorization();

// — 5. App Services —————
builder.Services.AddScoped<IProductService, ProductService>();
builder.Services.AddScoped<IAuthService, AuthService>();
builder.Services.AddScoped<ITokenService, TokenService>();
builder.Services.AddSingleton<IStorageService, StorageService>();

// — 6. Rate Limiting —————
builder.Services.AddRateLimiter(options =>
{
    options.AddFixedWindowLimiter("api", o =>
    {
        o.Window      = TimeSpan.FromMinutes(1);
        o.PermitLimit = 100;
    });
});

```

```
});

builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

// — Build —————
var app = builder.Build();

// — Auto migrate —————
using (var scope = app.Services.CreateScope())
{
    var db = scope.ServiceProvider
        .GetRequiredService<AppDbContext>();
    db.Database.Migrate();
}

// — Middleware pipeline —————
app.UseMiddleware<GlobalExceptionMiddleware>(); // first
app.UseMiddleware<SecurityHeadersMiddleware>();

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();
app.UseCors("Production");
app.UseRateLimiter();
app.UseAuthentication();
app.UseAuthorization();
app.MapControllers();

app.Run();
```

Interview Questions

Q1: What is a Resource Group in Azure?

A logical container that holds all related Azure resources for one project or environment. Allows managing, billing, and deleting all resources together.

Q2: What is Azure App Service?

A fully managed hosting platform for web applications. You deploy your code — Microsoft manages the server, OS, runtime, and infrastructure.

Q3: What is the difference between scale up and scale out?

Scale up means upgrading to a more powerful machine (more CPU/RAM). Scale out means adding more instances (copies) of your app to distribute traffic. Scale out is preferred for high availability.

Q4: What is Azure Key Vault and why use it?

A secure cloud service for storing secrets, API keys, and certificates. Your code fetches them at runtime via Managed Identity — no secrets ever stored in code or config files.

Q5: What is a Managed Identity?

An Azure feature that gives your App Service an automatic identity. Other Azure services like Key Vault can trust this identity — no username or password needed in your code.

Q6: What are Azure Functions?

Serverless compute that runs code only when triggered. Pay per execution. Perfect for background jobs, scheduled tasks, and event-driven processing.

Q7: What is Application Insights?

Azure's monitoring service that tracks all requests, errors, performance, dependencies, and custom events in your running application. Shows real-time dashboards and lets you query logs.

Q8: What is a deployment slot?

A separate environment for your app (like staging) running on the same App Service. You deploy and test in staging, then swap to production — zero downtime.

✅ Stage 6 Complete — All Covered

- ✅ Cloud computing – why it **exists**, benefits
- ✅ Azure infrastructure – regions, hierarchy
- ✅ Resource **Groups** – organization, best practices
- ✅ App Service – plans, deployment, scaling, slots
- ✅ Azure **SQL** – **create**, **connect**, backups, retry logic
- ✅ Key Vault – secrets, Managed **Identity**, **no** passwords **in** code
- ✅ Storage Account – **blob** storage, file upload service
- ✅ Azure Functions – HTTP, Timer, **Blob** triggers
- ✅ Application Insights – monitoring, custom telemetry
- ✅ Azure Monitor – alerts, KQL log queries
- ✅ Complete architecture – **all** pieces wired together
- ✅ Production Program.cs – enterprise ready

Reply "**next**" and we go into **Stage 7 — DevOps & CI/CD** — Git branching, Azure DevOps pipelines, YAML, and deploying from code to production automatically like real enterprise teams. 🚀

20/02/2026, 16:50:09

21/02/2026, 10:14:26

Thought process: Inventoried completed stages and prepared remaining curriculum.

The user wants me to cover the remaining stages. Let me check what's been covered:

- ✅ Stage 1 – C# Deep Fundamentals
- ✅ Stage 2 – ASP.NET Core Web Development
- ✅ Stage 3 – SQL Server & Database
- ✅ Stage 4 – ORM & Entity Framework Core
- ✅ Stage 5 – Authentication & Security
- ✅ Stage 6 – Azure Cloud

Remaining:

- Stage 7 – DevOps & CI/CD
- Stage 8 – Clean Architecture
- Stage 9 – System Design Basics

- Stage 10 – Production-Level Knowledge

Now I need to cover all the remaining stages comprehensively.

Remaining Stages 7, 8, 9, 10

**Complete Final Guide — DevOps → Clean Architecture
→ System Design → Production**



STAGE 7 — DevOps & CI/CD



What Is DevOps?

Before tools, understand the concept:

OLD WAY (without DevOps):

Developer writes code → manually zips it → emails **to** server team

Server team manually deploys → hope **nothing** breaks

If something breaks → blame game between teams

Time **from** code **to** production: WEEKS

DevOps WAY:

Developer pushes code **to** git

Pipeline automatically:

- Runs tests

- Builds code

- Deploys **to** staging

- Runs more tests

- Deploys **to** production

Time **from** code **to** production: MINUTES

DevOps = Development + Operations working together with automation.



Lesson 1 — Git Branching Strategy

Real enterprise teams don't just push to main. They use a structured branching strategy.

GitFlow — Most Common in Enterprise



MAIN BRANCHES (always exist):

main → Production code. Always stable.
Only updated via Pull Request.

develop → Integration branch.
Where features merge before going to main.

SUPPORTING BRANCHES (temporary, deleted after merge):

feature/xxx → New feature development

bugfix/xxx → Fixing a bug

hotfix/xxx → Emergency fix directly to production

release/xxx → Preparing a release

FLOW:

1. New feature starts:

```
git checkout develop
```

```
git checkout -b feature/product-search
```

2. Developer works on feature

```
git add .
```

```
git commit -m "feat: add product search by name"
```

3. Feature done → Pull Request to develop

Code review happens here

4. Approved → merges to develop

Automated tests run

5. Release time:

```
git checkout -b release/1.2.0 from develop
```


Final testing on release branch

6. Release approved:

Merges to main (production deploy)

Merges back to develop (keep in sync)

7. Emergency bug in production:

git checkout -b hotfix/fix-payment-crash from main

Fix applied

Merges to BOTH main AND develop

Git Commands — Daily Usage in Real Projects

— SETUP —

Clone repository

git clone https://dev.azure.com/company/project/_git/shopapp-api

Set your identity

git config --global user.name "Ali Ahmed"

git config --global user.email "ali@company.com"

— DAILY WORKFLOW —

Always start by getting latest

git checkout develop

git pull origin develop

Create feature branch

git checkout -b feature/add-product-search

Check what changed

```
git status
git diff

# Stage specific files
git add src/Services/ProductService.cs
git add src/Controllers/ProductController.cs

# Stage all changes
git add .

# Commit with meaningful message
git commit -m "feat: add product search by name and category"

# Push your branch
git push origin feature/add-product-search

# — WORKING WITH BRANCHES —————

# List all branches
git branch -a

# Switch branch
git checkout develop

# Delete local branch after merge
git branch -d feature/add-product-search

# Delete remote branch after merge
git push origin --delete feature/add-product-search

# — KEEPING UP TO DATE —————

# Update your feature branch with latest develop changes
git checkout feature/add-product-search
git rebase origin/develop
```

```
# or
git merge origin/develop

# — UNDOING MISTAKES —————

# Undo last commit but keep changes
git reset --soft HEAD~1

# Undo changes in a file
git checkout -- src/Services/ProductService.cs

# See commit history
git log --oneline --graph

# — STASHING — save work temporarily —————

# Save current unfinished work
git stash push -m "work in progress on search"

# Get it back later
git stash pop
```

Commit Message Convention — Industry Standard

Format: type(scope): short description

Types:

feat	→ New feature
fix	→ Bug fix
docs	→ Documentation change
style	→ Code formatting (no logic change)
refactor	→ Code restructure (no feature/fix)

test → Adding tests
chore → Build process, dependencies

Examples:

feat(products): add search by category and price range
fix(auth): fix token expiry not being validated correctly
docs(readme): update deployment instructions
refactor(orders): extract order calculation to separate service
test(products): add unit tests for ProductService.Create
chore(deps): update EF Core to 8.0.2

Pull Request — What It Is and How It Works

A Pull Request (PR) = a REQUEST to pull YOUR code into another branch.

Flow:

1. You push feature branch
2. You open PR: "merge feature/search → develop"
3. Team members review your code
4. They leave comments, suggestions
5. You fix issues and push again
6. Reviewers approve
7. PR merged

PR should include:

Title: feat: add product search functionality

Description:

What does this PR do?

Adds search endpoint for products by name, category, and price

range.

How to test?

GET /api/products/search?name=laptop&minPrice=500

Screenshots (if UI change)

[screenshot here]

Checklist:

- ✓ Tests added
- ✓ No breaking changes
- ✓ appsettings updated
- ✓ Swagger docs updated

Lesson 2 — Azure DevOps

What Is Azure DevOps?

"Microsoft's all-in-one DevOps platform. Includes code repository, task boards, pipelines, and test management."

Azure DevOps has:

Boards → Track tasks, bugs, sprints (like Jira)

Repos → Git [repository](#) (like GitHub)

Pipelines → CI/CD automation

Test Plans → Test management

Artifacts → Package [management](#) (NuGet, npm)

Lesson 3 — Build Pipeline (CI)

CI = **Continuous Integration**

"Every time code is pushed, automatically build and test it."

Developer pushes **code**



Build Pipeline (CI)

1. Restore packages
2. Build solution
3. Run unit tests
4. Run **code** analysis
5. Create artifact

Pass →  Artifact ready **to** deploy

Fail →  Developer notified immediately

YAML Pipeline — Complete Build Pipeline

azure-pipelines.yml – put this in root of your project

```
trigger:
  branches:
    include:
      - main          # run when code pushed to main
      - develop       # run when code pushed to develop
```

```

paths:
  exclude:
    - README.md # don't run pipeline for doc changes

pool:
  vmImage: 'ubuntu-latest' # Microsoft-hosted build agent

variables:
  buildConfiguration: 'Release'
  dotnetVersion: '8.0.x'

stages:

# =====
# STAGE 1 – BUILD & TEST
# =====
- stage: Build
  displayName: 'Build and Test'
  jobs:
    - job: BuildJob
      displayName: 'Build, Test, Publish'
      steps:

        # Step 1 – Setup .NET
        - task: UseDotNet@2
          displayName: 'Install .NET $(dotnetVersion)'
          inputs:
            version: $(dotnetVersion)
            packageType: sdk

        # Step 2 – Restore NuGet packages
        - task: DotNetCoreCLI@2
          displayName: 'Restore packages'
          inputs:
            command: restore

```

```
    projects: '**/*.csproj'

# Step 3 – Build
- task: DotNetCoreCLI@2
  displayName: 'Build solution'
  inputs:
    command: build
    projects: '**/*.csproj'
    arguments: '--configuration $(buildConfiguration) --no-restore'

# Step 4 – Run unit tests
- task: DotNetCoreCLI@2
  displayName: 'Run unit tests'
  inputs:
    command: test
    projects: '**/*Tests/*.csproj'
    arguments: '--configuration $(buildConfiguration) --no-build --collect:"XPlat Code Coverage"'

# Step 5 – Publish test results
- task: PublishTestResults@2
  displayName: 'Publish test results'
  inputs:
    testResultsFormat: 'VSTest'
    testResultsFiles: '**/*.trx'

# Step 6 – Publish code coverage
- task: PublishCodeCoverageResults@1
  displayName: 'Publish code coverage'
  inputs:
    codeCoverageTool: 'Cobertura'
    summaryFileLocation: '$(Agent.TempDirectory)/**/coverage.cobertura.xml'
```



```

# Step 7 – Publish app (create deployable artifact)
- task: DotNetCoreCLI@2
  displayName: 'Publish app'
  inputs:
    command: publish
    publishWebProjects: true
    arguments: '--configuration $(buildConfiguration) --
output $(Build.ArtifactStagingDirectory)'
    zipAfterPublish: true

# Step 8 – Save artifact for release pipeline
- task: PublishBuildArtifacts@1
  displayName: 'Save build artifact'
  inputs:
    PathToPublish: $(Build.ArtifactStagingDirectory)
    ArtifactName: 'shopapp-api'
    publishLocation: 'Container'

```

Lesson 4 — Release Pipeline (CD)

CD = **Continuous Delivery / Continuous Deployment**

"Automatically deploy the built artifact to different environments."

Build **Artifact** (from CI)



Development	← auto deploy, no approval
app-shopapp-dev	

|
▼ (manual approval gate)

QA / Staging	← QA team tests here
app-shopapp-staging	

|
▼ (manager approval gate)

Production	← live for real users
app-shopapp-prod	

Complete YAML — Build + Deploy All Environments

```
# azure-pipelines.yml – full CI/CD pipeline
```

```
trigger:
  branches:
    include:
      - main

pool:
  vmImage: 'ubuntu-latest'

variables:
  buildConfiguration: 'Release'
  dotnetVersion: '8.0.x'
  resourceGroup: 'rg-shopapp-production'

stages:
```

```
# =====  
# STAGE 1 – BUILD  
# =====  
  
- stage: Build  
  displayName: '🔨 Build & Test'  
  jobs:  
    - job: Build  
      steps:  
        - task: UseDotNet@2  
          inputs:  
            version: $(dotnetVersion)  
  
        - task: DotNetCoreCLI@2  
          displayName: 'Restore'  
          inputs:  
            command: restore  
            projects: '**/*.csproj'  
  
        - task: DotNetCoreCLI@2  
          displayName: 'Build'  
          inputs:  
            command: build  
            arguments: '--configuration $(buildConfiguration) --no-  
restore'  
  
        - task: DotNetCoreCLI@2  
          displayName: 'Test'  
          inputs:  
            command: test  
            projects: '**/*Tests/*.csproj'  
            arguments: '--no-build'  
  
        - task: DotNetCoreCLI@2  
          displayName: 'Publish'  
          inputs:
```

```

        command: publish
        arguments: '--configuration $(buildConfiguration) --
output $(Build.ArtifactStagingDirectory)'
        zipAfterPublish: true

    - publish: $(Build.ArtifactStagingDirectory)
      artifact: drop

# =====
# STAGE 2 – DEPLOY TO DEVELOPMENT
# =====

- stage: DeployDev
  displayName: '🚀 Deploy to Development'
  dependsOn: Build
  condition: succeeded() # only if build passed
  jobs:
    - deployment: DeployDev
      environment: 'Development' # environment defined in Azure
DevOps
      strategy:
        runOnce:
          deploy:
            steps:
              - task: AzureWebApp@1
                displayName: 'Deploy to Dev App Service'
                inputs:
                  azureSubscription: 'Azure-Service-Connection'
                  appType: 'webApp'
                  appName: 'app-shopapp-dev'
                  package: $(Pipeline.Workspace)/drop/*.zip
                  deploymentMethod: 'auto'

# =====
# STAGE 3 – DEPLOY TO STAGING
# =====

```

```

- stage: DeployStaging
  displayName: '🔪 Deploy to Staging'
  dependsOn: DeployDev
  condition: succeeded()
  jobs:
    - deployment: DeployStaging
      environment: 'Staging' # has approval gate configured
      strategy:
        runOnce:
          deploy:
            steps:
              # Run database migrations before deploy
              - task: AzureCLI@2
                displayName: 'Run DB migrations'
                inputs:
                  azureSubscription: 'Azure-Service-Connection'
                  scriptType: 'bash'
                  scriptLocation: 'inlineScript'
                  inlineScript: |
                    cd $(Pipeline.Workspace)/drop
                    dotnet ef database update \
                      --connection "$(StagingConnectionString)"

              - task: AzureWebApp@1
                displayName: 'Deploy to Staging Slot'
                inputs:
                  azureSubscription: 'Azure-Service-Connection'
                  appType: 'webApp'
                  appName: 'app-shopapp-prod'
                  deployToSlotOrASE: true
                  slotName: 'staging'
                  package: $(Pipeline.Workspace)/drop/*.zip

```

```

# =====
# STAGE 4 – DEPLOY TO PRODUCTION

```

```
#  
  
- stage: DeployProduction  
  displayName: '🏭 Deploy to Production'  
  dependsOn: DeployStaging  
  condition: succeeded()  
  jobs:  
    - deployment: DeployProduction  
      environment: 'Production' # requires manager approval  
      strategy:  
        runOnce:  
          deploy:  
            steps:  
              # Swap staging slot → production (zero downtime)  
              - task: AzureAppServiceManage@0  
                displayName: 'Swap staging → production'  
                inputs:  
                  azureSubscription: 'Azure-Service-Connection'  
                  Action: 'Swap Slots'  
                  WebAppName: 'app-shopapp-prod'  
                  ResourceGroupName: $(resourceGroup)  
                  SourceSlot: 'staging'  
                  SwapWithProduction: true  
  
              # Notify team  
              - task: SendEmail@1  
                displayName: 'Notify team'  
                inputs:  
                  To: 'team@company.com'  
                  Subject: '✅ ShopApp deployed to production'  
                  Body: 'New version deployed successfully.'
```

Environment Promotion — The Full Picture

Code pushed to main branch



CI Build runs

- └─ restore packages ✓
- └─ build ✓
- └─ unit tests ✓ (must pass)
- └─ artifact created ✓



Auto deploy → Development

- └─ Smoke test runs automatically ✓



Auto deploy → Staging

- └─ QA team tests manually
- └─ Regression tests run ✓



[Manager approves in Azure DevOps portal]



Deploy → Production

- └─ Swap deployment slots (zero downtime)
- └─ Health check runs ✓
- └─ Team notified

STAGE 8 — Clean Architecture

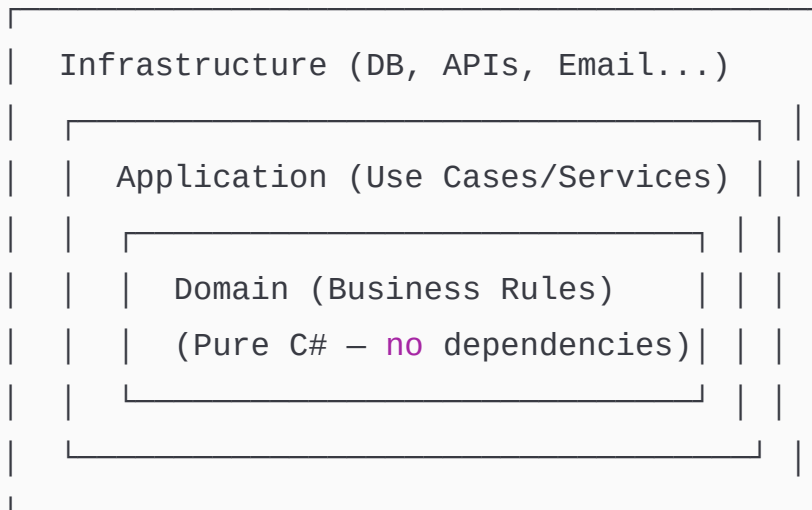
What Is Clean Architecture?

"A way to organize your code so business rules are completely independent of frameworks, databases, and external tools."

Invented by Robert C. Martin (Uncle Bob).

The core idea:

Dependencies point INWARD **only**:



Outer layers know about **inner** layers

Inner layers know NOTHING about **outer** layers

Domain doesn't know about EF Core

Domain doesn't know about ASP.NET

Domain doesn't know about SQL Server

→ You can swap any outer layer without touching business logic

Project Structure — Clean Architecture

ShopApp.sln

|


```

├─ ShopApp.Domain/          ← innermost – pure business logic
|   ├─ Entities/
|   ├─ Interfaces/
|   ├─ Exceptions/
|   └─ ValueObjects/
|
├─ ShopApp.Application/     ← use cases / orchestration
|   ├─ Services/
|   ├─ DTOs/
|   ├─ Interfaces/
|   └─ Validators/
|
├─ ShopApp.Infrastructure/  ← external concerns
|   ├─ Persistence/        (EF Core, DbContext)
|   ├─ Repositories/
|   ├─ Email/
|   └─ Storage/
|
└─ ShopApp.API/            ← entry point (ASP.NET Core)
    ├─ Controllers/
    ├─ Middleware/
    └─ Program.cs

```

Layer 1 — Domain (Innermost, No Dependencies)

```

// ShopApp.Domain/Entities/Product.cs
// NO EF Core, NO ASP.NET, NO nothing – pure C#
namespace ShopApp.Domain.Entities
{
    public class Product
    {
        public int Id { get; private set; }
    }
}

```

```

public string Name { get; private set; }
public decimal Price { get; private set; }
public int Stock { get; private set; }
public bool IsActive { get; private set; }
public int CategoryId { get; private set; }
public DateTime CreatedAt { get; private set; }

// Private constructor – force use of factory method
private Product() { }

// Factory method – validates before creating
public static Product Create(
    string name, decimal price,
    int stock, int categoryId)
{
    if (string.IsNullOrEmpty(name))
        throw new DomainException("Product name is
required.");

    if (price < 0)
        throw new DomainException("Price cannot be
negative.");

    if (stock < 0)
        throw new DomainException("Stock cannot be
negative.");

    return new Product
    {
        Name          = name,
        Price          = price,
        Stock          = stock,
        CategoryId     = categoryId,
        IsActive       = true,
        CreatedAt      = DateTime.UtcNow
    }
}

```

```

        };
    }

    // Domain methods – business rules live here
    public void UpdatePrice(decimal newPrice)
    {
        if (newPrice < 0)
            throw new DomainException("Price cannot be
negative.");

        Price = newPrice;
    }

    public void AddStock(int quantity)
    {
        if (quantity <= 0)
            throw new DomainException("Quantity must be
positive.");

        Stock += quantity;
    }

    public bool TrySell(int quantity)
    {
        if (quantity <= 0) return false;
        if (quantity > Stock) return false;

        Stock -= quantity;
        return true;
    }

    public void Deactivate() => IsActive = false;
    public void Activate()   => IsActive = true;
}
}

```

```
// ShopApp.Domain/Exceptions/DomainException.cs
namespace ShopApp.Domain.Exceptions
{
    // Custom domain exception – thrown when business rule
    violated
    public class DomainException : Exception
    {
        public DomainException(string message) : base(message) { }
    }

    public class NotFoundException : Exception
    {
        public NotFoundException(string name, object key)
            : base($"{name} with id '{key}' was not found.") { }
    }
}
```

```
// ShopApp.Domain/Interfaces/IPProductRepository.cs
// Interface lives in Domain – implementation lives in
Infrastructure
namespace ShopApp.Domain.Interfaces
{
    public interface IPProductRepository
    {
        Task<Product?> GetByIdAsync(int id);
        Task<List<Product>> GetAllAsync();
        Task<Product> AddAsync(Product product);
        Task UpdateAsync(Product product);
        Task DeleteAsync(Product product);
    }
}
```

Layer 2 — Application (Use Cases)

```
// ShopApp.Application/DTOs/ProductDtos.cs
namespace ShopApp.Application.DTOs
{
    public record CreateProductRequest(
        string Name,
        decimal Price,
        int Stock,
        int CategoryId);

    public record UpdateProductRequest(
        string Name,
        decimal Price,
        int Stock);

    public record ProductResponse(
        int Id,
        string Name,
        decimal Price,
        int Stock,
        bool IsActive,
        string StockStatus,
        DateTime CreatedAt);
}
```

```
// ShopApp.Application/Services/ProductService.cs
using ShopApp.Domain.Entities;
using ShopApp.Domain.Exceptions;
using ShopApp.Domain.Interfaces;
using ShopApp.Application.DTOs;

namespace ShopApp.Application.Services
{

```

```

public interface IProductService
{
    Task<ProductResponse> CreateAsync(CreateProductRequest
request);
    Task<ProductResponse?> GetByIdAsync(int id);
    Task<List<ProductResponse>> GetAllAsync();
    Task<ProductResponse> UpdateAsync(int id,
UpdateProductRequest request);
    Task DeleteAsync(int id);
}

public class ProductService : IProductService
{
    private readonly IProductRepository _repository;
    private readonly ILogger<ProductService> _logger;

    public ProductService(
        IProductRepository repository,
        ILogger<ProductService> logger)
    {
        _repository = repository;
        _logger      = logger;
    }

    public async Task<ProductResponse> CreateAsync(
        CreateProductRequest request)
    {
        // Domain handles validation via factory method
        var product = Product.Create(
            request.Name,
            request.Price,
            request.Stock,
            request.CategoryId);

        await _repository.AddAsync(product);
    }
}

```

```

        _logger.LogInformation(
            "Product created: {Id} - {Name}",
            product.Id, product.Name);

        return MapToResponse(product);
    }

    public async Task<ProductResponse?> GetByIdAsync(int id)
    {
        var product = await _repository.GetByIdAsync(id);
        return product == null ? null :
            MapToResponse(product);
    }

    public async Task<List<ProductResponse>> GetAllAsync()
    {
        var products = await _repository.GetAllAsync();
        return products.Select(MapToResponse).ToList();
    }

    public async Task<ProductResponse> UpdateAsync(
        int id, UpdateProductRequest request)
    {
        var product = await _repository.GetByIdAsync(id)
            ?? throw new NotFoundException(nameof(Product),
id);

        // Domain method handles business rule validation
        product.UpdatePrice(request.Price);

        await _repository.UpdateAsync(product);

        return MapToResponse(product);
    }

```

```

    public async Task DeleteAsync(int id)
    {
        var product = await _repository.GetByIdAsync(id)
            ?? throw new NotFoundException(nameof(Product),
id);

        product.Deactivate(); // soft delete via domain
method
        await _repository.UpdateAsync(product);
    }

    private static ProductResponse MapToResponse(Product p)
=>
        new(
            p.Id,
            p.Name,
            p.Price,
            p.Stock,
            p.IsActive,
            p.Stock == 0 ? "Out of Stock"
            : p.Stock <= 5 ? "Low Stock"
            : "In Stock",
            p.CreatedAt
        );
    }
}

```

Layer 3 — Infrastructure (EF Core Implementation)

```

// ShopApp.Infrastructure/Persistence/AppDbContext.cs
using Microsoft.EntityFrameworkCore;

```



```

using ShopApp.Domain.Entities;

namespace ShopApp.Infrastructure.Persistence
{
    public class AppDbContext : DbContext
    {
        public AppDbContext(DbContextOptions<AppDbContext>
options)
            : base(options) { }

        public DbSet<Product> Products { get; set; }

        protected override void OnModelCreating(ModelBuilder
modelBuilder)
        {
            modelBuilder.Entity<Product>(entity =>
            {
                entity.HasKey(p => p.Id);
                entity.Property(p =>
p.Name).IsRequired().HasMaxLength(100);
                entity.Property(p =>
p.Price).HasColumnType("decimal(18,2)");
            });
        }
    }
}

```

```

// ShopApp.Infrastructure/Repositories/ProductRepository.cs
using Microsoft.EntityFrameworkCore;
using ShopApp.Domain.Entities;
using ShopApp.Domain.Interfaces;
using ShopApp.Infrastructure.Persistence;

namespace ShopApp.Infrastructure.Repositories
{

```

```
// IMPLEMENTS Domain interface – Domain has no idea this exists
```

```
public class ProductRepository : IProductRepository
{
    private readonly AppDbContext _context;

    public ProductRepository(AppDbContext context)
    {
        _context = context;
    }

    public async Task<Product?> GetByIdAsync(int id) =>
        await _context.Products
            .AsNoTracking()
            .FirstOrDefaultAsync(p => p.Id == id);

    public async Task<List<Product>> GetAllAsync() =>
        await _context.Products
            .Where(p => p.IsActive)
            .AsNoTracking()
            .OrderBy(p => p.Name)
            .ToListAsync();

    public async Task<Product> AddAsync(Product product)
    {
        _context.Products.Add(product);
        await _context.SaveChangesAsync();
        return product;
    }

    public async Task UpdateAsync(Product product)
    {
        _context.Products.Update(product);
        await _context.SaveChangesAsync();
    }
}
```

```

        public async Task DeleteAsync(Product product)
        {
            _context.Products.Remove(product);
            await _context.SaveChangesAsync();
        }
    }
}

```

Layer 4 — API (Entry Point)

```

// ShopApp.API/Controllers/ProductController.cs
using Microsoft.AspNetCore.Mvc;
using ShopApp.Application.DTOS;
using ShopApp.Application.Services;
using ShopApp.Domain.Exceptions;

namespace ShopApp.API.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class ProductController : ControllerBase
    {
        private readonly IProductService _productService;

        public ProductController(IProductService productService)
        {
            _productService = productService;
        }

        [HttpGet]
        public async Task<IActionResult> GetAll()

```

```

{
    var products = await _productService.GetAllAsync();
    return Ok(products);
}

[HttpGet("{id}")]
public async Task<IActionResult> GetById(int id)
{
    var product = await _productService.GetByIdAsync(id);
    if (product == null) return NotFound();
    return Ok(product);
}

[HttpPost]
public async Task<IActionResult> Create(
    [FromBody] CreateProductRequest request)
{
    var product = await
_productService.CreateAsync(request);
    return CreatedAtAction(
        nameof(GetById), new { id = product.Id },
product);
}

[HttpPut("{id}")]
public async Task<IActionResult> Update(
    int id, [FromBody] UpdateProductRequest request)
{
    var product = await _productService.UpdateAsync(id,
request);
    return Ok(product);
}

[HttpDelete("{id}")]
public async Task<IActionResult> Delete(int id)

```

```

        {
            await _productService.DeleteAsync(id);
            return NoContent();
        }
    }
}

```

```

// ShopApp.API/Program.cs – wire all layers together
using Microsoft.EntityFrameworkCore;
using ShopApp.Application.Services;
using ShopApp.Domain.Interfaces;
using ShopApp.Infrastructure.Persistence;
using ShopApp.Infrastructure.Repositories;

var builder = WebApplication.CreateBuilder(args);

// Infrastructure
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("DefaultConnection")));

// Dependency wiring – Domain interface ← Infrastructure
// implementation
builder.Services.AddScoped<IProductRepository, ProductRepository>
();

// Application services
builder.Services.AddScoped<IProductService, ProductService>();

builder.Services.AddControllers();

var app = builder.Build();
app.UseAuthorization();

```

```
app.MapControllers();
app.Run();
```

SOLID Principles — Simple Explanation

SOLID = 5 principles that make code clean and maintainable.

S – Single Responsibility Principle

"A class should have only ONE reason to change"

 BAD:

```
public class OrderService
{
    public void ProcessOrder() { }    // business logic
    public void SendEmail() { }       // email logic
    public void SaveToDatabase() { }  // data logic
    // 3 reasons to change = violation
}
```

 GOOD:

```
public class OrderService    { public void ProcessOrder() { } }
public class EmailService    { public void SendEmail() { } }
public class OrderRepository { public void Save() { } }
// Each class = one job = one reason to change
```

O – Open/Closed Principle

"Open for extension, closed for modification"

Add new features by adding new code, not changing existing code

❌ BAD:

```
public decimal CalculateDiscount(string customerType)
{
    if (customerType == "VIP") return 0.2m;
    if (customerType == "Regular") return 0.1m;
    // Adding "Premium" = edit this method = risky!
}
```

✅ GOOD:

```
public interface IDiscountStrategy
{
    decimal Calculate(decimal price);
}

public class VipDiscount : IDiscountStrategy { ... }
public class RegularDiscount : IDiscountStrategy { ... }
public class PremiumDiscount : IDiscountStrategy { ... }
// Add new type = add new class = existing code untouched
```

L – Liskov Substitution Principle

"Child classes must be usable in place of parent class"

❌ BAD:

```
public class Bird { public virtual void Fly() { } }
public class Penguin : Bird
{
    public override void Fly()
    {
        throw new Exception("Penguins can't fly!"); // breaks LSP
    }
}
```

✅ GOOD:

```
public class Bird { }
```

```
public interface IFlyable { void Fly(); }
public class Eagle : Bird, IFlyable { public void Fly() { } }
public class Penguin : Bird { } // doesn't implement IFlyable
```

I – Interface Segregation Principle

"Don't force classes to implement methods they don't need"

❌ BAD:

```
public interface IWorker
{
    void Work();
    void Eat(); // robots don't eat!
}
public class Robot : IWorker
{
    public void Work() { }
    public void Eat() { throw new Exception(); } // forced,
meaningless
}
```

✅ GOOD:

```
public interface IWorkable { void Work(); }
public interface IEatable { void Eat(); }
public class Human : IWorkable, IEatable { ... }
public class Robot : IWorkable { ... } // only what it
needs
```

D – Dependency Inversion Principle

"Depend on abstractions (interfaces), not concrete classes"

❌ BAD:


```
public class OrderService
{
    private SmtplibEmailService _email = new SmtplibEmailService(); //
    concrete!
    // Can't swap email provider without changing OrderService
}
```

✅ GOOD:

```
public class OrderService
{
    private readonly IEmailService _email; // abstraction!
    public OrderService(IEmailService email) { _email = email; }
    // Inject any email implementation – OrderService doesn't
    care
}
```



STAGE 9 — System Design Basics



What Is Load Balancer?

WITHOUT Load Balancer:

All 10,000 users → one server → server dies

WITH Load Balancer:

10,000 users → Load Balancer → distributes across 5 servers

— Server 1 (2,000 users)

— Server 2 (2,000 users)

— Server 3 (2,000 users)

— Server 4 (2,000 users)



└─ Server 5 (2,000 users)

If Server 3 crashes → LB detects it → routes to remaining 4
Users don't notice anything

In Azure → Azure Load Balancer or Application Gateway does this
App Service with multiple instances → already has built-in load
balancing

What Is Caching?

WITHOUT caching:

User requests "Top 10 Products"

- API hits database
- Database runs query (takes 200ms)
- Returns data
- EVERY request does this = expensive, slow

WITH caching:

First request:

- API hits database (200ms)
- Stores result in cache for 5 minutes

Next 1000 requests in those 5 minutes:

- API reads from cache (2ms)
- Database not touched at all
- 100x faster, 1000x less DB load

Implement Caching in ASP.NET Core

```
// Program.cs – add memory cache
builder.Services.AddMemoryCache();

// For distributed cache (Redis – shared across multiple instances)
builder.Services.AddStackExchangeRedisCache(options =>
{
    options.Configuration =
builder.Configuration["Redis:ConnectionString"];
});
```

```
// Services/ProductService.cs – with caching
using Microsoft.Extensions.Caching.Memory;

public class ProductService : IProductService
{
    private readonly IProductRepository _repository;
    private readonly IMemoryCache _cache;

    // Cache key constants
    private const string AllProductsCacheKey = "all-products";
    private const string ProductCacheKeyPrefix = "product-";

    public ProductService(
        IProductRepository repository,
        IMemoryCache cache)
    {
        _repository = repository;
        _cache      = cache;
    }

    public async Task<List<ProductResponse>> GetAllAsync()
    {
        // Try cache first
```

```

    if (_cache.TryGetValue(AllProductsCacheKey,
        out List<ProductResponse>? cached))
    {
        return cached!; // cache hit – return immediately
    }

    // Cache miss – get from database
    var products = await _repository.GetAllAsync();
    var response = products.Select(MapToResponse).ToList();

    // Store in cache for 5 minutes
    var cacheOptions = new MemoryCacheEntryOptions
    {
        AbsoluteExpirationRelativeToNow =
        TimeSpan.FromMinutes(5),
        SlidingExpiration                =
        TimeSpan.FromMinutes(2),
        Priority                          =
        CacheItemPriority.Normal
    };

    _cache.Set(AllProductsCacheKey, response, cacheOptions);

    return response;
}

public async Task<ProductResponse> CreateAsync(
    CreateProductRequest request)
{
    var product = Product.Create(
        request.Name, request.Price,
        request.Stock, request.CategoryId);

    await _repository.AddAsync(product);
}

```

```
// Invalidate cache – data changed
_cache.Remove(AllProductsCacheKey);

return MapToResponse(product);
}
}
```

Scalability — Vertical vs Horizontal

VERTICAL SCALING (Scale Up):

Make the server BIGGER

1 server: 2 CPU, 4GB RAM → handles 1,000 users
Upgrade to: 8 CPU, 32GB RAM → handles 4,000 users

Pros: Simple, no code changes

Cons: Expensive, has hardware limits, single point of failure

Azure: Change App Service Plan tier (B1 → P3v3)

HORIZONTAL SCALING (Scale Out):

Add MORE servers (copies of your app)

1 server → handles 1,000 users
Add 4 more servers = 5 total → handles 5,000 users

Pros: Near unlimited scale, no single point of failure

Cons: App must be stateless, more complex

Azure: App Service auto-scaling (1 → 10 instances)

FOR WEB APIs → always design for HORIZONTAL scaling

Your API must be STATELESS:

- No session stored on server
- Use JWT (token carries state)
- Use distributed cache (Redis) if needed
- Database is shared – not per-server

Microservices vs Monolith

MONOLITH – one big application:

ShopApp.API (one project, one deployment)

- ├ Products
- ├ Orders
- ├ Users
- ├ Payments
- └ Notifications

Pros:

- ✓ Simple to develop initially
- ✓ Simple to deploy (one thing)
- ✓ Easy to debug (one codebase)
- ✓ Good for small teams and startups

Cons:

- ✗ All teams step on each other's code
 - ✗ Must redeploy entire app for any change
 - ✗ One crash can take down everything
 - ✗ Can't scale individual parts (must scale all)
-

MICROSERVICES – separate small services:

product-service → manages products
order-service → manages orders
user-service → manages users
payment-service → manages payments
notification-service → sends emails/SMS

Each is independent:

- Separate codebase
- Separate database
- Separate deployment
- Communicate via HTTP or messaging

Pros:

- ✓ Teams work independently
- ✓ Deploy one service without touching others
- ✓ Scale only what needs scaling (scale payment service only)
- ✓ One service fails → others keep working
- ✓ Different services can use different tech

Cons:

- ✗ Complex infrastructure
- ✗ Network calls between services (latency)
- ✗ Distributed system problems (harder to debug)
- ✗ Needs DevOps maturity

REALITY – when to use what:

Starting out / small team → Monolith
Growing, 5+ dev teams → Modular Monolith first
Enterprise, 20+ teams, high scale → Microservices

API Gateway

Without API Gateway:

Client → calls product-service directly

Client → calls **order**-service directly

Client → calls **user**-service directly

Client knows **all** internal URLs

Must authenticate **with each** service

CORS configured **on each** service

With API Gateway:

Client → API Gateway (single entry point)

└─ routes /api/products → product-service

└─ routes /api/orders → **order**-service

└─ routes /api/users → **user**-service

API Gateway handles:

→ Authentication (**one place for all**)

→ Rate limiting

→ Logging

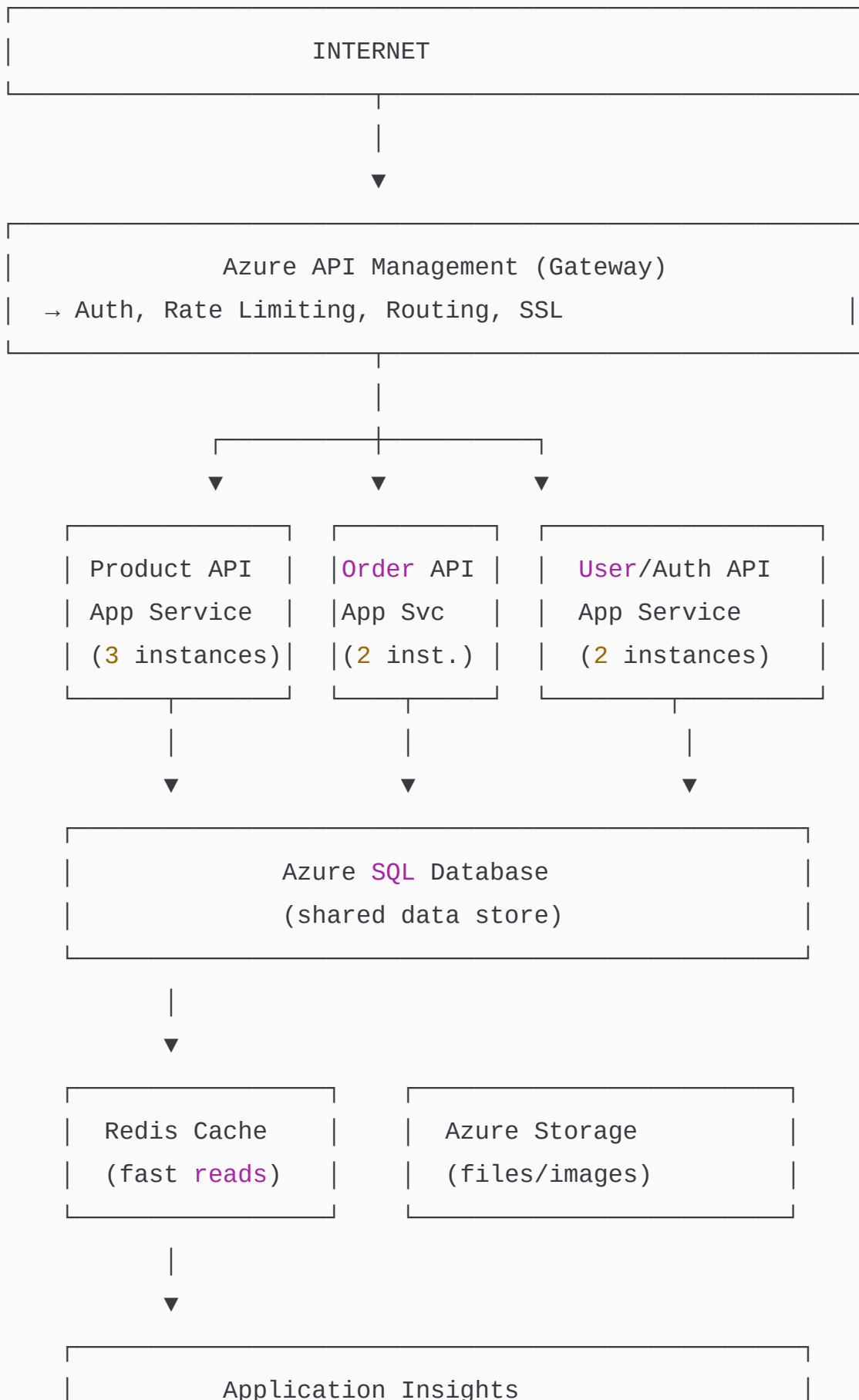
→ CORS

→ Load balancing

→ SSL termination

In Azure: Azure API Management (APIM)

Simple System Design Example — ShopApp Architecture



(monitoring **all** services)

STAGE 10 — Production-Level Knowledge

Logging Best Practices

```
// Program.cs – structured logging setup
using Serilog;
using Serilog.Events;

// Configure Serilog before anything else
Log.Logger = new LoggerConfiguration()
    .MinimumLevel.Information()
    .MinimumLevel.Override("Microsoft", LogEventLevel.Warning)
    .MinimumLevel.Override("Microsoft.EntityFrameworkCore",
LogEventLevel.Warning)
    .Enrich.FromLogContext()
    .Enrich.WithMachineName()
    .Enrich.WithEnvironmentName()
    .WriteTo.Console(outputTemplate:
        "[{Timestamp:HH:mm:ss} {Level:u3}] {Message:l} {NewLine}
{Exception}")
    .WriteTo.ApplicationInsights(

builder.Configuration["ApplicationInsights:ConnectionString"],
    TelemetryConverter.Traces)
```

```

        .CreateLogger();

builder.Host.UseSerilog();

// Logging BEST PRACTICES in services

public class ProductService : IProductService
{
    private readonly ILogger<ProductService> _logger;

    // ✅ Structured logging – use placeholders, not string
    concat
    public async Task<ProductResponse>
CreateAsync(CreateProductRequest req)
    {
        // ✅ GOOD – structured, searchable in Application
        Insights
        _logger.LogInformation(
            "Creating product: {Name} at price {Price}",
            req.Name, req.Price);

        // ❌ BAD – string concatenation, not searchable
        _logger.LogInformation("Creating product: " + req.Name);

        try
        {
            var product = await _repository.CreateAsync(req);

            // Log important business events
            _logger.LogInformation(
                "Product created successfully. Id: {ProductId},
Name: {Name}",
                product.Id, product.Name);

            return product;
        }
    }
}

```

```

    }
    catch (DomainException ex)
    {
        // Business rule violations – Warning level
        _logger.LogWarning(
            "Domain validation failed for product {Name}:
{Error}",
            req.Name, ex.Message);
        throw;
    }
    catch (Exception ex)
    {
        // Unexpected errors – Error level
        _logger.LogError(ex,
            "Unexpected error creating product {Name}",
            req.Name);
        throw;
    }
}
}

```

Log Levels — When to Use Each

```

// Use the RIGHT log level:

_logger.LogTrace("Entering method GetById with id={Id}", id);
// Trace → extremely detailed, every step. Dev only.

_logger.LogDebug("Cache miss for product {Id}, querying
database", id);
// Debug → useful for debugging. Dev/staging only.

```

```
_logger.LogInformation("User {Email} logged in successfully",
email);
// Information → normal business events. Production on.

_logger.LogWarning("Product {Id} stock is low: {Stock}
remaining", id, stock);
// Warning → something unexpected but handled. Pay attention.

_logger.LogError(ex, "Failed to process payment for order
{OrderId}", orderId);
// Error → something failed, needs investigation. Alert team.

_logger.LogCritical(ex, "Database connection failed. System
unavailable.");
// Critical → system is down. Wake someone up at 3AM.
```

Global Exception Handling — Production Grade

```
// Middleware/GlobalExceptionMiddleware.cs
using System.Net;
using System.Text.Json;
using ShopApp.Domain.Exceptions;

public class GlobalExceptionMiddleware
{
    private readonly RequestDelegate _next;
    private readonly ILogger<GlobalExceptionMiddleware> _logger;

    public GlobalExceptionMiddleware(
        RequestDelegate next,
        ILogger<GlobalExceptionMiddleware> logger)
    {
```

```

        _next    = next;
        _logger = logger;
    }

    public async Task InvokeAsync(HttpContext context)
    {
        try
        {
            await _next(context);
        }
        catch (Exception ex)
        {
            await HandleExceptionAsync(context, ex);
        }
    }

    private async Task HandleExceptionAsync(
        HttpContext context, Exception ex)
    {
        var traceId = context.TraceIdentifier;

        // Map exception type to HTTP status
        var (statusCode, userMessage) = ex switch
        {
            DomainException          => (400, ex.Message),
            NotFoundException         => (404, ex.Message),
            UnauthorizedAccessException
                                    => (401, "Authentication
required."),
            InvalidOperationException   => (409, ex.Message),
            ArgumentException         => (400, ex.Message),
            _                         => (500, "An unexpected error
occurred.")
        };
    }

```

```

// Log properly – 500s are errors, 4xx are warnings
if (statusCode >= 500)
{
    _logger.LogError(ex,
        "Unhandled exception. TraceId: {TraceId}, Path:
{Path}",
        traceId, context.Request.Path);
}
else
{
    _logger.LogWarning(
        "Handled exception {ExType}. TraceId: {TraceId},
Message: {Message}",
        ex.GetType().Name, traceId, ex.Message);
}

context.Response.StatusCode = statusCode;
context.Response.ContentType = "application/json";

// In production – NEVER return stack trace to client
var response = new
{
    error    = userMessage,
    traceId  = traceId,    // client gives this to support
    timestamp = DateTime.UtcNow
};

var json = JsonSerializer.Serialize(response,
    new JsonSerializerOptions
    {
        PropertyNamingPolicy = JsonNamingPolicy.CamelCase
    });

await context.Response.WriteAsync(json);

```

```
}  
}
```

API Versioning

"When you change your API, old clients must not break. Versioning solves this."

```
dotnet add package Asp.Versioning.Mvc
```

```
// Program.cs – add versioning  
builder.Services.AddApiVersioning(options =>  
{  
    options.DefaultApiVersion          = new ApiVersion(1,  
0);  
    options.AssumeDefaultVersionWhenUnspecified = true;  
    options.ReportApiVersions          = true; // shows  
versions in response headers  
})  
.AddApiExplorer(options =>  
{  
    options.GroupNameFormat          = "'v'VVV";  
    options.SubstituteApiVersionInUrl = true;  
});
```

```
// Controllers/v1/ProductController.cs  
using Asp.Versioning;  
  
namespace ShopApp.API.Controllers.V1  
{  
    [ApiController]  
    [ApiVersion("1.0")]
```



```

[Route("api/v{version:apiVersion}/[controller]")]
public class ProductController : ControllerBase
{
    [HttpGet]
    public IActionResult GetAll()
    {
        return Ok(new { version = "v1", message = "Products
from v1" });
    }
}

// Controllers/v2/ProductController.cs
namespace ShopApp.API.Controllers.V2
{
    [ApiController]
    [ApiVersion("2.0")]
    [Route("api/v{version:apiVersion}/[controller]")]
    public class ProductController : ControllerBase
    {
        [HttpGet]
        public IActionResult GetAll()
        {
            // V2 returns richer data, different shape
            return Ok(new { version = "v2", message = "Enhanced
products from v2" });
        }
    }
}

// Now you have:
// GET api/v1/product → old format (still works for old clients)
// GET api/v2/product → new format (new clients use this)

```

Performance Optimization

// 1. Async all the way – never block threads

// ❌ BAD

```
public IActionResult GetProducts()
{
    var products = _repository.GetAllAsync().Result; // BLOCKING!
    return Ok(products);
}
```

// ✅ GOOD

```
public async Task<IActionResult> GetProducts()
{
    var products = await _repository.GetAllAsync(); // non-
    blocking
    return Ok(products);
}
```

// _____

// 2. Response compression – send less data over network

// Program.cs

```
builder.Services.AddResponseCompression(options =>
{
    options.EnableForHttps = true;
    options.Providers.Add<GzipCompressionProvider>();
});
app.UseResponseCompression();
```

// _____

// 3. Response caching – cache entire HTTP responses

```
builder.Services.AddResponseCaching();
app.UseResponseCaching();
```

```

[HttpGet]
[ResponseCache(Duration = 60, VaryByQueryKeys = new[] {
    "category" })]
public async Task<IActionResult> GetAll([FromQuery] string?
category)
{
    // Response cached for 60 seconds
    // Different cache per category query param
    return Ok(await _service.GetAllAsync(category));
}

```

```

// 4. Output caching (ASP.NET 7+) – more powerful
builder.Services.AddOutputCache(options =>
{
    options.AddPolicy("products", builder =>
        builder.Expire(TimeSpan.FromMinutes(5))
            .SetVaryByQuery("category", "page"));
});
app.UseOutputCache();

```

```

[HttpGet]
[OutputCache(PolicyName = "products")]
public async Task<IActionResult> GetAll()
{
    return Ok(await _service.GetAllAsync());
}

```

```

// 5. IAsyncEnumerable – stream large results
[HttpGet("stream")]
public async IAsyncEnumerable<ProductResponse> StreamAll(
    [EnumeratorCancellation] CancellationToken ct)

```

```
{  
    // Streams products one by one instead of loading all into  
    memory  
    await foreach (var product in _repository.StreamAllAsync(ct))  
    {  
        yield return MapToResponse(product);  
    }  
}
```

Memory Leak Detection

// Common memory leak causes and how to fix them:

// ❌ LEAK 1 – Not disposing resources

```
public async Task<string> ReadFile(string path)  
{  
    var reader = new StreamReader(path); // NOT disposed!  
    return await reader.ReadToEndAsync();  
}
```

// ✅ FIX – use using statement

```
public async Task<string> ReadFile(string path)  
{  
    using var reader = new StreamReader(path); // auto-disposed  
    return await reader.ReadToEndAsync();  
}
```

// _____

// ❌ LEAK 2 – Static event handler never unsubscribed

```
public class MyService  
{
```

```

    public MyService(EventEmitter emitter)
    {
        emitter.OnEvent += HandleEvent; // subscribed but never
unsubscribed!
    }
}

// ✅ FIX – implement IDisposable
public class MyService : IDisposable
{
    private readonly EventEmitter _emitter;

    public MyService(EventEmitter emitter)
    {
        _emitter = emitter;
        _emitter.OnEvent += HandleEvent;
    }

    public void Dispose()
    {
        _emitter.OnEvent -= HandleEvent; // unsubscribe!
    }
}

// _____

// ❌ LEAK 3 – Unbounded cache growth
private static Dictionary<int, Product> _cache = new();

public void CacheProduct(Product p)
{
    _cache[p.Id] = p; // grows forever! never evicted
}

// ✅ FIX – use IMemoryCache with expiry

```

```
_cache.Set(p.Id, p, TimeSpan.FromMinutes(10)); // auto-evicts

// -----

// ❌ LEAK 4 – DbContext kept alive too long (Singleton instead
of Scoped)
builder.Services.AddSingleton<AppDbContext>(); // WRONG!
// DbContext tracks all entities forever = grows unbounded

// ✅ FIX – always Scoped
builder.Services.AddScoped<AppDbContext>(); // new per request,
disposed after
```

Health Checks — Know If Your App Is Alive

```
// Program.cs – add health checks
builder.Services.AddHealthChecks()
    .AddSqlServer(

builder.Configuration.GetConnectionString("DefaultConnection")!,
    name: "database",
    tags: new[] { "db", "sql" })
    .AddAzureKeyVault(
        new Uri(builder.Configuration["KeyVaultUrl"]!),
        new DefaultAzureCredential(),
        name: "keyvault",
        tags: new[] { "azure" })
    .AddCheck("self", () => HealthCheckResult.Healthy(), tags:
new[] { "api" });

// Map health endpoints
app.MapHealthChecks("/health", new HealthCheckOptions
```

```

{
    ResponseWriter = UIResponseWriter.WriteHealthCheckUIResponse
});

// Separate endpoint for load balancer (simple alive check)
app.MapHealthChecks("/health/live", new HealthCheckOptions
{
    Predicate = check => check.Tags.Contains("api")
});

// DB + dependencies check
app.MapHealthChecks("/health/ready", new HealthCheckOptions
{
    Predicate = _ => true // all checks
});

```

Response from `/health`:

```

{
  "status": "Healthy",
  "duration": "00:00:00.045",
  "entries": {
    "database": {
      "status": "Healthy",
      "duration": "00:00:00.032"
    },
    "keyvault": {
      "status": "Healthy",
      "duration": "00:00:00.011"
    },
    "self": {
      "status": "Healthy",
      "duration": "00:00:00.000"
    }
  }
}

```

```
}  
}
```

Security Best Practices Checklist

```
// Program.cs – complete security setup
```

```
// 1.  HTTPS always
```

```
app.UseHttpsRedirection();
```

```
app.UseHsts(); // HTTP Strict Transport Security
```

```
// 2.  Security headers
```

```
app.Use(async (context, next) =>
```

```
{
```

```
    context.Response.Headers.Add("X-Frame-Options", "DENY");
```

```
    context.Response.Headers.Add("X-Content-Type-Options",  
"nosniff");
```

```
    context.Response.Headers.Add("X-XSS-Protection", "1;  
mode=block");
```

```
    context.Response.Headers.Add("Referrer-Policy", "no-  
referrer");
```

```
    context.Response.Headers.Remove("Server"); // hide server  
info
```

```
    context.Response.Headers.Remove("X-Powered-By");
```

```
    await next();
```

```
});
```

```
// 3.  CORS – only allowed origins
```

```
app.UseCors("ProductionPolicy");
```

```
// 4.  Rate limiting
```

```
app.UseRateLimiter();
```



```
// 5. ✅ Authentication + Authorization
app.UseAuthentication();
app.UseAuthorization();

// 6. ✅ Input validation via Data Annotations (already done in
DTOs)

// 7. ✅ Sensitive data never in logs
// ❌ BAD
_logger.LogInformation("User {Email} with password {Password}",
email, password);
// ✅ GOOD
_logger.LogInformation("User {Email} logged in", email);

// 8. ✅ No stack traces in production responses (global
exception handler)

// 9. ✅ Secrets in Key Vault, never in code

// 10. ✅ SQL injection protection – EF Core handles this
automatically
```

Complete Production Checklist

Before going live, verify:

CODE QUALITY:

- ✅ No hardcoded secrets or connection strings
- ✅ All sensitive config in Key Vault
- ✅ Global exception handler in place
- ✅ Proper logging at correct levels

- ✓ Input validation **on** all endpoints
- ✓ DTOs – **not** exposing raw entities
- ✓ **Async** all the way
- ✓ No synchronous DB calls

SECURITY:

- ✓ HTTPS enforced
- ✓ Security headers added
- ✓ CORS configured **for** production URLs only
- ✓ Rate limiting configured
- ✓ JWT expiry **is short** (60 min **or** less)
- ✓ Passwords hashed **with** BCrypt
- ✓ No sensitive data **in** JWT payload

DATABASE:

- ✓ Connection retry logic configured
- ✓ Indexes **on** frequently queried columns
- ✓ Migrations applied
- ✓ Backup retention configured
- ✓ No raw SQL **string** concatenation

AZURE:

- ✓ App Service plan appropriate **for** load
- ✓ **Auto**-scaling configured
- ✓ Application Insights connected
- ✓ Alerts **set for** errors **and** slow responses
- ✓ Health checks configured
- ✓ Deployment slots configured
- ✓ Managed Identity **for** **Key** Vault

DEVOPS:

- ✓ CI pipeline runs tests **on** every push
- ✓ Failed tests block deployment
- ✓ Separate environments (dev/staging/prod)

- ✓ Approval gates before production
- ✓ Rollback strategy in place (swap slots)

Final Interview Questions — Production Level

Q1: How do you handle an unhandled exception in production?

Global exception middleware catches all unhandled exceptions, logs the full details with stack trace internally, returns a safe user-friendly error message with a trace ID to the client, and Application Insights alerts the team.

Q2: How do you debug a performance issue in production?

Check Application Insights for slow requests, look at dependency tracking for slow DB queries, check the execution plan in SQL Server, identify N+1 query issues, add caching or indexes as needed, then verify improvement in metrics.

Q3: What is the difference between Clean Architecture and layered architecture?

Clean Architecture has strict dependency rules — inner layers cannot reference outer layers. The Domain knows nothing about EF Core or ASP.NET. Standard layered architecture allows any layer to reference lower layers but has no strict isolation of business logic.

Q4: How would you scale an ASP.NET Core API?

Make it stateless — use JWT, no server-side sessions. Use distributed caching with Redis. Scale horizontally with multiple App Service instances and load balancing. Add database read replicas for read-heavy workloads. Use async everywhere to maximize throughput per instance.

Q5: What is the N+1 problem and how do you fix it?

Querying a list of N entities then making one additional query per entity to get related data — total $N+1$ database queries. Fix with EF Core `Include()` for eager loading, or `Select()` to project directly into DTOs joining in one query.

Q6: How do you ensure zero downtime deployment?

Use deployment slots in Azure App Service. Deploy new version to staging slot, run health checks and smoke tests, then swap staging and production slots — the swap is instant and users never experience downtime. Old version stays in staging for easy rollback.

✅ Complete Roadmap — 100% Done

- ✅ Stage 1 — C# Deep Fundamentals
 - Value/Reference types, Nullable, OOP (4 pillars), Interfaces, Abstract, Sealed, Delegates, Events, LINQ, Async/Await, Exception Handling
- ✅ Stage 2 — ASP.NET Core
 - Request lifecycle, MVC, REST, HTTP verbs, Status codes, DTOs, Validation, Middleware, Dependency Injection, Configuration
- ✅ Stage 3 — SQL Server
 - DDL, DML, SELECT, WHERE, JOINS, GROUP BY, HAVING, Indexes, Execution Plan, Stored Procedures, Transactions, Normalization, ACID
- ✅ Stage 4 — Entity Framework Core
 - DbContext, Code First, Migrations, CRUD, Eager/Lazy/Explicit Loading, Tracking vs NoTracking, Performance, Repository Pattern, Pagination

- ✓ Stage 5 – Authentication & Security
JWT structure, Access + Refresh tokens, OAuth 2.0, Microsoft Entra ID, BCrypt, CORS, Rate Limiting, Key Vault, Security headers, Common mistakes
- ✓ Stage 6 – Azure Cloud
Resource Groups, App Service, Azure SQL, Key Vault, Managed Identity, Storage Account, Azure Functions, Application Insights, Alerts
- ✓ Stage 7 – DevOps & CI/CD
GitFlow, Branching, Pull Requests, Commit conventions,
YAML pipelines, Build CI, Release CD, Environment promotion, Approval gates
- ✓ Stage 8 – Clean Architecture
Domain layer, Application layer, Infrastructure layer,
API layer, SOLID principles (all 5 explained)
- ✓ Stage 9 – System Design
Load Balancer, Caching (Memory + Redis), Vertical vs Horizontal scaling, Microservices vs Monolith,
API Gateway, Real architecture diagram
- ✓ Stage 10 – Production Level
Structured logging, Log levels, Global exception handling,
API versioning, Performance optimization, Memory leak detection, Health checks, Security checklist, Production deployment checklist

What To Do Now — Your Action Plan

- WEEK 1-2: Build a simple CRUD API **with** EF Core
→ Products, Categories, basic auth
- WEEK 3-4: Add JWT authentication
→ Register, Login, **Protected** endpoints
→ Role-based access
- WEEK 5-6: Deploy **to** Azure
→ App Service + Azure SQL + **Key** Vault
→ **Set** up Application Insights
- WEEK 7-8: Add CI/CD pipeline
→ Azure DevOps, YAML pipeline
→ **Auto** deploy **to** staging
- WEEK 9-10: Refactor **to** Clean Architecture
→ Separate Domain, Application, Infrastructure
→ Add caching, health checks
- WEEK 11-12: Build a complete project **with** all stages
→ This becomes your portfolio project
→ You can talk about EVERYTHING **in** interviews

You now have everything needed to work as a backend developer in the Microsoft ecosystem.

The knowledge is here. Execution is what separates you from everyone else. Start building.



21/02/2026, 10:18:24

👤 So gives as for react not much depth but proper cover all

23/02/2026, 13:19:49

Thought process: Architected accessible React guide balancing breadth with comprehensiveness.

The user wants a React guide - not too deep but proper coverage of all important topics. Let me create a comprehensive but accessible React guide covering all the essentials.

React — Complete Proper Guide

From Basics to Real Project Level

What Is React?

Before React — plain HTML/JS way:

User clicks button

- You manually find the element
- You manually **update** the DOM
- You manage everything yourself
- Gets messy fast **with** complex UI

React way:

You **define** WHAT the UI should look **like**

React figures **out** HOW **to update** the DOM

You manage STATE, React handles rendering

- Clean, organized, scalable

React is a **JavaScript library** for building user interfaces, made by Facebook/Meta.

Backend (ASP.NET Core) → provides data via API

Frontend (React) → displays data, handles user interaction

They communicate via HTTP (fetch/axios)

Setup — Create React Project

Modern way – Vite (faster than Create React App)

```
npm create vite@latest my-shop-app -- --template react
```

```
cd my-shop-app
```

```
npm install
```

```
npm run dev
```

Opens at: http://localhost:5173

Project structure:

```
my-shop-app/  
|  
├─ src/  
|   ├─ components/    ← reusable UI pieces  
|   ├─ pages/          ← full page components  
|   ├─ hooks/          ← custom hooks  
|   ├─ services/       ← API calls  
|   ├─ context/        ← global state  
|   ├─ App.jsx         ← root component  
|   └─ main.jsx        ← entry point  
|  
├─ public/             ← static files  
└─ index.html
```



```
|— package.json
|— vite.config.js
```

Lesson 1 — Components

What Is a Component?

"A reusable piece of UI. Like a LEGO brick — build small pieces, combine into big things."

```
Full Page = Header + ProductList + Footer
ProductList = ProductCard + ProductCard + ProductCard
ProductCard = Image + Title + Price + Button
```

Each piece = one component.

Functional Component — The Only Way Now

```
// src/components/ProductCard.jsx

// A component is just a JavaScript function that returns JSX
function ProductCard() {
  return (
    <div className="card">
      <h2>Laptop</h2>
      <p>$999.99</p>
      <button>Add to Cart</button>
    </div>
  )
}
```

```
    </div>
  );
}

export default ProductCard;
```

```
// src/App.jsx – use the component
import ProductCard from './components/ProductCard';

function App() {
  return (
    <div>
      <h1>My Shop</h1>
      <ProductCard />    {/* use like an HTML tag */}
      <ProductCard />    {/* reuse as many times as needed */}
      <ProductCard />
    </div>
  );
}

export default App;
```

JSX — What It Is

JSX = JavaScript + XML syntax. Looks like HTML but it's JavaScript.

```
// JSX rules you must know:

// 1. Return ONE root element (wrap in div or Fragment)
function Good() {
  return (
    <div>                {/* one root */}
```

```

    <h1>Title</h1>
    <p>Text</p>
  </div>
);
}

// Or use Fragment – renders no extra DOM element
function AlsoGood() {
  return (
    <>                                {/* Fragment shorthand */}
    <h1>Title</h1>
    <p>Text</p>
    </>
  );
}

// 2. className not class (class is reserved in JS)
<div className="my-class">

// 3. JavaScript inside {} curly braces
const name = "Ali";
<h1>Hello {name}</h1>                // Hello Ali
<p>{2 + 2}</p>                        // 4
<p>{isActive ? "Yes" : "No"}</p>    // conditional

// 4. Self-closing tags must close
           // ✓
<input type="text" />              // ✓
<br />                             // ✓

```



Lesson 2 — Props

What Are Props?

"Data passed FROM parent TO child component. Like function arguments for components."

```
// Child component – receives props
function ProductCard({ name, price, isActive }) {
  return (
    <div className={`card ${isActive ? 'active' : 'inactive'}`}>
      <h2>{name}</h2>
      <p>${price}</p>
      <span>{isActive ? '✅ In Stock' : '❌ Out of Stock'}</span>
    </div>
  );
}

// Parent – passes props
function App() {
  return (
    <div>
      <ProductCard name="Laptop" price={999} isActive={true} />
      <ProductCard name="Phone" price={599} isActive={true} />
      <ProductCard name="Monitor" price={399} isActive={false} />
    </div>
  );
}
```

Props With Objects and Arrays

```
// Pass objects as props
function ProductCard({ product }) {
```

```

return (
  <div>
    <h2>{product.name}</h2>
    <p>${product.price}</p>
    <p>Category: {product.category}</p>
  </div>
);
}

function App() {
  const laptop = {
    id: 1,
    name: "Laptop",
    price: 999,
    category: "Electronics"
  };

  return <ProductCard product={laptop} />;
}

```

Children Prop — Wrapping Components

```

// Card is a wrapper – content passed via children
function Card({ title, children }) {
  return (
    <div className="card">
      <h2>{title}</h2>
      <div className="content">
        {children}  {/* whatever you put between tags */}
      </div>
    </div>
  );
}

```

```
}

// Usage
function App() {
  return (
    <Card title="Product Details">
      <p>This laptop is great!</p>
      <button>Buy Now</button>
    </Card>
  );
}
```

Lesson 3 — State (useState)

What Is State?

"Data that belongs to a component and when it changes, the component re-renders automatically."

```
Props   = data FROM parent (read only, can't change)
State   = data OWNED by component (can change, triggers re-render)
```

useState — Basic Usage

```
import { useState } from 'react';

function Counter() {
```

```
// [currentValue, functionToChange] = useState(initialValue)
const [count, setCount] = useState(0);

return (
  <div>
    <p>Count: {count}</p>
    <button onClick={() => setCount(count + 1)}>+</button>
    <button onClick={() => setCount(count - 1)}>-</button>
    <button onClick={() => setCount(0)}>Reset</button>
  </div>
);
}
```

useState With Objects

```
import { useState } from 'react';

function ProductForm() {
  const [formData, setFormData] = useState({
    name: '',
    price: '',
    stock: ''
  });

  const handleChange = (e) => {
    // spread operator keeps other fields, updates only changed
    one
    setFormData({
      ...formData,
      [e.target.name]: e.target.value
    });
  };
}
```

```
const handleSubmit = (e) => {
  e.preventDefault(); // prevent page refresh
  console.log('Form submitted:', formData);
};

return (
  <form onSubmit={handleSubmit}>
    <input
      name="name"
      value={formData.name}
      onChange={handleChange}
      placeholder="Product name"
    />
    <input
      name="price"
      type="number"
      value={formData.price}
      onChange={handleChange}
      placeholder="Price"
    />
    <input
      name="stock"
      type="number"
      value={formData.stock}
      onChange={handleChange}
      placeholder="Stock"
    />
    <button type="submit">Create Product</button>
  </form>
);
}
```


useState With Arrays

```
import { useState } from 'react';

function ShoppingCart() {
  const [items, setItems] = useState([]);

  const addItem = (product) => {
    setItems([...items, product]); // spread + add new
  };

  const removeItem = (id) => {
    setItems(items.filter(item => item.id !== id)); // remove by
    id
  };

  const clearCart = () => {
    setItems([]); // empty array
  };

  const total = items.reduce((sum, item) => sum + item.price, 0);

  return (
    <div>
      <h2>Cart ({items.length} items)</h2>

      {items.length === 0
        ? <p>Cart is empty</p>
        : items.map(item => (
            <div key={item.id}>
              <span>{item.name} - ${item.price}</span>
              <button onClick={() =>
removeItem(item.id)}>Remove</button>
            </div>
          ))
      )
    )
  );
}
```

```
    }

    <p>Total: ${total}</p>
    <button onClick={clearCart}>Clear Cart</button>
  </div>
);
}
```

Lesson 4 — useEffect

What Is useEffect?

"Run side effects — code that happens AFTER render. Fetching data, subscriptions, timers."

```
import { useState, useEffect } from 'react';

// useEffect(function, dependencyArray)
//
// dependency array controls WHEN effect runs:
// []          → run once when component mounts (like
componentDidMount)
// [value]     → run when 'value' changes
// no array    → run after EVERY render (rare, usually wrong)
```

Fetch Data on Component Load

```

import { useState, useEffect } from 'react';

function ProductList() {
  const [products, setProducts] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    // Fetch products when component mounts
    const fetchProducts = async () => {
      try {
        setLoading(true);

        const response = await
fetch('https://api.yourapp.com/api/product');

        if (!response.ok) {
          throw new Error(`HTTP error! status:
${response.status}`);
        }

        const data = await response.json();
        setProducts(data);

      } catch (err) {
        setError(err.message);
      } finally {
        setLoading(false);
      }
    };

    fetchProducts();
  }, []); // [] = run once on mount

  if (loading) return <div>Loading products...</div>;

```

```
if (error)    return <div>Error: {error}</div>;

return (
  <div>
    <h1>Products</h1>
    {products.map(product => (
      <ProductCard
        key={product.id}
        product={product}
      />
    ))}
  </div>
);
}
```

useEffect With Dependency — Re-fetch When Value Changes

```
import { useState, useEffect } from 'react';

function ProductSearch() {
  const [query, setQuery]      = useState('');
  const [products, setProducts] = useState([]);
  const [loading, setLoading]  = useState(false);

  useEffect(() => {
    // Don't search if query is empty
    if (!query.trim()) {
      setProducts([]);
      return;
    }
  })
}
```

```

const fetchResults = async () => {
  setLoading(true);
  try {
    const response = await fetch(
      `https://api.yourapp.com/api/product/search?
name=${query}`
    );
    const data = await response.json();
    setProducts(data);
  } catch (err) {
    console.error(err);
  } finally {
    setLoading(false);
  }
};

// Debounce – wait 500ms after user stops typing
const timer = setTimeout(fetchResults, 500);

// Cleanup – cancel timer if query changes before 500ms
return () => clearTimeout(timer);

}, [query]); // runs every time 'query' changes

return (
  <div>
    <input
      value={query}
      onChange={e => setQuery(e.target.value)}
      placeholder="Search products..."
    />

    {loading && <p>Searching...</p>}

    {products.map(p => (

```

```
        <div key={p.id}>{p.name} – ${p.price}</div>
      )})
    </div>
  );
}
```

Lesson 5 — Event Handling

```
function EventExamples() {
  // Click
  const handleClick = () => {
    alert('Button clicked!');
  };

  // Click with data
  const handleProductClick = (productId) => {
    console.log('Product clicked:', productId);
  };

  // Input change
  const handleChange = (e) => {
    console.log('Value:', e.target.value);
  };

  // Form submit
  const handleSubmit = (e) => {
    e.preventDefault(); // MUST have this – prevents page refresh
    console.log('Form submitted');
  };

  // Keyboard
```

```

const handleKeyDown = (e) => {
  if (e.key === 'Enter') console.log('Enter pressed');
  if (e.key === 'Escape') console.log('Escape pressed');
};

return (
  <div>
    {/* Inline vs function reference */}
    <button onClick={handleClick}>Click me</button>

    {/* Pass argument – use arrow function wrapper */}
    <button onClick={() => handleProductClick(42)}>Product
42</button>

    <input onChange={handleChange} onKeyDown={handleKeyDown} />

    <form onSubmit={handleSubmit}>
      <input type="text" />
      <button type="submit">Submit</button>
    </form>
  </div>
);
}

```

Lesson 6 — Lists & Conditional Rendering

Rendering Lists

```
function ProductList({ products }) {
  return (
    <div>
      {products.map(product => (
        // key is REQUIRED – helps React track items
        // use unique stable ID, never array index if list can
change
        <div key={product.id}>
          <h3>{product.name}</h3>
          <p>${product.price}</p>
        </div>
      ))}
    </div>
  );
}
```

Conditional Rendering — 4 Ways

```
function ProductCard({ product }) {

  // Way 1 – if/else before return
  if (!product) {
    return <div>No product found</div>;
  }

  return (
    <div>
      <h2>{product.name}</h2>

      {/* Way 2 – ternary operator (if/else inline) */}
      {product.isActive
```



```

    ? <span className="badge-green">In Stock</span>
    : <span className="badge-red">Out of Stock</span>
  }

  {/* Way 3 – && operator (show only if true) */}
  {product.stock <= 5 && product.stock > 0 && (
    <span className="badge-yellow">Only {product.stock} left!
</span>
  )}

  {/* Way 4 – nullish coalescing */}
  <p>{product.description ?? 'No description available'}</p>
</div>
);
}

```

Lesson 7 — Custom Hooks

What Is a Custom Hook?

"Extract reusable logic from components into a function. Starts with 'use'."

```

// hooks/useFetch.js – reusable data fetching hook
import { useState, useEffect } from 'react';

function useFetch(url) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {

```

```

    if (!url) return;

    const fetchData = async () => {
      setLoading(true);
      setError(null);
      try {
        const response = await fetch(url);
        if (!response.ok) throw new Error(`Error:
${response.status}`);
        const result = await response.json();
        setData(result);
      } catch (err) {
        setError(err.message);
      } finally {
        setLoading(false);
      }
    };

    fetchData();
  }, [url]);

  return { data, loading, error };
}

export default useFetch;

```

```

// Now any component can use it cleanly
import useFetch from '../hooks/useFetch';

function ProductList() {
  const {
    data: products,
    loading,
    error
  } = useFetch('https://api.yourapp.com/api/product');

```

```
if (loading) return <div>Loading...</div>;
if (error)   return <div>Error: {error}</div>;

return (
  <div>
    {products?.map(p => (
      <div key={p.id}>{p.name}</div>
    ))}
  </div>
);
}
```

Another Real Custom Hook — useAuth

```
// hooks/useAuth.js
import { useState } from 'react';

function useAuth() {
  const [user, setUser]      = useState(null);
  const [loading, setLoading] = useState(false);
  const [error, setError]    = useState(null);

  const login = async (email, password) => {
    setLoading(true);
    setError(null);
    try {
      const response = await fetch('/api/auth/login', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ email, password })
      });
    }
  };
}
```

```
    if (!response.ok) throw new Error('Invalid credentials');

    const data = await response.json();

    // Store token
    localStorage.setItem('token', data.accessToken);
    setUser(data.user);

    return true;
  } catch (err) {
    setError(err.message);
    return false;
  } finally {
    setLoading(false);
  }
};

const logout = () => {
  localStorage.removeItem('token');
  setUser(null);
};

const isAuthenticated = () => !!localStorage.getItem('token');

return { user, loading, error, login, logout, isAuthenticated };
}

export default useAuth;
```

Lesson 8 — Context API (Global State)

What Is Context?

"Share data between components without passing props through every level."

The problem it solves:

Without Context – prop drilling:

App (has `user`)

└─ Layout (passes `user` down)

└─ Header (passes `user` down)

└─ UserMenu (finally uses `user`!)

Props passed through 3 levels just to reach UserMenu

With Context:

App provides `user` in Context

UserMenu reads from Context directly

No prop drilling needed

Create and Use Context

```
// context/AuthContext.jsx
import { createContext, useContext, useState } from 'react';

// 1. Create the context
const AuthContext = createContext(null);

// 2. Create provider component
```

```
export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const [token, setToken] = useState(
    localStorage.getItem('token') || null
  );

  const login = async (email, password) => {
    const response = await fetch('/api/auth/login', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ email, password })
    });

    const data = await response.json();

    localStorage.setItem('token', data.accessToken);
    setToken(data.accessToken);
    setUser(data.user);
  };

  const logout = () => {
    localStorage.removeItem('token');
    setToken(null);
    setUser(null);
  };

  // Everything available to all children
  const value = {
    user,
    token,
    login,
    logout,
    isAuthenticated: !!token
  };
};
```

```

    return (
      <AuthContext.Provider value={value}>
        {children}
      </AuthContext.Provider>
    );
  }

  // 3. Custom hook for easy use
  export function useAuthContext() {
    const context = useContext(AuthContext);
    if (!context) {
      throw new Error('useAuthContext must be used inside
AuthProvider');
    }
    return context;
  }
}

```

```

// main.jsx – wrap entire app with provider
import { AuthProvider } from '../context/AuthContext';

ReactDOM.createRoot(document.getElementById('root')).render(
  <AuthProvider>
    <App />
  </AuthProvider>
);

```

```

// Any component – use auth directly, no props needed
import { useAuthContext } from '../context/AuthContext';

function Header() {
  const { user, logout, isAuthenticated } = useAuthContext();

  return (
    <header>

```

```
<h1>Shop App</h1>
{isAuthenticated
  ? (
    <div>
      <span>Hello, {user?.fullName}</span>
      <button onClick={logout}>Logout</button>
    </div>
  )
  : <a href="/login">Login</a>
}
</header>
);
}
```

Lesson 9 — React Router (Navigation)

```
npm install react-router-dom
```

```
// App.jsx – setup routes
import { BrowserRouter, Routes, Route, Navigate } from 'react-router-dom';
import { useAuthContext } from '../context/AuthContext';

// Pages
import HomePage from './pages/HomePage';
import ProductsPage from './pages/ProductsPage';
import ProductDetail from './pages/ProductDetail';
import LoginPage from './pages/LoginPage';
import DashboardPage from './pages/DashboardPage';
```



```

import NotFoundPage    from './pages/NotFoundPage';

// Protected route wrapper
function ProtectedRoute({ children }) {
  const { isAuthenticated } = useAuthContext();

  if (!isAuthenticated) {
    return <Navigate to="/login" replace />;
  }

  return children;
}

function App() {
  return (
    <BrowserRouter>
      <Routes>
        { /* Public routes */}
        <Route path="/"                element={<HomePage />}
      />
        <Route path="/products"        element={<ProductsPage />}
      />
        <Route path="/products/:id"    element={<ProductDetail />}
      />
        <Route path="/login"           element={<LoginPage />}
      />

        { /* Protected routes – must be logged in */}
        <Route
          path="/dashboard"
          element={
            <ProtectedRoute>
              <DashboardPage />
            </ProtectedRoute>
          }
        />
      </Routes>
    </BrowserRouter>
  );
}

```

```

    />

    { /* 404 */ }
    <Route path="*" element={<NotFoundPage />} />
  </Routes>
</BrowserRouter>
);
}

```

```

// pages/ProductDetail.jsx – use URL params
import { useParams, useNavigate } from 'react-router-dom';
import { useState, useEffect } from 'react';

function ProductDetail() {
  const { id } = useParams();           // get :id from URL
  const navigate = useNavigate();       // programmatic
  navigation

  const [product, setProduct] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const fetchProduct = async () => {
      try {
        const res = await fetch(`/api/product/${id}`);
        if (!res.ok) throw new Error('Not found');
        const data = await res.json();
        setProduct(data);
      } catch {
        navigate('/products'); // redirect if not found
      } finally {
        setLoading(false);
      }
    };
  });
}

```

```

    fetchProduct();
  }, [id]);

  if (loading) return <div>Loading...</div>;

  return (
    <div>
      <button onClick={() => navigate(-1)}>← Back</button>
      <h1>{product?.name}</h1>
      <p>${product?.price}</p>
      <button onClick={() => navigate('/products')}>
        All Products
      </button>
    </div>
  );
}

```

```

// Navigation links
import { Link, NavLink } from 'react-router-dom';

function Navbar() {
  return (
    <nav>
      {/* Link – basic navigation */}
      <Link to="/">Home</Link>
      <Link to="/products">Products</Link>

      {/* NavLink – adds 'active' class automatically when on
that page */}
      <NavLink
        to="/products"
        className={({ isActive }) => isActive ? 'nav-active' :
''}
      >
        Products

```

```
    </NavLink>
  </nav>
);
}
```

Lesson 10 — API Service Layer

Never Write fetch() Directly in Components

```
// services/api.js – centralized API client

const BASE_URL = import.meta.env.VITE_API_URL ||
  'http://localhost:5000';

// Helper – add token to every request automatically
const getHeaders = () => {
  const token = localStorage.getItem('token');
  return {
    'Content-Type': 'application/json',
    ...(token && { 'Authorization': `Bearer ${token}` })
  };
};

// Helper – handle all responses uniformly
const handleResponse = async (response) => {
  if (response.status === 401) {
    localStorage.removeItem('token');
    window.location.href = '/login'; // force logout
    return;
  }
}
```

```

    if (!response.ok) {
      const error = await response.json().catch(() => ({}));
      throw new Error(error.message || `HTTP Error
${response.status}`);
    }

    if (response.status === 204) return null; // no content

    return response.json();
  };

  // Generic request function
  const request = async (method, endpoint, body = null) => {
    const options = {
      method,
      headers: getHeaders()
    };

    if (body) {
      options.body = JSON.stringify(body);
    }

    const response = await fetch(`${BASE_URL}${endpoint}`,
options);
    return handleResponse(response);
  };

  // — Product API —————
  export const productApi = {
    getAll: (params = {}) => {
      const query = new URLSearchParams(params).toString();
      return request('GET', `/api/product${query ? '?' + query :
''}`);
    },
  },

```

```

    getById: (id) =>
      request('GET', `/api/product/${id}`),

    create: (data) =>
      request('POST', '/api/product', data),

    update: (id, data) =>
      request('PUT', `/api/product/${id}`, data),

    delete: (id) =>
      request('DELETE', `/api/product/${id}`)
  };

// — Auth API —————
export const authApi = {
  login: (credentials) =>
    request('POST', '/api/auth/login', credentials),

  register: (data) =>
    request('POST', '/api/auth/register', data),

  logout: (refreshToken) =>
    request('POST', '/api/auth/logout', { refreshToken }),

  refreshToken: (refreshToken) =>
    request('POST', '/api/auth/refresh', { refreshToken })
};

```

```

// Now use cleanly in components
import { productApi } from '../services/api';

function ProductsPage() {
  const [products, setProducts] = useState([]);
  const [loading, setLoading] = useState(true);

```

```

useEffect(() => {
  const loadProducts = async () => {
    try {
      const data = await productApi.getAll({ page: 1, pageSize:
10 });
      setProducts(data);
    } catch (err) {
      console.error(err.message);
    } finally {
      setLoading(false);
    }
  };

  loadProducts();
}, []);

const handleDelete = async (id) => {
  if (!confirm('Delete this product?')) return;

  try {
    await productApi.delete(id);
    setProducts(products.filter(p => p.id !== id));
  } catch (err) {
    alert('Failed to delete: ' + err.message);
  }
};

if (loading) return <div>Loading...</div>;

return (
  <div>
    {products.map(product => (
      <div key={product.id}>
        <h3>{product.name}</h3>
        <button onClick={() =>

```

```
handleDelete(product.id)}>Delete</button>
    </div>
  )})
</div>
);
}
```

Lesson 11 — Complete Real Page Example

Full CRUD Page — Products Management

```
// pages/ProductsPage.jsx — complete working example
import { useState, useEffect } from 'react';
import { useNavigate }          from 'react-router-dom';
import { productApi }           from '../services/api';
import { useAuthContext }       from '../context/AuthContext';

function ProductsPage() {
  const navigate = useNavigate();
  const { user } = useAuthContext();

  const [products, setProducts] = useState([]);
  const [loading, setLoading]   = useState(true);
  const [error, setError]       = useState(null);
  const [search, setSearch]     = useState('');

  // Create form state
  const [showForm, setShowForm] = useState(false);
  const [formData, setFormData] = useState({
```



```

    name: '', price: '', stock: ''
  });
  const [saving, setSaving] = useState(false);

  // Load products
  useEffect(() => {
    loadProducts();
  }, []);

  const loadProducts = async () => {
    setLoading(true);
    setError(null);
    try {
      const data = await productApi.getAll();
      setProducts(data);
    } catch (err) {
      setError(err.message);
    } finally {
      setLoading(false);
    }
  };

  // Filter products by search term (client-side)
  const filteredProducts = products.filter(p =>
    p.name.toLowerCase().includes(search.toLowerCase())
  );

  // Handle form input
  const handleFormChange = (e) => {
    setFormData({ ...formData, [e.target.name]: e.target.value
  });
  };

  // Create product
  const handleCreate = async (e) => {

```

```

e.preventDefault();
setSaving(true);
try {
  const newProduct = await productApi.create({
    name: formData.name,
    price: parseFloat(formData.price),
    stock: parseInt(formData.stock)
  });
  setProducts([...products, newProduct]);
  setFormData({ name: '', price: '', stock: '' });
  setShowForm(false);
} catch (err) {
  alert('Failed to create: ' + err.message);
} finally {
  setSaving(false);
}
};

// Delete product
const handleDelete = async (id, name) => {
  if (!confirm(`Delete "${name}"?`)) return;

  try {
    await productApi.delete(id);
    setProducts(products.filter(p => p.id !== id));
  } catch (err) {
    alert('Failed to delete: ' + err.message);
  }
};

if (loading) return <div className="loading">Loading
products...</div>;
if (error) return <div className="error">Error: {error}
<button onClick={loadProducts}>Retry</button></div>;

```

```

return (
  <div className="products-page">

    {/* Header */}
    <div className="page-header">
      <h1>Products ({filteredProducts.length})</h1>
      {user?.role === 'Admin' && (
        <button
          className="btn-primary"
          onClick={() => setShowForm(!showForm)}
        >
          {showForm ? 'Cancel' : '+ Add Product'}
        </button>
      )}
    </div>

    {/* Create Form */}
    {showForm && (
      <form className="create-form" onSubmit={handleCreate}>
        <h2>New Product</h2>
        <input
          name="name"
          value={formData.name}
          onChange={handleFormChange}
          placeholder="Product name"
          required
        />
        <input
          name="price"
          type="number"
          step="0.01"
          value={formData.price}
          onChange={handleFormChange}
          placeholder="Price"
          required

```

```

    />
    <input
      name="stock"
      type="number"
      value={formData.stock}
      onChange={handleFormChange}
      placeholder="Stock quantity"
      required
    />
    <button type="submit" disabled={saving}>
      {saving ? 'Saving...' : 'Create Product'}
    </button>
  </form>
)}

{/* Search */}
<input
  className="search-input"
  value={search}
  onChange={e => setSearch(e.target.value)}
  placeholder="Search products..."
/>

{/* Product List */}
{filteredProducts.length === 0
  ? <p>No products found.</p>
  : (
    <div className="product-grid">
      {filteredProducts.map(product => (
        <div
          key={product.id}
          className={`product-card ${!product.isActive ?
'inactive' : ''}`}
        >
          <h3>{product.name}</h3>

```

```

    <p className="price">${product.price}</p>
    <p className="stock">Stock: {product.stock}</p>
    <span className={`badge ${
      product.stock === 0 ? 'badge-red'
      : product.stock <= 5 ? 'badge-yellow'
      : 'badge-green'
    }`}>
      {product.stock === 0 ? 'Out of Stock'
      : product.stock <= 5 ? 'Low Stock'
      : 'In Stock'}
    </span>

    <div className="card-actions">
      <button onClick={() =>
navigate(`/products/${product.id}`)}>
        View
      </button>
      {user?.role === 'Admin' && (
        <button
          className="btn-danger"
          onClick={() => handleDelete(product.id,
product.name)}
        >
          Delete
        </button>
      )}
    </div>
  </div>
    )}
  </div>
)
}
</div>
);
}

```

```
export default ProductsPage;
```

Lesson 12 — Other Important Hooks

useRef — Access DOM or Keep Value Without Re-render

```
import { useRef, useEffect } from 'react';

function SearchInput() {
  const inputRef = useRef(null);

  // Auto-focus input when page loads
  useEffect(() => {
    inputRef.current.focus();
  }, []);

  return (
    <input
      ref={inputRef}
      placeholder="Start typing to search..."
    />
  );
}
```

```
// useRef to keep value WITHOUT triggering re-render
function Timer() {
  const [count, setCount] = useState(0);
```

```

const intervalRef = useRef(null); // store interval
ID

const start = () => {
  intervalRef.current = setInterval(() => {
    setCount(c => c + 1);
  }, 1000);
};

const stop = () => {
  clearInterval(intervalRef.current);
};

return (
  <div>
    <p>Count: {count}</p>
    <button onClick={start}>Start</button>
    <button onClick={stop}>Stop</button>
  </div>
);
}

```

useMemo — Cache Expensive Calculations

```

import { useState, useMemo } from 'react';

function ProductStats({ products }) {
  const [filter, setFilter] = useState('all');

  // Without useMemo – recalculates on EVERY render
  // With useMemo – only recalculates when products changes
  const stats = useMemo(() => {

```

```

    console.log('Calculating stats...');
    return {
      total:      products.length,
      active:     products.filter(p => p.isActive).length,
      totalValue: products.reduce((sum, p) => sum + p.price *
p.stock, 0),
      avgPrice:   products.reduce((sum, p) => sum + p.price, 0)
/ products.length
    };
  }, [products]); // only recalculate when products array changes

  return (
    <div>
      <p>Total: {stats.total}</p>
      <p>Active: {stats.active}</p>
      <p>Total Value: ${stats.totalValue.toFixed(2)}</p>
      <p>Avg Price: ${stats.avgPrice.toFixed(2)}</p>
    </div>
  );
}

```

useCallback — Cache Functions

```

import { useState, useCallback } from 'react';

function ProductList() {
  const [products, setProducts] = useState([]);

  // Without useCallback – new function created on every render
  // With useCallback – same function reference unless deps
  change
  const handleDelete = useCallback(async (id) => {

```



```

    await productApi.delete(id);
    setProducts(prev => prev.filter(p => p.id !== id));
  }, []); // no dependencies – function never recreated

  return (
    <div>
      {products.map(p => (
        <ProductCard
          key={p.id}
          product={p}
          onDelete={handleDelete} // stable reference = no
unnecessary re-renders
        />
      ))}
    </div>
  );
}

```

Lesson 13 — Environment Variables & Config

```

# .env file (root of project)
VITE_API_URL=http://localhost:5000
VITE_APP_NAME=ShopApp

# .env.production
VITE_API_URL=https://api.yourapp.com
VITE_APP_NAME=ShopApp

# Rules:

```

```
# Must start with VITE_ to be exposed to browser
# Never put secrets here – they are visible in browser!
```

```
// Access in code
const apiUrl = import.meta.env.VITE_API_URL;
const appName = import.meta.env.VITE_APP_NAME;
const isDev = import.meta.env.DEV; // true in development
const isProd = import.meta.env.PROD; // true in production
```

Lesson 14 — Project Folder Structure (Real)

```
src/
|
├── assets/                ← images, fonts, icons
|
├── components/           ← reusable small components
|   ├── common/
|   |   ├── Button.jsx
|   |   ├── Input.jsx
|   |   ├── Modal.jsx
|   |   ├── Spinner.jsx
|   |   └── Badge.jsx
|   ├── layout/
|   |   ├── Header.jsx
|   |   ├── Footer.jsx
|   |   ├── Sidebar.jsx
|   |   └── Layout.jsx
|   └── products/
|       └── ProductCard.jsx
```

```
|      |─ ProductForm.jsx
|      |─ ProductGrid.jsx
|
|─ pages/                                ← full page components (one per
route)
|  |─ HomePage.jsx
|  |─ ProductsPage.jsx
|  |─ ProductDetailPage.jsx
|  |─ LoginPage.jsx
|  |─ DashboardPage.jsx
|  |─ NotFoundPage.jsx
|
|─ hooks/                                ← custom hooks
|  |─ useFetch.js
|  |─ useAuth.js
|  |─ useLocalStorage.js
|
|─ context/                              ← global state
|  |─ AuthContext.jsx
|  |─ CartContext.jsx
|
|─ services/                             ← API calls
|  |─ api.js                             ← base API client
|  |─ productService.js
|  |─ authService.js
|
|─ utils/                                ← helper functions
|  |─ formatters.js                      ← format dates, currency
|  |─ validators.js                     ← validation helpers
|
|─ App.jsx                               ← routes
|─ main.jsx                             ← entry point
```

Interview Questions

Q1: What is the difference between state and props?

Props are read-only data passed from parent to child. State is data owned and managed by the component itself. When state changes, the component re-renders automatically.

Q2: What is the virtual DOM?

React keeps a lightweight copy of the real DOM in memory. When state changes, React updates the virtual DOM first, compares it with the previous version (diffing), then only updates the actual DOM where changes occurred. This is faster than updating the real DOM directly.

Q3: When does useEffect run?

With empty array `[]` — once when component mounts. With dependencies `[value]` — when value changes. With no array — after every render. The cleanup function runs before next effect and on unmount.

Q4: What is the key prop and why is it important?

Key helps React identify which items in a list changed, were added, or removed. Without unique stable keys, React may re-render wrong elements or lose component state in lists.

Q5: What is Context API and when to use it?

Context provides a way to share data globally without passing props through every level. Use for data many components need — user auth, theme, language. Don't overuse — it re-renders all consumers when context changes.

Q6: What is the difference between useMemo and useCallback?

useMemo caches a computed value. useCallback caches a function. Both re-compute only when dependencies change. Use to prevent unnecessary re-renders of child components that receive these as props.

✅ React Complete — All Covered

- ✅ What React **is and** why it **exists**
- ✅ Components – functional, JSX rules
- ✅ Props – passing data, children prop
- ✅ useState – primitives, objects, arrays
- ✅ useEffect – mount, dependencies, cleanup
- ✅ Event handling – click, change, submit
- ✅ Lists – map, key prop
- ✅ Conditional rendering – ternary, &&, nullish
- ✅ Custom Hooks – useFetch, useAuth
- ✅ Context API – **global** state, provider **pattern**
- ✅ React Router – routes, params, navigation, protected routes
- ✅ API Service Layer – centralized **fetch**, auth headers
- ✅ Complete CRUD page – **real** working example
- ✅ useRef – DOM access, persistent **values**
- ✅ useMemo – cached calculations
- ✅ useCallback – cached functions
- ✅ Environment variables
- ✅ **Real** project folder structure

What to Build Next

Project **1** – Product **Catalog** (beginner):

- Fetch products from your ASP.NET API
- Display in grid with search/filter
- Product detail page

Project **2** – Shop **App** (intermediate):

- Auth (login/register)
- Protected routes
- Add to `cart` (Context)
- Admin CRUD panel

Project `3` – Full `Stack` (advanced):

- Your ASP.NET Core `API` (stages `1-6`)
- Azure `deployment` (stage `6`)
- React frontend
- CI/CD `pipeline` (stage `7`)
- This becomes your portfolio project

You now have the **complete full-stack Microsoft ecosystem roadmap** — from C# basics all the way to React frontend and Azure deployment. 🚀

23/02/2026, 13:22:07

 Yes cover theory in good way proepr cover all topic past with one proper guide well structured and proper simple format

23/02/2026, 13:26:29

Thought process: Architected comprehensive theory guide spanning all prior topics systematically.

The user wants a comprehensive theory guide covering all the topics we've discussed throughout this conversation. They want it well-structured, proper, simple format covering everything from Stage 1 to React. Let me create a complete theory reference guide.



Complete Theory Guide

Microsoft Backend + React — Everything In One Place



PART 1 — C# FUNDAMENTALS

1.1 Value Types vs Reference Types

Value Types store actual data directly in Stack memory. Assignment creates independent copy.

Examples: `int`, `double`, `float`, `bool`, `char`, `struct`, `enum`
Default: `0`, `false`, `'\0'`

Location: Stack

Copy: Independent – changing one doesn't *affect other*

Reference Types store memory address in Stack, actual object in Heap. Assignment copies the address, both variables point to same object.

Examples: `class`, `string`, `array`, `List<T>`, `object`

Default: `null`

Location: Heap (address `in` Stack)

Copy: `Shared` – changing one affects other

Stack vs Heap:

Stack

Heap

Fast access

Slower access

Small size

`Large` size

Auto managed (LIFO)

Managed `by` Garbage Collector

Stores `value` types

Stores reference type objects

Stores `references`

`Method` variables live here

Objects live here

String is special: Reference type but behaves like value type because it is immutable. Every "change" creates a new string object.

1.2 Nullable Types

By default, value types cannot be null. Nullable types allow value types to hold null — common when reading from databases.


```
int? = Nullable<int> → can hold int OR null
bool? = Nullable<bool> → can hold true, false, OR null
```

Key members:

- `.HasValue` → check if it has a real value
- `.Value` → get the value (throws if null — always check HasValue first)
- `.GetValueOrDefault()` → returns value or 0/false/default
- `.GetValueOrDefault(10)` → returns value or your specified default

Key operators:

- `??` → null coalescing: return right side if left is null
- `??=` → null coalescing assignment: assign if currently null
- `?.` → null conditional: access member only if not null, otherwise return null

1.3 OOP — Four Pillars

Encapsulation

Hide internal data. Expose only controlled access through properties and methods. The class protects its own state.

```
Private fields → hidden data
Public properties → controlled access with validation
Public methods → controlled behavior
```

Access modifiers:

- `public` — everyone can access
- `private` — only same class

- `protected` — same class + child classes
- `internal` — same project only

Abstraction

Hide HOW something works. Show only WHAT it does. User interacts with simple interface without knowing complex internals.

You press car accelerator — you don't *know how fuel injection works*

You `call` `ProcessPayment()` — you don't *know if it's Stripe or PayPal*

Achieved through: abstract classes and interfaces.

Inheritance

Child class inherits properties and methods from parent class. Avoids code duplication.

```
Employee (parent)           → has Name, BaseSalary,
CalculateSalary()
Manager : Employee (child) → inherits everything + adds Bonus
Intern  : Employee (child) → inherits everything + overrides
salary calc
```

Rules:

- C# allows only single inheritance (one parent class)
- Use `: ParentClass` to inherit
- Use `base()` to call parent constructor
- Child can override `virtual` methods using `override`

Polymorphism

Same method call behaves differently depending on actual object type.

Compile-time: Method Overloading

Same name, different parameters

Resolved at compile time

Runtime: Method Overriding

`virtual` in parent + `override` in child

Resolved at runtime based on actual object type

1.4 Interfaces vs Abstract Classes

Interface:

- Pure contract — defines WHAT, not HOW
- No fields, no constructor
- All members public
- Class can implement multiple interfaces
- Use when: defining a contract, multiple inheritance needed

Abstract Class:

- Can have fields, constructors, implemented methods
- Can have access modifiers
- Class can inherit only ONE abstract class
- Use when: sharing common code/state among related classes

Feature	Abstract Class	Interface
Fields	✓ Yes	✗ No

Constructor	✓ Yes	✗ No
Implementation	✓ Yes	⚠ C# 8+ only
Multiple inheritance	✗ No	✓ Yes
Access modifiers	✓ Yes	Public only

1.5 Important C# Concepts

Delegates

Type-safe function pointer. A variable that holds a reference to a method. Enables passing methods as parameters.

```
Func<int, int, int> → takes two ints, returns int
Action<string>      → takes string, returns void
Predicate<int>      → takes int, returns bool
```

Events

Built on delegates. Allows publisher-subscriber pattern. A class notifies other classes when something happens. Multiple subscribers can react to one event.

LINQ

Language Integrated Query. Query collections using C# instead of loops.

```
Key methods:
Where()          → filter
Select()         → transform/project
OrderBy()        → sort ascending
OrderByDescending → sort descending
GroupBy()        → group elements
```

<code>Include()</code>	→ load related data (EF Core)
<code>FirstOrDefault()</code>	→ first match or null
<code>Any()</code>	→ check if any match
<code>All()</code>	→ check if all match
<code>Count()</code>	→ count elements
<code>Sum()</code>	→ add values
<code>Average()</code>	→ calculate average
<code>Min()</code> / <code>Max()</code>	→ find extremes
<code>Skip()</code> / <code>Take()</code>	→ pagination

Async / Await

Run operations without blocking the main thread. Essential for APIs, DB calls, HTTP requests.

<code>async</code>	→ marks method as asynchronous
<code>await</code>	→ pause here, free the thread, resume when done
<code>Task</code>	→ represents asynchronous operation (void return)
<code>Task<T></code>	→ represents <code>async</code> operation that returns T

<code>await Task.WhenAll(t1, t2, t3)</code>	→ wait for ALL to finish
<code>await Task.WhenAny(t1, t2)</code>	→ wait for FIRST to finish

Exception Handling

<code>try</code>	→ code that might throw
<code>catch(Type e)</code>	→ handle specific exception
<code>finally</code>	→ ALWAYS runs (cleanup code)
<code>throw</code>	→ manually throw exception
<code>when</code>	→ conditional catch filter

Custom exceptions: inherit from Exception class

Global handling: middleware catches all unhandled exceptions

Garbage Collector

Automatically frees memory in Heap that is no longer referenced. You don't manually manage memory in C#. Implement `IDisposable` for objects that hold unmanaged resources (files, DB connections). Use `using` statement for automatic disposal.



PART 2 — ASP.NET CORE

2.1 What Is ASP.NET Core

Microsoft's cross-platform framework for building web APIs, web apps, and microservices. Runs on Windows, Linux, and macOS.

2.2 Request Lifecycle

Every HTTP request follows this exact path:

HTTP Request

- Kestrel Server (receives raw HTTP)
- Middleware Pipeline (runs in order)
 - Exception Handler
 - HTTPS Redirection
 - Authentication
 - Authorization
 - Routing (which controller/action?)
- Controller Action (your code)
 - Service Layer
 - Repository

→ Database
→ Response built
→ Middleware runs in REVERSE
HTTP Response sent back

2.3 MVC Pattern

Model → data shape (C# class)
View → what **user** sees (HTML in MVC, JSON in Web API)
Controller → handles request, calls services, **returns** response

Web API = Controller + Model + JSON
(no HTML view – frontend handles that)

2.4 REST Principles

1. Client-Server → frontend and backend are separate
2. Stateless → each request is independent
3. Uniform Interface → standard URLs and HTTP verbs
4. Resource-based → URLs are nouns, verbs are HTTP methods

URL Design:

GET	/api/products	→ get all
GET	/api/products/ 1	→ get one
POST	/api/products	→ create new
PUT	/api/products/ 1	→ full update

PATCH /api/products/1 → partial update
DELETE /api/products/1 → delete

2.5 HTTP Verbs

GET → read data, no body, idempotent (same result every call)
POST → create data, has body, not idempotent
PUT → full replace, has body, idempotent
PATCH → partial update, has body
DELETE → remove data, no body, idempotent

2.6 HTTP Status Codes

2xx Success:

200 OK → success, data returned
201 Created → new resource created
204 No Content → success, nothing to return

4xx Client Error:

400 Bad Request → invalid data sent
401 Unauthorized → not authenticated
403 Forbidden → authenticated but no permission
404 Not Found → resource doesn't exist
409 Conflict → duplicate or conflict
422 Unprocessable → validation failed
429 Too Many Requests → rate limit hit

5xx Server Error:

500 Internal Error → something crashed in our code
503 Unavailable → server down or overloaded

2.7 Middleware

Code that runs on every request before it reaches your controller, and on every response before it leaves.

Key facts:

- Runs in order they are registered
- Each middleware can pass to next or short-circuit
- Bidirectional – runs on request AND response
- ORDER MATTERS – authentication before authorization

Common middleware:

- ExceptionHandler → global error handling (register FIRST)
- UseHttpsRedirection → force HTTPS
- UseAuthentication → who are you?
- UseAuthorization → what can you do?
- UseCors → allow cross-origin requests
- UseRateLimiter → limit requests per IP/user
- MapControllers → connect routes to controllers

2.8 Dependency Injection

Provide dependencies from outside instead of creating them inside a class. Enables loose coupling, testability, and easy swapping of implementations.

Service Lifetimes:

Transient → new instance every time requested
Use for: lightweight, stateless services

Scoped → new instance per HTTP request
Same instance shared within one request
Use for: DbContext, repositories, services

Singleton → one instance for entire app lifetime
Use for: caching, configuration, shared state

Registration:

```
services.AddTransient<IEmailValidator, EmailValidator>();  
services.AddScoped<IProductService, ProductService>();  
services.AddSingleton<ICacheService, CacheService>();
```

2.9 DTO Pattern

Data Transfer Object — separate class for what comes IN and goes OUT of API.

Why:

- Control exactly what data is exposed
- Never accidentally expose PasswordHash, Salt, etc.
- Client can't set system fields like CreatedAt, IsAdmin
- Input shape can differ from output shape

Types:

CreateDto → what client sends when creating
UpdateDto → what client sends when updating
ResponseDto → what server returns to client

2.10 Configuration

appsettings.json	→ base config, committed to git
appsettings.Development	→ dev overrides (committed to git)
appsettings.Production	→ production overrides (NOT committed)
Environment Variables	→ override any config at runtime
Azure Key Vault	→ secrets for production (most secure)

Hierarchy: Key Vault > Environment Variables > appsettings
Last one wins on conflicts



PART 3 — SQL SERVER

3.1 Core Concepts

Database → container for tables **Table** → stores data in rows and columns **Primary Key** → unique identifier for each row **Foreign Key** → column that references another table's PK **Index** → speeds up data retrieval **Stored Procedure** → saved SQL script **Transaction** → group of operations that succeed or fail together **View** → saved query that acts like a table

3.2 SQL Command Types

DDL (Data Definition Language) — structure:
CREATE TABLE, ALTER TABLE, DROP TABLE

DML (Data Manipulation Language) – data:

SELECT, INSERT, UPDATE, DELETE

DCL (Data Control Language) – permissions:

GRANT, REVOKE

3.3 JOINS Explained

INNER JOIN → only rows that match in BOTH tables

Most common – use when you only want records with related data

LEFT JOIN → ALL rows from LEFT table + matching from right

Right side shows NULL if no match

Use when you want all records even without related data

RIGHT JOIN → ALL rows from RIGHT table + matching from left

Left shows NULL if no match

Rarely used – just flip to LEFT JOIN instead

FULL JOIN → ALL rows from both tables

NULL on either side where no match

3.4 Normalization

Organizing tables to eliminate data duplication and inconsistency.

1NF (First Normal Form):

- Each cell has ONE value (no comma-separated lists)
- Each row is unique (has primary key)

2NF (Second Normal Form):

- Satisfies 1NF
- Every non-key column depends on the WHOLE primary key

3NF (Third Normal Form):

- Satisfies 2NF
- No column depends on another non-key column
- Don't store CategoryName if you store CategoryId
(CategoryName depends on CategoryId, not on the row's PK)

Practical rule: Follow 3NF for most tables

3.5 ACID Properties

A – Atomicity

"All or nothing"

All operations in a transaction succeed, or NONE are saved.

Bank transfer: debit + credit both succeed, or both rollback.

C – Consistency

"Data always stays valid"

Transaction brings DB from one valid state to another.

Stock can't go below 0. Balance can't go negative.

I – Isolation

"Transactions don't interfere"

Concurrent transactions don't see each other's incomplete

changes.

Two users buying **last** item – **only one** succeeds.

D – Durability

"Committed data is permanent"

After **COMMIT**, data survives crashes **and** restarts.

Confirmed **order** stays even after server crash.

3.6 Indexes

Without index: **Table Scan** → **reads every row** (slow for large tables)

With index: **Index Seek** → jumps directly **to** matching **rows** (fast)

When to add index:

- Columns **in WHERE** clauses frequently
- Columns **in JOIN** conditions
- Columns **in ORDER BY** frequently
- **Foreign** key columns

When NOT to add:

- Small tables (**full scan is** fine)
- Columns rarely searched
- Tables **with** very frequent writes (indexes slow down inserts/updates)

Types:

Clustered → determines physical **row order** (**one per table**)

Non-clustered → separate structure, pointer **to row** (**many per table**)

Unique → enforces uniqueness + faster lookup
Composite → index **on** multiple columns together

3.7 Transactions

BEGIN TRANSACTION → **start** recording changes
COMMIT → save **all** changes permanently
ROLLBACK → undo **all** changes back **to BEGIN**

TRY/CATCH **in SQL**:

```
BEGIN TRANSACTION  
BEGIN TRY  
    -- operations  
    COMMIT  
END TRY  
BEGIN CATCH  
    ROLLBACK  
END CATCH
```



PART 4 — ENTITY FRAMEWORK CORE

4.1 What Is EF Core

Microsoft's ORM (Object-Relational Mapper). Maps C# classes to database tables. Translates LINQ to SQL automatically. Eliminates manual SQL writing.

```
C# Class    ↔ Database Table
Property   ↔ Column
LINQ query ↔ SQL query (auto-generated)
SaveChanges ↔ INSERT / UPDATE / DELETE
```

4.2 DbContext

The bridge between your app and database. Manages:

- Database connections
- Change tracking (watches what changed)
- Query translation (LINQ → SQL)
- Transaction management

```
One DbContext per request (Scoped lifetime)
DbSet<T> = represents a table
SaveChangesAsync() = executes all pending changes in one
transaction
```

4.3 Code First vs Database First

```
Code First (modern, recommended):
→ Write C# entity classes
→ EF Core generates database from them
→ Migrations track and apply changes
```


Database **First** (legacy):

- Design database tables **first**
- EF Core generates C# classes **from** DB
- Used **when** working **with** existing databases

4.4 Migrations

Version control for database schema.

Each migration **contains**:

Up() → how **to** apply this change (**ADD column, CREATE table**)

Down() → how **to** reverse this change (**DROP column, DROP table**)

Commands:

- | | |
|---------------------------------------|--|
| dotnet ef migrations add Name | → create migration |
| dotnet ef database update | → apply all pending migrations |
| dotnet ef migrations list | → see all migrations |
| dotnet ef database update 0 | → revert ALL migrations |
| dotnet ef migrations remove migration | → remove last unapplied migration |

4.5 Loading Strategies

Eager Loading (.Include()):

- Loads related data **in** SAME query **using** JOIN
- One database trip
- Use this most **of** the time 

→ Example: `.Include(p => p.Category)`

Explicit Loading:

- Load related data separately **on** demand
- Extra database trip
- Use **when** you conditionally need related data

Lazy Loading (virtual properties):

- **Auto**-loads related data **when** you access the **property**
- Causes N+1 query problem **if** used **with** lists
- Use **with** caution ⚠

4.6 N+1 Problem

Loading N entities then making one extra query per entity for related data = N+1 total queries. Terrible for performance.

Bad (N+1 queries):

Get 100 products → 1 query
Get category **for each** product → 100 more queries
Total: 101 queries

Good (1 query):

Get 100 products **WITH** categories → 1 **JOIN** query
`.Include(p => p.Category)`

4.7 Tracking vs No-Tracking

Tracking (**default**):

- EF Core watches objects **for** changes
- SaveChanges() knows what **to** update automatically
- Uses more memory
- Use **when** you will modify data

AsNoTracking():

- EF Core *doesn't watch objects*
- Faster, uses less memory
- Use **for** read-only **GET** endpoints ✅
- Cannot use SaveChanges() without manually marking state

4.8 Repository Pattern

Abstracts data access logic behind an interface. Controller/Service doesn't know about EF Core. Easy to swap implementation or mock for testing.

Interface (Domain layer) → defines contract
Implementation (Infrastructure) → uses EF Core
Service uses **interface** → *doesn't know about implementation*
DI container wires them together

PART 5 — AUTHENTICATION & SECURITY

5.1 Authentication vs Authorization

Authentication = WHO ARE YOU?

- Prove your identity
- Login with email + password
- Returns a token

Authorization = WHAT CAN YOU DO?

- What are you allowed to access?
- Check roles and permissions
- [Authorize(Roles = "Admin")]

Authentication MUST happen before Authorization

5.2 JWT (JSON Web Token)

Self-contained token that proves identity. Sent with every API request in Authorization header.

Structure — 3 parts separated by dots:

HEADER.PAYLOAD.SIGNATURE

Header: algorithm used to sign (HS256)

Payload: claims — user data (id, email, role, expiry)

Signature: proves token wasn't tampered with

Important:

- Payload is BASE64 ENCODED, not encrypted
- Anyone can decode and read claims
- NEVER put passwords or secrets in payload
- Signature is cryptographic — only your server can create/verify

JWT Flow:

1. Login → server creates token → returns to client
2. Client stores token
3. Every request → sends token in header: "Bearer eyJhbG..."
4. Server validates signature → reads claims → allows/denies
5. Token expires → client uses refresh token to get new one

5.3 Access Token vs Refresh Token

Access Token:

- Short-lived (15-60 minutes)
- Used for every API request
- Stored in memory (not localStorage – XSS risk)
- If stolen → expires quickly → low risk

Refresh Token:

- Long-lived (7-30 days)
- Used ONLY to get new access token
- Stored in HttpOnly cookie or secure storage
- Saved in database → can be revoked (logout)
- Rotating → new refresh token issued each time it's used

5.4 OAuth 2.0

Standard protocol for granting third-party apps limited access to user's account without sharing password.

"Login with Google" / "Login with Microsoft" = OAuth 2.0

Key roles:

Resource Owner	→ the user (you)
Client	→ the app requesting access
Authorization Server	→ Google/Microsoft handles login
Resource Server	→ API that holds user's <i>data</i>
Access Token	→ what app gets to access data
Scope	→ what access is requested (email, profile)

Flow:

1. User clicks "Login with Google"
2. Redirected **to** Google's *login page*
3. User logs **in** + consents **to** permissions
4. Google redirects back **with** authorization CODE
5. App exchanges CODE **for** ACCESS TOKEN
6. App uses token **to** access Google APIs
7. User never gave their Google password **to** your app

5.5 Microsoft Entra ID (formerly Azure AD)

Microsoft's cloud identity service for enterprise applications.

Key concepts:

Tenant	→ your organization's <i>identity directory</i>
App Registration	→ register your API with Entra ID (gets Client ID)
Client ID	→ unique identifier for your registered app
Client Secret	→ your app's <i>password</i> (store in Key Vault)
Tenant ID	→ your organization's <i>unique ID</i>
Scope	→ permissions your app requests

RBAC

→ Role-Based Access Control

When to use:

- Internal company apps
- Microsoft ecosystem / outsourcing projects
- Need SSO (Single Sign-On) with Microsoft accounts
- Enterprise projects where employees use Microsoft 365

Benefit:

- No password management in your app
- Microsoft handles all authentication complexity
- Built-in MFA, conditional access, compliance

5.6 Password Security

Never store plain text passwords – EVER

Hashing vs Encryption:

Encryption = reversible (if key stolen → all passwords exposed)

Hashing = one-way (can't reverse, even if DB stolen)

BCrypt:

- Industry standard for password hashing
- Automatically adds unique salt per password
- Same password → different hash every time
- Adjustable cost factor (slower = harder to crack)
- BCrypt.HashPassword("password") → "\$2a\$11\$..."
- BCrypt.Verify("password", hash) → true/false

5.7 Common Security Principles

Never expose stack traces **to** clients **in** production
Always **return** same **error for** wrong email vs wrong password
Short-lived access tokens + refresh token strategy
Store secrets **in** Azure **Key** Vault, never **in** code
HTTPS always – never send tokens over HTTP
CORS – only allow your actual frontend domains
Rate limiting – prevent brute force **on** login endpoints
SQL injection – use parameterized queries (EF Core **handles** this)
XSS – validate/sanitize all inputs
CSRF – use SameSite cookies **or** CSRF tokens



PART 6 — AZURE CLOUD

6.1 Cloud Computing Benefits

Old way (**on**-premise):

- Buy **and** maintain physical servers
- Expensive upfront cost
- Takes weeks **to get new** capacity
- Pay even **when** idle

Cloud way:

- Rent what you need, pay **per** use
- Scale **in** minutes (click a button)
- Microsoft manages hardware

- Global data centers for low latency
- Built-in security, backups, monitoring

6.2 Azure Organization Hierarchy

Azure Account

- └ Subscription (billing unit)
 - └ Resource Group (logical container for one project)
 - └ App Service
 - └ Azure SQL
 - └ Key Vault
 - └ Storage Account
 - └ Application Insights

6.3 Resource Group

Logical container for all related Azure resources. Delete the group → deletes everything inside. Use for: cost tracking, access control, organized deployment.

Best practice:

```
rg-projectname-development
rg-projectname-staging
rg-projectname-production
```

6.4 App Service

Fully managed web hosting. Deploy code → Microsoft manages the rest (server, OS, runtime, patching, SSL).

App **Service Plan** = the underlying **VM** (size/power)

Web App = your application running on that plan

Features:

- Auto-scaling (**scale out** = add instances)
- Deployment **slots** (staging → production swap, zero downtime)
- Built-in SSL
- Custom domains
- Environment variables
- Managed Identity

Scale Up = bigger **machine** (more CPU/RAM)

Scale Out = more **instances** (copies of your app) ← preferred

6.5 Deployment Slots — Zero Downtime

Production Slot = live version users see

Staging Slot = **new version** being tested

Workflow:

1. Deploy **new version** to Staging
 2. Test thoroughly
 3. Swap slots → Staging becomes Production instantly
 4. Old Production becomes **Staging** (easy rollback)
- Users see zero downtime

6.6 Azure SQL Database

Fully managed SQL Server in the cloud. Same SQL Server you know — Microsoft runs it.

Automatic:

- Backups (every 5-10 minutes for transaction logs)
- High availability (99.99% SLA)
- Security patches
- Point-in-time restore (any moment in last 7-35 days)

EF Core best practice:

- Enable retry on failure (transient connection issues)
- `sqlOptions.EnableRetryOnFailure(5, TimeSpan.FromSeconds(30), null)`

6.7 Azure Key Vault

Secure vault for secrets, API keys, connection strings, certificates.

Why:

- Secrets never in code or config files (*can't go to git*)
- Centralized secret management
- Audit log of who accessed what
- Automatic rotation possible
- Access controlled by Azure RBAC

Managed Identity:

- App Service gets automatic identity
- Key Vault trusts this identity
- No username/password needed anywhere
- Most secure approach — ZERO credentials in code

6.8 Azure Storage

Blob Storage → files, images, videos, backups (most common)
Table Storage → NoSQL key-value (cheap)
Queue Storage → messages between services
File Storage → SMB file [share](#) (network drive in cloud)

Blob containers:

Public → files accessible via [URL](#) (images, [public](#) assets)
Private → [requires authentication](#) (documents, invoices)

6.9 Azure Functions

Serverless compute — code runs only when triggered. Pay per execution. First 1 million executions free per month.

Triggers:

HTTP [Trigger](#) → [called](#) via HTTP endpoint
Timer [Trigger](#) → cron schedule ([every](#) night [at](#) [2](#) AM)
[Blob](#) [Trigger](#) → [when](#) file uploaded [to](#) storage
Queue [Trigger](#) → [when](#) message added [to](#) queue
Service Bus → enterprise messaging

Perfect [for](#):

- Background jobs (cleanup, reports)
- Scheduled tasks (nightly processing)
- Event-driven processing
- Lightweight endpoints that rarely run

6.10 Application Insights

Azure's monitoring service. Tracks everything in your running application automatically.

Automatically tracks:

- Every HTTP request (URL, status, duration)
- All exceptions with stack trace
- Database query performance
- External HTTP call performance
- Server metrics (CPU, memory)

You can also add:

- Custom events (business events like "OrderPlaced")
- Custom metrics (how many products returned)
- Custom traces (specific log messages)

Use KQL (Kusto Query Language) to query logs:

```
exceptions | where timestamp > ago(24h) | summarize count() by  
type  
requests | where duration > 3000 | order by duration desc
```



PART 7 — DEVOPS & CI/CD

7.1 What Is DevOps

Development + Operations working together with automation. Goal: reduce time from code to production, increase reliability, eliminate manual processes.

Without DevOps:

Code → manual test → manual zip → email to ops → manual deploy → hope for the best

With DevOps:

Code pushed → pipeline runs → tests → build → auto deploy → monitoring alerts if broken

7.2 GitFlow Branching Strategy

Permanent branches:

main → production code, always stable, never push directly
develop → integration, features merge here first

Temporary branches:

feature/xxx → new feature (branches from develop)
bugfix/xxx → fixing bug (branches from develop)
hotfix/xxx → emergency fix (branches from main)
release/xxx → preparing release (branches from develop)

Flow:

Feature started → create feature/xxx from develop
Feature done → PR to develop → code review → merge
Release ready → create release/xxx → testing → merge to main
AND develop
Bug in prod → create hotfix/xxx from main → fix → merge to
main AND develop

7.3 Pull Request Process

1. Push feature branch to remote
2. Open PR: "merge feature/xxx → develop"
3. Team reviews code (comments, suggestions)
4. CI pipeline runs automatically (tests must pass)
5. Reviewer approves
6. PR merged – feature branch deleted

Good PR:

- Small and focused (one feature or fix)
- Clear description of what and why
- Tests included
- No breaking changes
- Passes all pipeline checks

7.4 CI — Continuous Integration

Every code push automatically triggers:

1. Restore packages
2. Build solution
3. Run all tests
4. Code analysis / linting
5. Create deployment artifact

If ANY step fails → notify developer immediately → do NOT deploy
Goal: catch bugs early, ensure code is always buildable and tested

7.5 CD — Continuous Delivery / Deployment

Automatically deploy built artifact through environments:

Build Artifact

↓

Development → auto deploy, no approval needed

↓

Staging/QA → auto deploy, QA team tests manually

↓ (approval gate – manager approves)

Production → deploy to live users

Approval gates: humans must approve before going to next environment

Environment-specific configs: different secrets, connection strings per env

7.6 YAML Pipeline

Pipeline defined as code in YAML file. Lives in your git repository. Changes to pipeline are tracked in version control like regular code.

Key sections:

trigger	→ which branches trigger this pipeline
pool	→ what agent (machine) to run on
variables	→ reusable values
stages	→ top-level groupings (Build, DeployDev, DeployProd)
jobs	→ work units within a stage
steps	→ individual tasks within a job

7.7 Environment Promotion

Dev → Staging → Production

Each environment:

- Own resource group
- Own database
- Own app service
- Own secrets
- Own monitoring

Never share resources between environments

Production data never touches dev environment



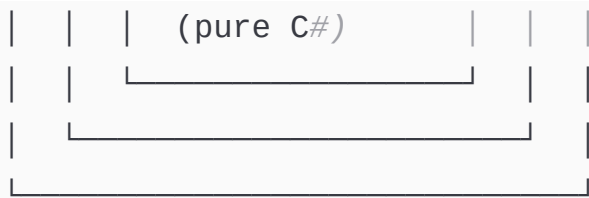
PART 8 — CLEAN ARCHITECTURE

8.1 What Is Clean Architecture

Code organized so business rules are independent of frameworks, databases, and external tools. Inner layers know nothing about outer layers.

Dependencies point INWARD only:





↑
API Layer wraps everything

8.2 Layers Explained

Domain Layer (innermost, **no** dependencies):

- Entity classes **with** business rules
- Domain exceptions
- Repository interfaces (**no** implementation)
- Pure C# – **no** EF Core, **no** ASP.NET, nothing
- This **is** the heart **of** your application

Application Layer:

- Use cases / services (orchestrate domain)
- DTOs
- Validators
- **References** Domain **only**

Infrastructure Layer:

- EF Core implementation **of** repositories
- DbContext
- **External** service implementations (email, storage)
- **References** Domain **and** Application

API Layer (outermost):

- Controllers
- Middleware

- Program.cs
- References all layers

8.3 SOLID Principles

S — Single Responsibility One class, one job, one reason to change. Don't mix concerns (business logic + email + data access in one class).

O — Open/Closed Open for extension, closed for modification. Add new features by adding new classes, not editing existing ones. Use interfaces and inheritance.

L — Liskov Substitution Child class must be substitutable for parent class without breaking behavior. If parent works somewhere, child must also work there.

I — Interface Segregation Don't force classes to implement methods they don't need. Create small, focused interfaces instead of one large interface.

D — Dependency Inversion Depend on abstractions (interfaces), not concrete classes. High-level modules shouldn't depend on low-level modules. Both should depend on abstractions.



PART 9 — SYSTEM DESIGN

9.1 Load Balancer

Distributes incoming traffic across multiple servers.

Without: all traffic → one server → server dies

With: traffic → load balancer → distributes evenly across servers

Benefits:

- No single point of failure
- Handle more traffic by adding servers
- Automatic failover if one server crashes

In Azure: App Service with multiple instances has built-in LB

9.2 Caching

Store expensive results temporarily for fast repeated access.

Without cache: every request → database → slow, expensive

With cache: first request → DB → store in cache
next 1000 requests → cache → instant, cheap

Cache types:

In-Memory (IMemoryCache):

- Fast, simple
- Lives on same server
- Lost if server restarts
- NOT shared across multiple instances

Distributed (Redis):

- Shared across all instances
- Survives server restarts
- Required for horizontally scaled apps

Cache invalidation:

- Set expiry time (absolute or sliding)
- Remove manually when data changes
- Key rule: delete cache when you update the underlying data

9.3 Scalability

Vertical Scaling (Scale Up):

Make server bigger – more CPU, more RAM
Simple, no code changes needed
Has hardware limits
Single point of failure remains
Azure: change App Service plan tier

Horizontal Scaling (Scale Out):

Add more servers (copies of your app)
Nearly unlimited scale
No single point of failure
App MUST be stateless
Azure: App Service instance count → auto-scaling

Stateless requirements for horizontal scaling:

- No session stored on server
- Use JWT (token carries user state)
- Use Redis for any shared cache
- Database is shared (not per-server)

9.4 Microservices vs Monolith

Monolith:

All features in one application

Pros: Simple to build, deploy, debug. Good for small teams.

Cons: Teams step on each other. Scale all or nothing. One crash = all down.

When: Starting out, small team, early stage

Microservices:

Separate small services communicating via HTTP or messaging

Each service: own codebase, own database, own deployment

Pros: Independent teams, deploy parts separately, scale parts independently

Cons: Complex infrastructure, network latency, distributed system problems

When: Large teams, enterprise, high scale needs, DevOps maturity

Practical path:

Start → Monolith

Grow → Modular Monolith

Scale → Microservices

9.5 API Gateway

Single entry point for all clients. Sits in front of your services.

Client → API Gateway → routes to correct service

API Gateway handles:

- Authentication (one place for all services)
- Rate limiting
- Logging and monitoring
- SSL termination
- Request routing
- Load balancing

In Azure: Azure API Management (APIM)

9.6 Message Queue

Asynchronous communication between services.

Without queue:

Order Service → directly calls Email Service

If Email Service is down → order fails

With queue:

Order Service → puts message in queue → returns success immediately

Email Service → reads from queue when ready → sends email

Services are decoupled – one going down doesn't affect other

In Azure: Azure Service Bus, Azure Queue Storage



PART 10 — PRODUCTION LEVEL

10.1 Logging Best Practices

Log **Levels** (use the right one):

- Trace → every tiny **step** (dev only, very verbose)
- Debug → useful **for debugging** (dev/staging)
- Information → normal business **events** (production on)
- Warning → unexpected but **handled** (pay attention)
- Error → something failed, needs **investigation** (alert team)
- Critical → system down, data loss **risk** (wake someone up)

Rules:

- Use structured logging: `_logger.LogInformation("User {Id} logged in", id)`
- Never concatenate strings in log messages
- Never log passwords, tokens, or sensitive data
- Include correlation/trace IDs **for** request tracking
- Log enough to debug without being verbose

10.2 Global Exception Handling

One middleware catches **ALL** unhandled exceptions

What it does:

- Log **full** exception **with** stack trace internally
- **Return** safe, **user**-friendly error message **to** client
- Include trace ID (client gives this **to** support)

- Map exception types to HTTP status codes
- NEVER expose stack trace to client in production

Register FIRST in middleware pipeline – catches everything

10.3 API Versioning

Allows you to change your API without breaking existing clients.

Old clients use: /api/v1/products (old format)
New clients use: /api/v2/products (new format)
Both work simultaneously

Approaches:

URL versioning → /api/v1/products (most common, clear)
Header versioning → Api-Version: 1.0 header
Query versioning → /api/products?version=1.0

Always version from day one – adding versioning later is painful
Never remove old versions without giving clients migration time

10.4 Performance Checklist

Code:

- Async/await everywhere – never block threads
- AsNoTracking() on all read queries
- Select() only needed columns
- Pagination – never load all records

- Cache expensive **or** frequently read data
- IQueryable – build queries before executing

Database:

- Indexes **on** filtered/joined columns
- Avoid N+1 queries – use Include()
- Use AnyAsync() instead **of** loading entity **to** check existence
- Use CountAsync() instead **of** loading all **then** counting
- Bulk operations **for** mass updates/deletes

API:

- Response compression (GZIP)
- Response caching **for** stable data
- HTTP/2 (App Service supports it)

10.5 Memory Leaks — Common Causes

Not disposing resources:

- Always use **'using'** **for** streams, connections, readers

Event handlers **not** unsubscribed:

- Implement IDisposable **and** unsubscribe **in** Dispose()

Unbounded cache:

- Use IMemoryCache **with** expiry, **not static** Dictionary

DbContext **as** Singleton:

- Always register DbContext **as** Scoped, never Singleton
- Scoped = disposed after each request

Blocking **async** code:

- `.Result` **or** `.Wait()` can cause deadlocks
- Always **await** properly

10.6 Health Checks

Endpoints:

- `/health/live` → **is** the API process alive? (**for** load balancer)
- `/health/ready` → **are all** dependencies ready? (DB, cache, vault)

Checks commonly added:

- Database connection **check**
- Key Vault accessibility
- Redis connection (if used)
- **External** API availability

Response: Healthy / Degraded / Unhealthy

Azure App Service uses health **check** endpoint **to** restart unhealthy instances

10.7 Security Checklist

Code:

- ✓ Secrets **in** Key Vault, never **in** code
- ✓ HTTPS enforced everywhere
- ✓ Security headers added
- ✓ CORS configured **for** production domains only
- ✓ Rate limiting **on** all endpoints, stricter **on** auth
- ✓ Input validation **on** all incoming data

- ✓ DTOs – never expose raw entities
- ✓ No sensitive data **in** JWT payload
- ✓ **Short** access token expiry (**60** min **or** less)
- ✓ Passwords hashed **with** BCrypt
- ✓ SQL injection: EF Core protects automatically

Monitoring:

- ✓ Alert **on error** rate spike
- ✓ Alert **on** slow response time
- ✓ Alert **on** unusual request patterns
- ✓ No stack traces **in** production responses
- ✓ Audit log **for** sensitive operations

PART 11 — REACT

11.1 What Is React

JavaScript library for building user interfaces. Declarative — you describe WHAT the UI should look like, React figures out HOW to update the DOM efficiently.

Virtual DOM:

- React keeps lightweight **copy of real** DOM **in** memory
- State changes → React updates virtual DOM
- Compares **with** previous version (diffing algorithm)
- **Only** updates **real** DOM **where** things actually changed
- Much faster than directly manipulating **real** DOM

Component model:

- UI broken **into** small reusable pieces

- Each component manages its own state
- Components combine to build complex UIs

11.2 Components

Functional component = JavaScript function that returns JSX
JSX = JavaScript + HTML-like syntax

Rules:

- Return ONE root element (or Fragment <>)
- className instead of class
- JavaScript inside {} curly braces
- Self-closing tags must close: , <input />
- Component names start with capital letter

11.3 Props

Data passed from parent to child. Read-only — child cannot modify props.

One-way data flow: parent → child (always)

Child cannot directly update parent — must call callback function passed as prop

Special prop: children → content between component tags

11.4 State (useState)

Data owned by component. When state changes, component re-renders automatically.

```
const [value, setValue] = useState(initialValue);
```

Rules:

- Never mutate **state** directly (never: `state.name = "x"`)
- Always **use** setter function: `setValue(newValue)`
- For objects: spread **and** replace: `{...obj, key: newValue}`
- For arrays: **use** `filter/map/spread`, never **push/splice** directly
- State updates are asynchronous – don't **rely on reading state right after setter**

11.5 useEffect

Run side effects after render. Fetch data, set up subscriptions, timers.

```
useEffect(() => {  
  // runs after render  
  
  return () => {  
    // cleanup – runs before next effect or on unmount  
  };  
}, [dependencies]);
```

- | | |
|-------------------------------|--|
| <code>[]</code> (empty array) | → runs ONCE when component mounts |
| <code>[value]</code> | → runs when value changes |
| no array | → runs after EVERY render (usually wrong) |

Common uses:

- Fetch data **on** page load

- Re-fetch **when** filter/search changes
- Subscribe/unsubscribe to events
- Set document title

11.6 Context API

Share data across components without prop drilling.

Problem it solves:

App has **user** state

Header needs **user** → passed down **3+** levels just **to** reach **where** needed

Solution:

- **Create** Context
- Wrap app **in** Provider
- **Any** component **reads** directly **from** context
- **No** prop drilling

Best **for**: auth state, theme, **language**, cart

Don't **overuse**: every context change re-renders all consumers

11.7 React Router

Client-side routing — navigate between pages without full page reload.

Key components:

BrowserRouter → wraps entire app

Routes → container **for** all routes

Route → maps path to component
Link → navigation (no page reload)
NavLink → link with active state styling
Navigate → programmatic redirect in JSX
useNavigate() → programmatic navigation in code
useParams() → get URL parameters (:id, :slug)
useSearchParams() → get query parameters (?page=1)

Protected routes:

- Check authentication in wrapper component
- Redirect to login if not authenticated
- Render children if authenticated

11.8 Custom Hooks

Extract and reuse stateful logic across components.

Rules:

- Must start with 'use' (*useFetch, useAuth, useLocalStorage*)
- Can use other hooks inside
- *Can't be called inside conditions or loops*

Common custom hooks:

useFetch(url) → loading/error/data state for any URL
useAuth() → login/logout/isAuthenticated
useDebounce() → delay updating value (search inputs)
useLocalStorage() → persist state in localStorage

11.9 Performance Hooks

useMemo(fn, deps):

- Caches computed **value**
- **Only** recalculates **when** dependencies change
- Use **for** expensive calculations
- Example: filtering/sorting **large** lists

useCallback(fn, deps):

- Caches **function** reference
- **Only** recreates **when** dependencies change
- Use **when** passing functions **as** props **to** child components
- Prevents unnecessary child re-renders

useRef(initialValue):

- Stores **value** that persists across renders **WITHOUT** causing re-render
- Access DOM elements directly
- Store timer IDs, previous **values**

11.10 State Management Summary

Local state (useState):

- State **only** needed **by one** component

Lifted state:

- State needed **by** siblings → lift **to** common parent

Context API:

- State needed **by** many components across tree
- Auth, theme, cart, **language**

External library (Redux, Zustand):

- Very complex state **with** many interactions
- **Large** apps **with** many developers
- Need **for time**-travel debugging
- (**Not** needed **for** most medium-sized apps)

11.11 Important Concepts

Key prop in lists: Helps React identify which items changed. Must be unique and stable. Never use array index as key when list can reorder or items can be deleted.

Controlled vs Uncontrolled inputs:

Controlled → input value driven by state (recommended)
`<input value={state} onChange={setState} />`

Uncontrolled → input manages its own value, accessed via ref
`<input ref={inputRef} />`

Component lifecycle:

Mount → component appears **in** DOM (useEffect **with** [])
Update → state **or** props change (useEffect **with** [deps])
Unmount → component removed **from** DOM (cleanup **function in** useEffect)

Re-render triggers:

State changes via setState
Props change **from** parent
Context **value** changes
Parent component re-renders

Quick Reference — What Does What

BACKEND (ASP.NET Core + EF Core):

Receive HTTP	→ Controller
Business logic	→ Service Layer
Database access	→ Repository + EF Core
Validate input	→ Data Annotations + ModelState
Security	→ JWT Middleware + [Authorize]
Configuration	→ IConfiguration + Options Pattern
Secrets	→ Azure Key Vault
Monitoring	→ Application Insights
Background jobs	→ Azure Functions (Timer Trigger)
File storage	→ Azure Blob Storage
Deployment	→ Azure App Service + DevOps Pipeline

FRONTEND (React):

Page structure	→ Components
Passed data	→ Props
Component data	→ useState
Side effects	→ useEffect
Navigate	→ React Router
Global data	→ Context API
API calls	→ fetch /axios in service layer
Cache values	→ useMemo
Cache functions	→ useCallback
DOM access	→ useRef



Production Deployment Checklist



Before going live verify:

CODE:

- ✓ No hardcoded secrets anywhere
- ✓ All secrets in Azure Key Vault
- ✓ Global exception handler registered first
- ✓ Structured logging at correct levels
- ✓ Input validation on all endpoints
- ✓ DTOs used – no raw entities exposed
- ✓ Async all the way through
- ✓ AsNoTracking on all read queries

SECURITY:

- ✓ HTTPS enforced (UseHttpsRedirection + UseHsts)
- ✓ Security headers added
- ✓ CORS set to production domains only
- ✓ Rate limiting configured
- ✓ JWT expiry 60 minutes or less
- ✓ Passwords hashed with BCrypt
- ✓ No sensitive data in logs

DATABASE:

- ✓ Retry logic for transient failures
- ✓ Indexes on common query columns
- ✓ Migrations applied
- ✓ Backup retention set (minimum 7 days)

AZURE:

- ✓ App Service plan scaled appropriately
- ✓ Auto-scaling configured
- ✓ Application Insights connected
- ✓ Alerts configured (errors + slow responses)
- ✓ Health check endpoints working
- ✓ Deployment slots configured
- ✓ Managed Identity for Key Vault access

DEVOPS:

- ✓ CI pipeline runs tests **on** every push
- ✓ Failed tests block deployment
- ✓ Environments separated (dev / staging / prod)
- ✓ Approval gate before production
- ✓ Rollback strategy ready (swap slots back)



Complete Learning Map — Everything Covered

C# Fundamentals

- └─ **Value** vs Reference Types + Stack vs Heap
- └─ Nullable Types + `??`, `?.`, `??=`
- └─ OOP: Encapsulation, Abstraction, Inheritance, Polymorphism
- └─ Interfaces vs Abstract Classes
- └─ Sealed, Virtual, Override
- └─ Delegates, Events
- └─ LINQ
- └─ Async/Await/Tasks
- └─ Exception Handling

ASP.NET Core

- └─ Request Lifecycle
- └─ Program.cs structure
- └─ MVC + Web API
- └─ REST + HTTP Verbs + Status Codes
- └─ DTOs + Validation
- └─ Middleware (custom + built-**in**)
- └─ Dependency Injection + Lifetimes
- └─ Configuration + Options **Pattern**

SQL Server

- └─ DDL + DML
- └─ **SELECT**, **WHERE**, **ORDER BY**, **TOP**, **DISTINCT**
- └─ **INNER JOIN**, **LEFT JOIN**, multiple JOINS
- └─ **GROUP BY**, **HAVING**, Aggregates
- └─ Indexes + Execution Plans
- └─ Stored Procedures
- └─ Transactions
- └─ Normalization (**1NF**, **2NF**, **3NF**)
- └─ ACID Properties

Entity Framework Core

- └─ ORM concept
- └─ DbContext + DbSet
- └─ Code **First** + Migrations
- └─ CRUD operations
- └─ Eager vs Lazy vs Explicit Loading
- └─ N+1 Problem
- └─ Tracking vs NoTracking
- └─ Performance (**Select**, Pagination, Bulk)
- └─ Repository **Pattern**

Authentication & Security

- └─ Auth vs **Authorization**
- └─ JWT Structure + Flow
- └─ Access Token + Refresh Token
- └─ OAuth **2.0**
- └─ Microsoft Entra ID
- └─ BCrypt Password Hashing
- └─ HTTPS + CORS + Rate Limiting
- └─ Key Vault + Managed **Identity**
- └─ Common Security Mistakes

Azure Cloud

- └─ Cloud Computing Benefits
- └─ Resource **Groups**

- └─ App Service + Plans + Slots
- └─ Azure **SQL** + Retry Logic
- └─ Key Vault + Managed **Identity**
- └─ **Blob** Storage
- └─ Azure Functions + Triggers
- └─ Application Insights
- └─ Azure Monitor + Alerts

DevOps & CI/CD

- └─ GitFlow Branching
- └─ Pull Requests
- └─ **Commit** Conventions
- └─ CI Pipeline (Build + Test)
- └─ CD Pipeline (Deploy)
- └─ YAML Pipelines
- └─ Environment Promotion

Clean Architecture

- └─ Layer separation (Domain, Application, Infrastructure, API)
- └─ Dependency rules (inward **only**)
- └─ SOLID (S, O, L, I, D)

System Design

- └─ Load Balancer
- └─ Caching (Memory + Redis)
- └─ Vertical vs Horizontal Scaling
- └─ Microservices vs Monolith
- └─ API Gateway
- └─ Message Queues

Production Level

- └─ Structured Logging + Levels
- └─ **Global** Exception Handling
- └─ API **Versioning**
- └─ Performance Optimization

- └─ Memory Leak Prevention
- └─ Health Checks

React

- └─ Components + JSX
- └─ Props + Children
- └─ useState (primitives, objects, arrays)
- └─ useEffect (mount, deps, cleanup)
- └─ Event Handling
- └─ Lists + Conditional Rendering
- └─ Custom Hooks
- └─ Context API
- └─ React Router
- └─ API Service Layer
- └─ useRef, useMemo, useCallback
- └─ Project Structure

This is your **complete theory reference** for everything covered. Return to any section whenever you need a quick reminder during your learning or before interviews. 🚀

23/02/2026, 13:30:58